

TÓPICOS EM QUALIDADE E TESTE DE SOFTWARE



Organizador:
Orlando Saraiva Jr

CASEMIRO SENEME JUNIOR
DAVI BONELLI RODRIGUES
GABRIEL BARBIERI
GABRIEL VICTORINO
GUSTAVO FRANCISCO HABERMANN
ISABELA BATISTA MACHADO DE ALMEIDA
JOÃO AUGUSTO SELEGATTO PACOLLA
LEANDRO ALVES DE FARIAS RODRIGUES
LUCAS FERNANDO ARANTES
LUCAS FERREIRA
MAIKON FABRÍCIO GINO
MARIA VITÓRIA SUZARTH

MATEUS AUGUSTO BRITO DE SOUSA
MIGUEL FERNANDES DO ESPIRITO SANTO
MIRAN ROMEIRO FERREIRA DA SILVA
PEDRO MIRA NICOLAU
PIETRA ALVES
RAFAELA DE JESUS BISPO DOS SANTOS
RIAN DE SOUZA SCAVAZZA
ROBERTO GUEDES DE ALMEIDA JUNIOR
THIAGO CESAR ALVAREZ
VALDIR DE JESUS GARCIA JÚNIOR
WESLEY GUSTAVO KILIAN

ISBN: 978-65-01-84647-7

Qualidade e Testes de Software: Criando-se uma cultura de qualidade

Orlando Saraiva Jr
orlando.nascimento@fatec.sp.gov.br

Este e-book apresenta as atividades realizadas no segundo semestre de 2025 na disciplina “Qualidade e Testes de Software” da Fatec Araras, ministrada de forma remota no último semestre do curso.

A qualidade de um software não é algo conquistado de forma definitiva; ela precisa ser construída diariamente. Isso exige atenção constante, disciplina e uma cultura sólida voltada para testes. Assim como uma planta depende de cuidados contínuos para se desenvolver, os testes também precisam ser praticados regularmente por toda a equipe durante o desenvolvimento. Ao incorporar testes desde o início — por meio de TDD, automação e revisões frequentes — evita-se o acúmulo de falhas ocultas que, mais tarde, podem comprometer o funcionamento do sistema como um todo. Esta é a mensagem que eu espero que meus alunos levem para suas carreiras.

Criar uma cultura de testes é semelhante a treinar para um esporte: no começo exige disciplina, entendimento das regras e prática constante, mas com o tempo, tudo se torna natural. Quando vocês enxergam os testes não como uma obrigação ou algo deixado para o final, mas como parte do próprio jogo do desenvolvimento, a qualidade deixa de ser obra do acaso e passa a refletir o empenho diário de cada um. Assim como uma equipe bem treinada alcança melhores resultados, um software testado com responsabilidade entrega valor, confiabilidade e uma vida útil muito maior para seus usuários. Tenho certeza de que vocês têm todo o potencial para evoluir nessa jornada e alcançar grandes conquistas tecnológicas.

Neste capítulo, apresento o que fizemos neste semestre, quais as novidades comparado ao semestres anteriores e aprendizados. Nos demais capítulos, cada aluno apresenta sua pesquisa e perspectiva sobre uma ferramenta aplicada na área de qualidade e testes de software.

Aprendizados

A disciplina apresenta uma série de desafios e oportunidades. Comumente, no primeiro bloco, o foco foi definir o que é qualidade, o que é qualidade de software, quais os padrões e formas de testes. Neste bloco, os conteúdos relacionados à segurança no desenvolvimento e ao ambiente de produção foram trabalhados com ênfase por meio de uma avaliação presencial realizada na unidade. Na primeira avaliação, os estudantes tiveram acesso ao código-fonte de uma aplicação de agenda desenvolvida em Laravel, disponibilizada no repositório público no GitHub <https://github.com/orlandosaraivajr/agenda-laravel>, e a um ambiente de produção configurado no

provedor DigitalOcean [DIGITALOCEAN, 2025]. Como entrega, cada aluno apresentou um relatório referente à auditoria realizada presencialmente durante a atividade.

No segundo bloco, denominado “Testes na Prática”, foi apresentada a biblioteca pytest [PYTEST, 2025], amplamente utilizada para testes automatizados na comunidade Python. Como em outros semestres, cada aluno recebeu a tarefa de pesquisar e apresentar uma ferramenta ou biblioteca de sua escolha, relacionada a tecnologia que melhor lhe agradar. Essa atividade visou não apenas expandir o repertório técnico, mas também promover o compartilhamento de conhecimentos e o debate construtivo sobre diferentes abordagens e soluções tecnológicas.

No terceiro módulo do curso, os estudantes foram convidados a selecionar um tema de sua escolha, dentro dos assuntos explorados durante as aulas, para elaborar um capítulo de um e-book coletivo. Essa etapa teve como objetivo incentivar a investigação mais aprofundada de tópicos específicos, além de documentar a atividade prática previamente realizada. A proposta previa que cada aluno produzisse um conteúdo bem organizado e de fácil compreensão. Um modelo de capítulo foi disponibilizado no início do módulo, e entregas semanais foram estabelecidas para permitir o acompanhamento contínuo por parte do professor. No primeiro dia dessa atividade, estabelecemos algumas diretrizes, entre elas a de que o desenvolvimento do capítulo aconteceria exclusivamente durante o horário de aula, de forma a não interferir nas demais demandas acadêmicas ou profissionais dos alunos.

Este e-book foi concebido com o objetivo de proporcionar ao leitor uma experiência de leitura flexível e adaptável. Cada capítulo foi elaborado de forma independente, tratando de um tema ou ferramenta específica com início, desenvolvimento e conclusão próprios. Isso garante que os conteúdos possam ser lidos separadamente, de acordo com o interesse ou a necessidade do leitor, sem depender da leitura sequencial do material.

Considerações Finais

Este e-book marca o quinto livro desenvolvido como parte da disciplina Qualidade e Testes de Software, refletindo a evolução contínua nas estratégias de ensino adotadas ao longo dos semestres.

A leitura dos quatro volumes anteriores [SARAIVA JUNIOR, Orlando, 2025] permite compreender a trajetória de aprimoramento pedagógico e metodológico da disciplina, evidenciando o compromisso em oferecer uma experiência de aprendizado mais interativa, prática e alinhada às demandas do mercado. Cada edição deste projeto é fruto de um esforço coletivo, onde a participação ativa dos alunos e o suporte institucional têm desempenhado papéis fundamentais.

Neste contexto, não podemos deixar de expressar nossa gratidão à direção da Fatec Araras e

à coordenação do curso DSM pelo incentivo constante, que viabilizam a execução dessas iniciativas.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 05 dez. 2025.

DIGITALOCEAN. . DigitalOcean, 2025. Disponível em: <https://www.digitalocean.com/>. Acesso em: 05 dez. 2025.

PYTEST. Documentation. pytest, 2025. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 20 jun. 2025.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 05 dez. 2025.

Teste em Delphi com DUnitX

Casemiro Seneme Junior

juninhoseneme@hotmail.com

casemiro.seneme@fatec.sp.gov.br

Resumo

No desenvolvimento de software, a qualidade do código é um dos pilares para garantir aplicações estáveis, seguras e fáceis de manter. Uma das boas práticas necessárias para a qualidade é o uso de testes, e isso não se apresenta diferentemente com o *Delphi*, pois quando não realizados os testes adequados, muitos projetos podem apresentar diversos erros e falhas evitáveis que prejudicam seu funcionamento.

Realizar testes, sejam eles unitários, integrados ou automatizados não é apenas uma boa prática, é um investimento que reduz custos, previne retrabalho e aumenta a confiabilidade das soluções construídas.

Agora manteremos o foco no uso específico dos testes unitários por meio do framework *DUnitX*.

Palavras-chaves: Delphi. UnitX. TDD.

Introdução

O *DUnitX* surgiu com a proposta de modernizar os testes unitários na plataforma Delphi. De acordo com o registro de uma das páginas de componentes Delphi, a versão inicial oficial do *DUnitX* foi disponibilizada em **08 de setembro de 2014** pela empresa *VSoft Technologies Pty Ltd.*

A criação do *DUnitX* foi motivada pelas limitações do framework anterior, *DUnit*, que embora tenha sido amplamente utilizado durante muitos anos, tornou-se defasado em face das evoluções da linguagem Delphi.

Enquanto o *DUnit* foi inspirado em frameworks de teste antigos, adaptados para Delphi, o *DUnitX* adota ideias de frameworks modernos de outras linguagens, como *NUnit* ou mesmo conceitos de *xUnit*. Tirando proveito dos recursos mais recentes do Delphi (como *generics*, *attributes*, *anonymous methods*, etc).

Algumas das características que destacam o *DUnitX* são a versatilidade e a facilidade de aplicação dos testes nos diferentes projetos, já que o Framework oferece suporte à diversas plataformas, execução via *Console* ou via *GUI* e por meio do uso de *attributes*, *teardown*, *fixtures* e outros métodos da flexibilidade e clareza ao código de testes.

Desde sua criação, o *DUnitX* se consolidou como o Framework recomendado para novos projetos Delphi que queiram adotar testes unitários de forma consistente, especialmente quando se deseja aproveitar recursos modernos da linguagem, garantir manutenção mais segura e facilitar automação de testes.

Desenvolvimento

Se aprofundando no DUnitX

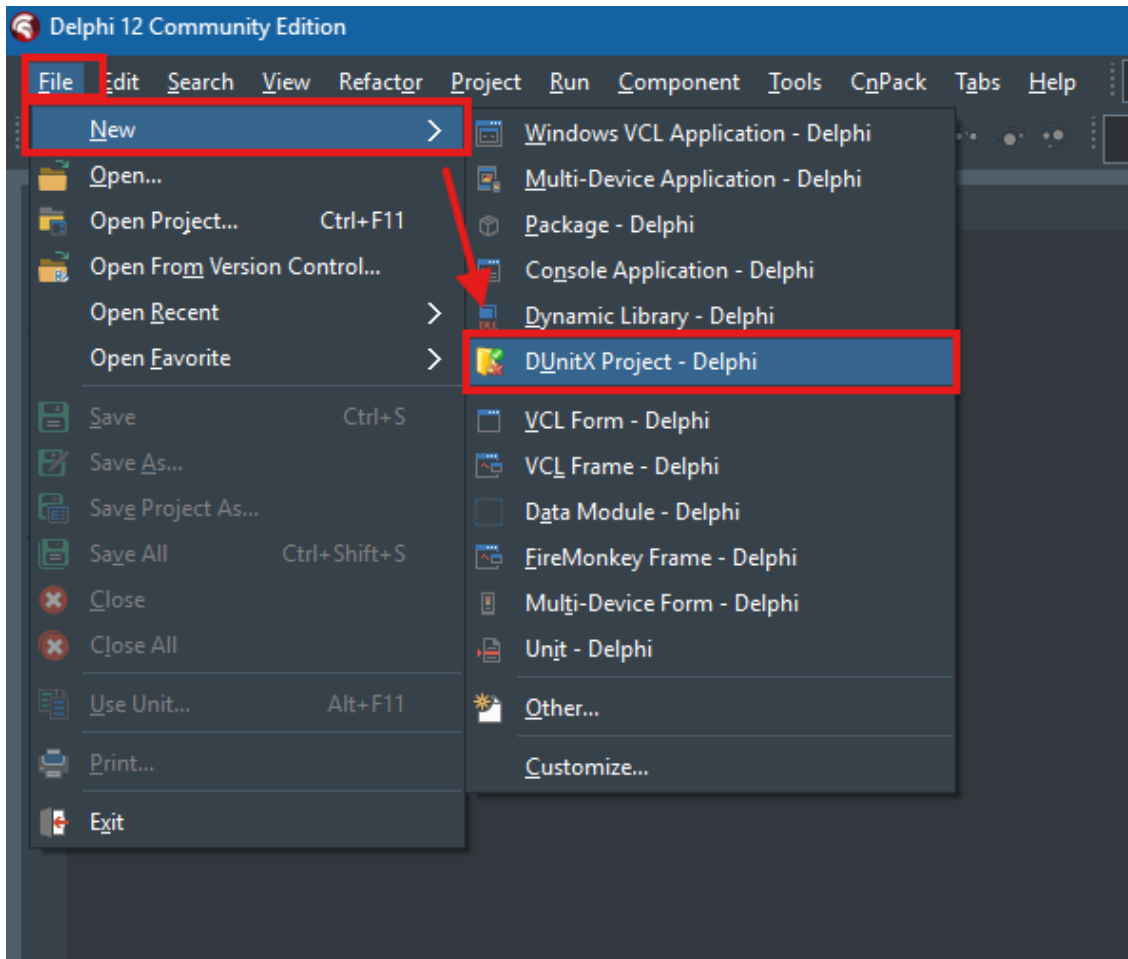
Durante a instalação do Delphi, é possível, através da tela de seleção de Features instalar o *DUnitX* junto. Assim, facilitando a integração do framework com novos projetos Delphi.

O DUnitX entra como um projeto separado. Nele, montamos um novo ambiente para cada unidade que vamos testar, por exemplo, se tivermos uma Unit responsável por realizar cálculos, você a atribui ao projeto criado.

Iniciando um projeto DUnitX

Para iniciar um novo projeto DUnitX, podemos seguir o seguinte caminho:
File > New > DUnitX Project – Delphi

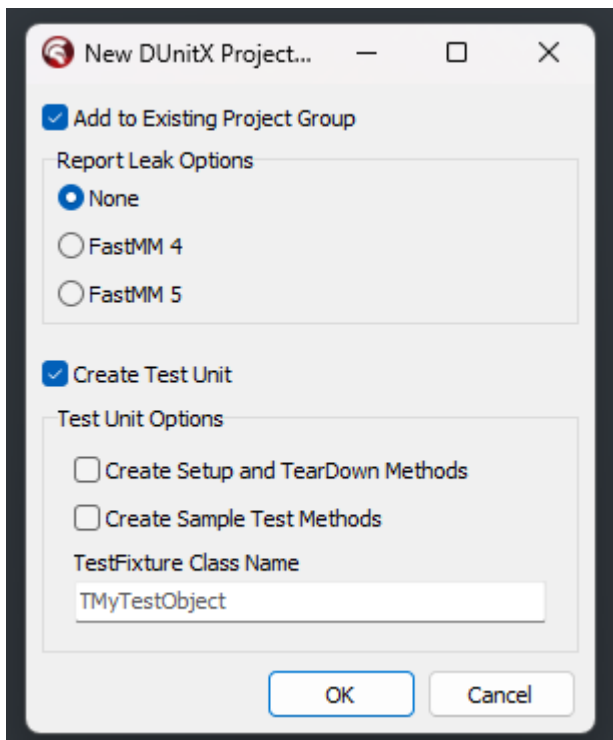
Figura 1 – Iniciando um Projeto DUnitX



Fonte: próprio autor

Após clicar em Novo projeto DUnitX, irá abrir uma tela que o Wizard criará com padrões (Figura 2). Vamos ver o que cada opção é abaixo:

Figura 2 – Opções do Wizard de Projeto.



Fonte: próprio autor

Add to Existing Project Group

Função: adiciona o novo projeto de testes ao Project Group atualmente aberto no Delphi.

Importância:

Permite trabalhar simultaneamente com a aplicação principal e o projeto de testes.

Facilita incluir units da aplicação dentro do projeto de testes sem precisar abrir projetos separados.

Melhora a organização em soluções que possuem múltiplos executáveis (app + testes + ferramentas auxiliares).

Quando usar:

Praticamente sempre, especialmente em projetos maiores ou quando há testes unitários contínuos.

Report Leak Options

Essas opções determinam como o projeto irá lidar com leaks (vazamentos de memória) durante a execução dos testes.

None

Função: não ativa nenhum mecanismo adicional de detecção de vazamentos de memória.

Consequência:

O projeto de testes não alertará se objetos forem criados e não liberados.

Utiliza apenas a detecção básica do Delphi, que geralmente não mostra relatórios detalhados.

Indicado para:

ambientes simples

quem ainda não quer analisar leaks

projetos acadêmicos ou protótipos

FastMM 4

Função: configura o projeto para usar o gerenciador de memória FastMM 4, amplamente utilizado no Delphi.

Benefícios:

Exibe relatórios completos sobre vazamentos de memória.

Permite rastrear onde um objeto foi criado e não destruído.

Ajuda a identificar ciclos, má gestão de ponteiros e objetos não liberados.

Quando escolher:

em projetos legados (Delphi 2010–XE8)

quando se precisa de compatibilidade com bibliotecas antigas

FastMM 5

Função: ativa o uso do FastMM 5, a versão mais recente e otimizada do memory manager.

Benefícios:

Melhor desempenho geral.

Melhor precisão na detecção de leaks.

Compatível com aplicações modernas, 64 bits e com uso pesado de objetos.

Quando escolher:

em projetos modernos

quando o desenvolvedor quer análises mais profundas de memória

quando já se utiliza FastMM 5 no projeto principal

Create Test Unit

- **Função:** instrui o Wizard a criar automaticamente uma unit de teste padrão (como MyTestObject.pas).
- **Conteúdo criado:**

- classe anotada com [TestFixture], permitindo que o DUnitX detecte testes automaticamente
- métodos padrão para testes
- bloco de registro da fixture no initialization
- **Vantagem:**
 - evita erros de estrutura
 - cria imediatamente um “esqueleto” funcional
 - ideal para começar rapidamente

Create Setup and TearDown Methods

- **Função:** gera automaticamente dois métodos especiais:
 - [Setup] → executado antes de cada teste
 - [TearDown] → executado depois de cada teste
- **Uso prático:**
 - inicializar objetos ou conexões antes de testar
 - liberar recursos e limpar ambiente após cada método
- **Exemplo de uso:**
 - criar uma classe para teste no Setup
 - destruir no TearDown
- **Por que é importante:**
 - garante isolamento entre testes
 - evita que um teste influencie o outro

Create Sample Test Methods

- **Função:** inclui automaticamente métodos de teste de exemplo com a marcação [Test].
- **O que esses exemplos trazem:**
 - uma estrutura de teste já completa
 - uso de Assert (como Assert.AreEqual, Assert.Pass, etc.)
 - demonstração da anatomia de um método de teste
- **Indicado para:**
 - quem está aprendendo a usar DUnitX

- projetos novos sem padrões de teste definidos
- **Vantagem:**
 - serve como referência inicial
 - reduz curva de aprendizado

TestFixture Class Name

- **Função:** define o nome da classe principal de testes a ser gerada.
- **Por que é importante:**
 - o nome aparece nos relatórios do DUnitX
 - facilita organizar testes por finalidade (ex.: TPedidoTests, TClienteTests, TCalcularIMCTests)
- **Boa prática:**
 - nomear seguindo objetivo da classe testada
 - começar com T e terminar com Tests

Criando o primeiro Teste

Após configurar o projeto DUnitX, o próximo passo é criar o primeiro teste unitário. O DUnitX utiliza *fixtures* para organizar e identificar testes dentro da aplicação.

A estrutura básica de uma unit de teste fica semelhante a esta:

Figura 3 – Exemplo de estrutura.

```
1  unit Unit1;
2
3  interface
4
5  uses
6      DUnitX.TestFramework;
7
8  type
9      [TestFixture]
10     TMyTestObject = class
11     public
12     end;
13
14 implementation
15
16 initialization
17     TUnitX.RegisterTestFixture(TMyTestObject);
18
19 end.
20
```

Fonte: próprio autor

O que essa estrutura significa:

- **[TestFixture]**
 - Indica ao DUnitX que essa classe contém testes e deve ser escaneada pelo framework.
- **[Test]**
 - Marca métodos que serão executados como testes.
- **[TestCase]**
 - Permite criar múltiplos cenários diferentes para um único método, alimentando valores por parâmetro.
- **Métodos de teste (ex.: TestIMCFeminino)**
 - São métodos comuns, mas o framework executa todos eles automaticamente.
- **Registro da *fixture***
 - Ao final da unit:

Figura 4 – Registro da fixture

```
1  initialization
2      TUnitX.RegisterTestFixture(TMyTestObject);
```

Fonte: próprio autor

Essa linha garante que o DUnitX reconheça seus testes ao executar o projeto.

Como estruturar os testes

Testes bem estruturados facilitam manutenção, legibilidade e evolução do código. Em testes unitários, é comum seguir o padrão AAA:

Arrange – Preparar o cenário

Criar variáveis, objetos e configurar o ambiente necessário para o teste.

Act – Executar a ação

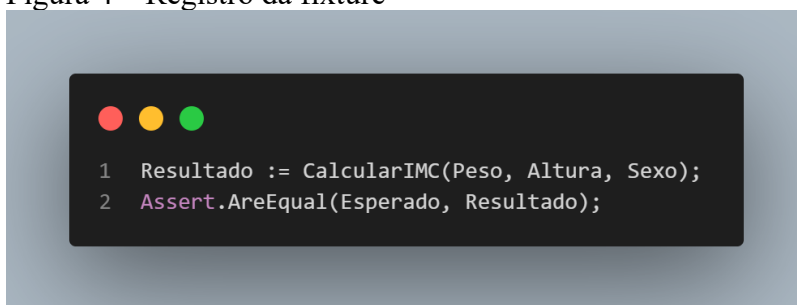
Chamar a função ou método que se deseja testar.

Assert – Verificar o resultado

Comparar retorno e validar comportamento esperado.

Na seguinte imagem, segue um exemplo de teste de IMC:

Figura 4 – Registro da fixture



Fonte: próprio autor

Boas práticas de estrutura:

- Testes devem ser independentes, sem depender da execução de outros.
- Testes devem ser curtos e testar apenas um comportamento por vez.
- Nomear testes de forma descritiva, como:
 - *TestIMC_DeveClassificarMulherAbaixoDoPeso*
- Evitar testar interfaces gráficas.
- Testar apenas lógica, regras de negócio e comportamentos isolados.

Executando os testes

O DUnitX cria um projeto do tipo console, responsável por localizar e executar todos os testes registrados.

Ao rodar o projeto (F9), você verá algo assim no console:

Figura 5 – Exemplo de testes no console

```
*****
*      DUnitX - (c) 2015-2020 Vincent Parrett & Contributors      *
*                                                                    *
*      License - http://www.apache.org/licenses/LICENSE-2.0      *
*****

Fixture : TestUnit
-----
Fixture : TestUnit.TCalcularIMTests
-----
Test : TestUnit.TCalcularIMTests.TestIMCFeminino.MulherAbaixoPeso
-----
Running Setup for : TestIMCFeminino.MulherAbaixoPeso

Executing Test : TestIMCFeminino.MulherAbaixoPeso

Running Teardown for Test : TestIMCFeminino.MulherAbaixoPeso

Success.

Test : TestUnit.TCalcularIMTests.TestIMCFeminino.MulherPesoNormal
-----
Running Setup for : TestIMCFeminino.MulherPesoNormal

Executing Test : TestIMCFeminino.MulherPesoNormal

Running Teardown for Test : TestIMCFeminino.MulherPesoNormal
```

Fonte: próprio autor

Dicas ao executar:

- Confirme que o projeto ativo é o *projeto de testes*.
- O DUnitX faz *scan* de todas as fixtures automaticamente.
- A saída XML pode ser usada em pipelines CI/CD.
- O console pausa no final para análise dos resultados.

Testes parametrizados

O DUnitX permite testar múltiplos cenários com o mesmo método por meio de [TestCase].

Exemplo:

Figura 6 – Exemplo de teste de métodos parametrizados

```
[Test]
[TestCase('MulherAbaixoPeso', '41.9,1.70,F,abaixo do peso')]
[TestCase('MulherPesoNormal', '64.9,1.70,F,no peso normal')]
[TestCase('MulherMarginalAcima', '76.7,1.70,F,marginalmente acima do peso')]
[TestCase('MulherAcimaIdeal', '86.1,1.70,F,acima do peso ideal')]
[TestCase('MulherObesa', '104.5,1.70,F,obeso')]
procedure TestIMCFeminino(const Peso, Altura: Double; const Sexo, Esperado: String);

[Test]
[TestCase('HomemAbaixoPeso', '49.6,1.80,M,abaixo do peso')]
[TestCase('HomemPesoNormal', '76.3,1.80,M,no peso normal')]
[TestCase('HomemMarginalAcima', '87.8,1.80,M,marginalmente acima do peso')]
[TestCase('HomemAcimaIdeal', '95.4,1.80,M,acima do peso ideal')]
[TestCase('HomemObeso', '115.2,1.80,M,obeso')]
procedure TestIMCMasculino(const Peso, Altura: Double; const Sexo, Esperado: String);
```

Fonte: próprio autor

O que cada TestCase faz?

Ele executa o mesmo método várias vezes, passando parâmetros diferentes:

- Peso
- Altura
- Sexo
- Resultado esperado

Vantagens:

- Diminui repetição
- Teste mais cenários com menos código
- Clareza sobre comportamentos esperados

Por que isso é bom?

Seu teste não precisa criar 5 métodos diferentes para 5 faixas de IMC. Basta repetir o atributo.

Considerações Finais

A adoção do DUnitX no desenvolvimento Delphi moderniza o fluxo de testes e melhora a qualidade geral da aplicação.

Com testes unitários, sua equipe ganha:

- mais segurança ao refatorar,
- menos retrabalho,
- código mais limpo,
- menos bugs em produção.

Bibliografia

Testes com DUnitX – Como implementar e como aplicar (Carlos Agnes).

Disponível em: <https://www.youtube.com/watch?v=mhoX-1eCqgM>. Acesso em: 02 dez. 2025.

DEV MEDIA. Testes unitários com DUnitX.

Disponível em: <https://www.devmedia.com.br/testes-unitarios-com-dunitx/36738>.

Acesso em: 02 dez. 2025.

Testes Automatizados Aplicados ao Cadastro de Livros com Ruby on Rails

Beatriz Machado de Oliveira
beatrizm.oli0102@gmail.com
beatriz.oliveira79@fatec.sp.gov.br

Resumo

Este trabalho apresenta o processo de implementação e validação de testes automatizados em uma aplicação Ruby on Rails desenvolvida para o cadastro de livros. O sistema possui como funcionalidades principais o registro de título, autor, categoria e editora. Para garantir a qualidade da aplicação, foram utilizadas diferentes abordagens de testes, incluindo testes de modelo, testes de sistema e geração automatizada de dados. As ferramentas adotadas compreendem RSpec para criação e execução dos testes, FactoryBot para construção de objetos de teste, Faker para geração de dados fictícios e Capybara para simulação de interações reais do usuário. Foram realizados testes unitários que validam a integridade dos modelos, além de testes de interface que garantem o correto funcionamento do fluxo de criação de livros. Os resultados demonstraram que a aplicação corresponde às regras de negócio definidas e que o ambiente de testes contribuiu significativamente para a prevenção de falhas. O trabalho evidencia a importância dos testes automatizados na garantia da estabilidade e confiabilidade de sistemas web, apresentando uma abordagem prática e acessível ao processo de QA.

Palavras-chaves: Ruby on Rails. Testes Automatizados. RSpec. Capybara. FactoryBot. Qualidade. TDD

Introdução

O desenvolvimento de aplicações web exige práticas que garantam a confiabilidade do software. Nesse contexto, os testes automatizados desempenham papel essencial ao verificar se cada funcionalidade opera corretamente. Este trabalho apresenta um estudo aplicado a uma aplicação Ruby on Rails voltada ao gerenciamento de livros. O

objetivo principal foi estruturar um conjunto sólido de testes que avaliam tanto o comportamento interno dos modelos quanto a experiência do usuário. Foram utilizadas ferramentas como RSpec, FactoryBot, Faker e Capybara para garantir a eficácia dos testes.

Desenvolvimento

A aplicação possui como funcionalidade principal o cadastro, edição e exclusão de livros, contendo título, autor, categoria e editora. Para validar o comportamento da aplicação, foram desenvolvidos testes de modelo e sistema utilizando RSpec.

Figura 1 – Teste de modelo utilizando RSpec

```
RSpec.describe Book, type: :model do
  it 'é válido com todos os atributos preenchidos' do
    book = Book.new(title: 'Dom Casmurro', author: 'Machado', publisher: 'Record', category: 'Romance')
    expect(book).to be_valid
  end
end
```

Fonte: próprio autor

Os testes de modelo verificam se as regras de validação são aplicadas corretamente, garantindo que somente livros completos sejam considerados válidos.

Figura 2 – Teste de sistema simulando cadastro de livro

```
RSpec.describe "Gerenciamento de Livros", type: :system do
  before do
    driven_by(:rack_test)
  end

  it "permite cadastrar um novo livro com sucesso" do
    visit root_path

    click_on "Cadastrar Novo Livro"

    fill_in "Título", with: "Ruby on Rails para Iniciantes"
    fill_in "Categoria", with: "Programação"
    fill_in "Autor", with: "Beatriz Oliveira"
    fill_in "Editora", with: "TechBooks"

    click_on "Cadastrar Livro"

    expect(page).to have_content("Livro criado com sucesso!")
  end
end
```

Fonte: próprio autor

Os testes de sistema com Capybara simulam o comportamento do usuário, validando o fluxo de cadastro e a navegação na aplicação.

Figura 3 – Uso de FactoryBot e Faker

```
spec > factories > posts.rb
1  FactoryBot.define do
2    factory :post do
3      title { Faker::Book.title }
4      category { Faker::Lorem.sentence }
5      author { Faker::Book.author }
6      publisher { Faker::Book.publisher }
7    end
8  end
```

Fonte: próprio autor

FactoryBot e Faker foram utilizados para a criação dinâmica de dados durante os testes, tornando o processo mais eficiente e flexível.

Considerações Finais

O uso de testes automatizados se mostrou fundamental para garantir a qualidade da aplicação. A combinação das ferramentas RSpec, FactoryBot, Faker e Capybara possibilitou validar regras de negócio e comportamentos essenciais. Como trabalho futuro, é possível expandir o conjunto de testes para edição, listagem e exclusão de livros.

Bibliografia

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software.

RUBY ON RAILS. Documentação. Disponível em: <https://rubyonrails.org/>

RSPEC. Documentação. Disponível em: <https://rspec.info/>

CAPYBARA. Documentação. Disponível em: <https://teamcopybara.github.io/capybara/>

A ferramenta Flutter Test

Gabriel Barbieri

barbierigabriel055@gmail.com

gabriel.barbieri@fatec.sp.gov.br

Resumo

Este trabalho apresenta o estudo da ferramenta Flutter Test, utilizada para a criação e execução de testes automatizados em projetos Flutter desenvolvidos com a linguagem Dart. A análise foi realizada a partir de um projeto Flutter já estruturado, no qual a ferramenta foi aplicada para validar comportamentos, widgets e funções internas da aplicação. O objetivo foi compreender como o Flutter Test contribui para a garantia da qualidade do software, oferecendo suporte para a detecção precoce de erros e para a manutenção contínua do código. A metodologia adotada consistiu na exploração da estrutura do projeto, interpretação dos testes presentes e investigação das funcionalidades da ferramenta, destacando seus principais comandos e padrões de uso. Os resultados obtidos mostraram que o Flutter Test é eficiente para validar a lógica da aplicação e garantir estabilidade durante o desenvolvimento, permitindo maior segurança em atualizações e incrementos de funcionalidades. Conclui-se que o uso adequado da ferramenta desempenha papel essencial na qualidade e confiabilidade do software produzido.

Palavras-chaves: Flutter Test. Dart. Qualidade de Software. Testes Automatizados.

Introdução

O desenvolvimento mobile moderno exige ferramentas que assegurem a confiabilidade e o bom funcionamento das aplicações. Dentro do ecossistema Flutter, o **Flutter Test** destaca-se como a solução oficial para executar testes automatizados, permitindo avaliar desde funções isoladas até a renderização de widgets completos.

A escolha da ferramenta como tema deste capítulo se justifica pela crescente necessidade de garantir qualidade em aplicações multiplataforma, especialmente em ambientes onde mudanças frequentes podem introduzir falhas indesejadas. Com o Flutter Test, é possível validar continuamente o comportamento esperado da aplicação, reduzindo riscos e aumentando a confiabilidade do produto

final.

O objetivo geral deste trabalho é apresentar a ferramenta Flutter Test e demonstrar sua aplicação prática dentro de um projeto Flutter real. Entre os objetivos específicos estão: analisar como a estrutura do Flutter favorece a criação de testes, compreender os principais comandos e funções utilizadas e discutir os benefícios da automatização no processo de desenvolvimento.

A metodologia adotada incluiu a observação da organização do projeto, exploração dos arquivos de teste, análise da execução dos scripts e comparação com práticas recomendadas de testes de software. Este capítulo está organizado em três partes: apresentação da ferramenta e sua estrutura; detalhamento do seu uso no projeto estudado; e, por fim, as considerações finais baseadas nos resultados obtidos.

Desenvolvimento

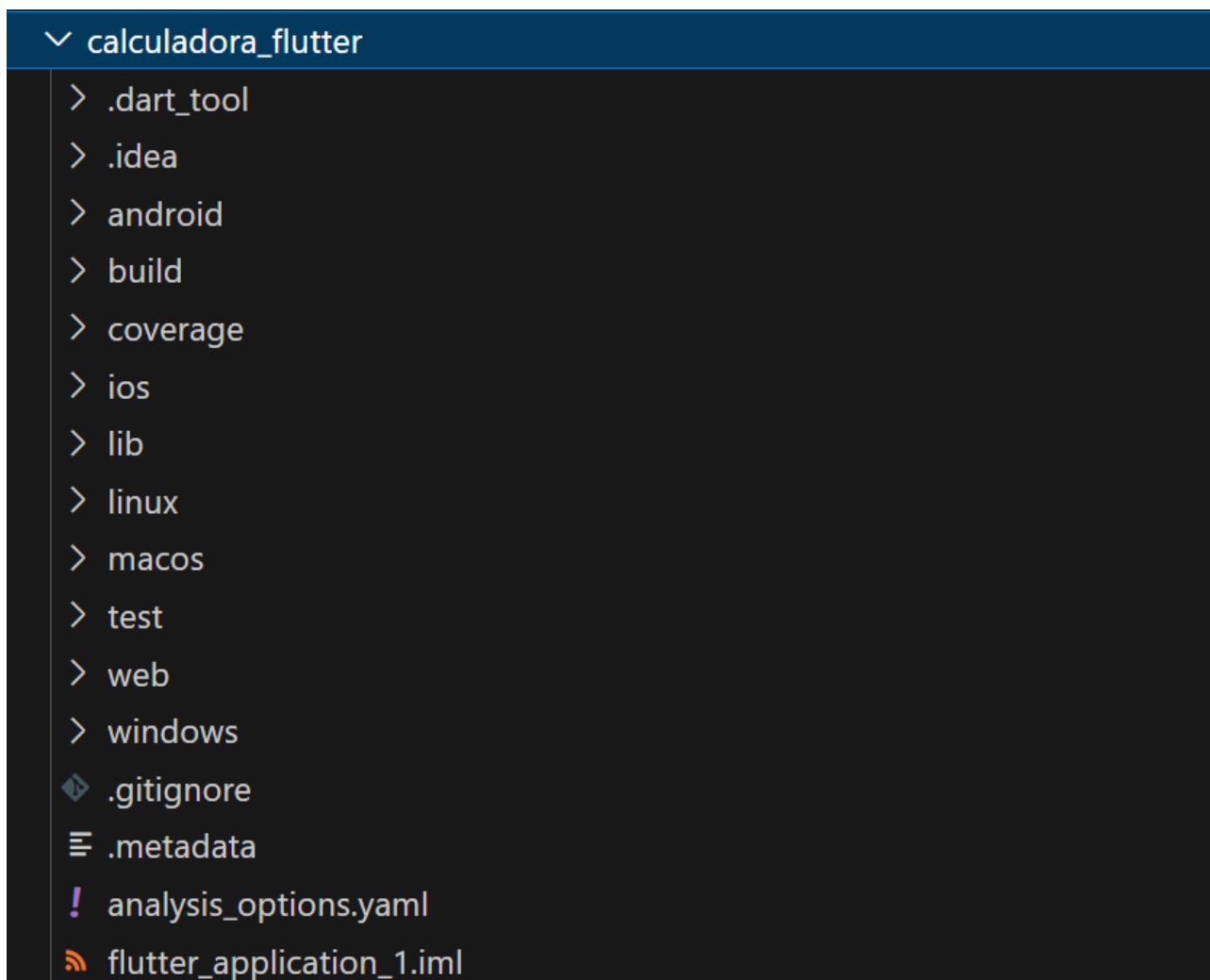
Estrutura do projeto

A seguir, descrevem-se os principais diretórios identificados:

- **.dart_tool/** — armazena arquivos internos utilizados pelo SDK do Dart, como cache, metadados e configurações automáticas de ferramentas.
- **.idea/** — contém configurações do ambiente de desenvolvimento IntelliJ/Android Studio.
- **android/** — reúne os arquivos necessários para compilar e executar o aplicativo em dispositivos Android.
- **build/** — diretório gerado automaticamente durante os processos de compilação; inclui artefatos temporários e resultados das builds.
- **coverage/** — pasta utilizada para armazenar relatórios de cobertura de testes, quando gerados.
- **ios/** — reúne arquivos específicos para execução em dispositivos iOS.
- **lib/** — diretório principal da aplicação, onde se encontram arquivos em Dart responsáveis pela lógica, interfaces e widgets do projeto.
- **linux/** — inclui arquivos e dependências para execução do aplicativo em sistemas Linux.

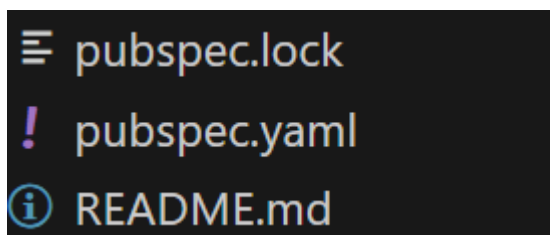
- **macos/** — contém os elementos necessários para execução no macOS.
- **test/** — pasta destinada exclusivamente aos arquivos de teste automatizado utilizando a ferramenta Flutter Test.
- **web/** — reúne recursos e configurações para execução da aplicação em navegadores web.
- **windows/** — pasta correspondente à plataforma Windows, contendo arquivos específicos para essa versão.
- **.gitignore** — define arquivos e diretórios a serem ignorados pelo sistema de controle de versão Git.
- **.metadata** — arquivo interno gerado pelo Flutter com informações sobre o ambiente do projeto.
- **analysis_options.yaml** — configura regras de lint para padronizar o estilo do código.
- **flutter_application_1.iml** — arquivo de configuração do projeto para IDEs baseadas em IntelliJ.
- **pubspec.lock** — registra versões exatas das dependências instaladas.
- **pubspec.yaml** — controla dependências e configurações gerais do aplicativo.
- **README.md** — arquivo de documentação inicial do projeto.
- Essa organização facilita a manutenção, o desenvolvimento multiplataforma e a criação de testes, garantindo que o projeto permaneça modular e padronizado.

Figura 1 – Estrutura de diretórios do projeto Flutter



Fonte: próprio autor

Figura 1.1 – Estrutura de diretórios do projeto Flutter



Fonte: próprio autor

Essa estrutura organizada facilita a aplicação de testes, permitindo separar claramente o código da aplicação e o código responsável pela validação da qualidade.

Análise dos Arquivos de Teste

O diretório test/ contém arquivos que utilizam essas funções para validar o comportamento da aplicação. Nesse projeto, a ferramenta é usada para verificar a correta execução de funcionalidades internas e o comportamento de widgets exibidos na interface.

- Um exemplo típico de teste unitário no Flutter Test utiliza a estrutura:

```
test('descrição do teste', () {  
  // lógica que será validada  
  expect(resultadoObtido, resultadoEsperado);  
});
```

- E um exemplo de teste de widget:

```
testWidgets('descrição do teste de widget', (tester) async {  
  await tester.pumpWidget(MeuWidget());  
  expect(find.text('Valor'), findsOneWidget);  
});
```

Durante a execução, o Flutter Test gera relatórios claros, exibindo:

- quantidade total de testes;
- número de testes aprovados;
- falhas e suas mensagens detalhadas;
- tempo de execução.

Figura 2 – Todos os testes

```
import 'package:flutter_test/flutter_test.dart';
import 'package:flutter_calc_app/calculator.dart';
```

Run | Debug

```
void main() {
  Run | Debug
  test("Soma funciona", () {
    final calc = Calculator();
    calc.input("2");
    calc.input("+");
    calc.input("3");
    expect(calc.input("="), "5.0");
  });
```

Run | Debug

```
test("Subtração funciona", () {
  final calc = Calculator();
  calc.input("9");
  calc.input("-");
  calc.input("4");
  expect(calc.input("="), "5.0");
});
```

Fonte: próprio autor

Figura 2.1 – Todos os testes

Run | Debug

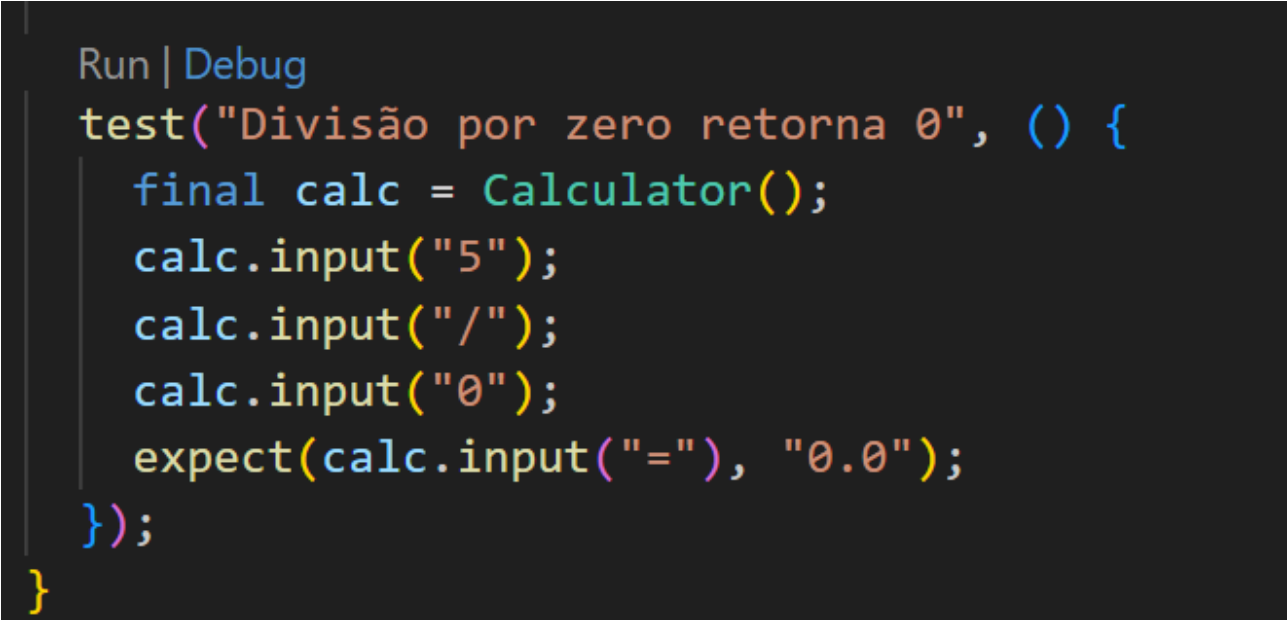
```
test("Multiplicação funciona", () {  
    final calc = Calculator();  
    calc.input("6");  
    calc.input("*");  
    calc.input("7");  
    expect(calc.input("="), "42.0");  
});
```

Run | Debug

```
test("Divisão funciona", () {  
    final calc = Calculator();  
    calc.input("8");  
    calc.input("/");  
    calc.input("2");  
    expect(calc.input("="), "4.0");  
});
```

Fonte: próprio autor

Figura 2.2 – Todos os testes

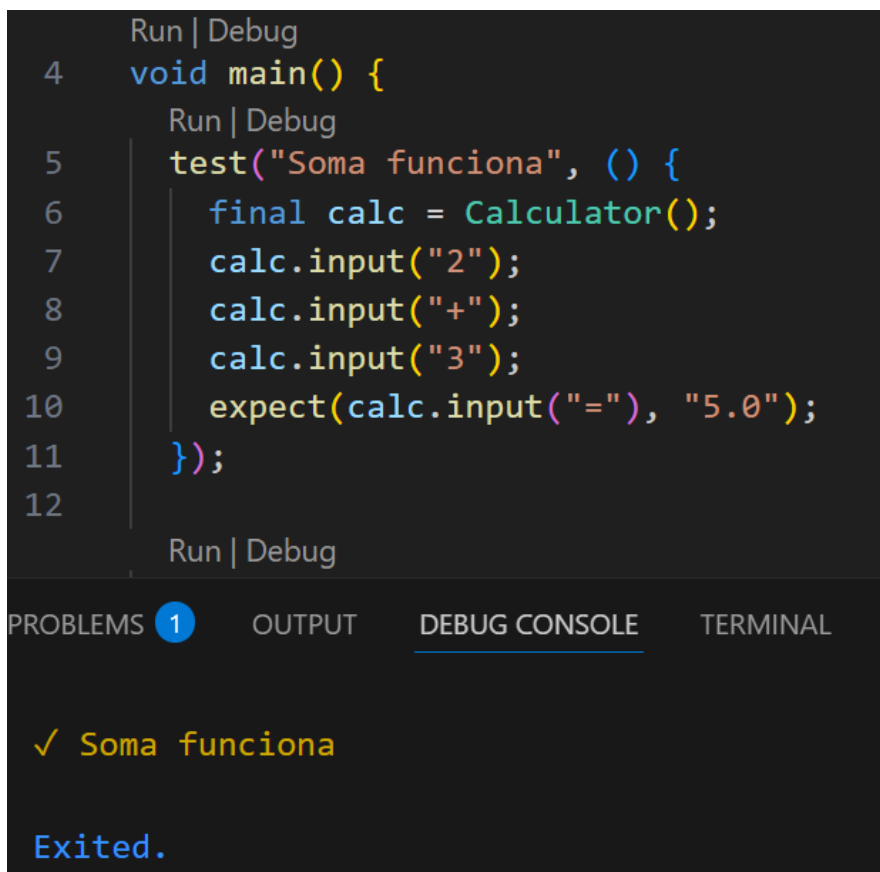
A screenshot of a code editor with a dark background. At the top left, there are two buttons: 'Run' and 'Debug'. Below them is a line of code: `test("Divisão por zero retorna 0", () {`. This is followed by an indented block of code: `final calc = Calculator();`, `calc.input("5");`, `calc.input("/");`, `calc.input("0");`, and `expect(calc.input("="), "0.0");`. The block ends with `});` and a closing curly brace `}`.

```
Run | Debug
test("Divisão por zero retorna 0", () {
  final calc = Calculator();
  calc.input("5");
  calc.input("/");
  calc.input("0");
  expect(calc.input("="), "0.0");
});
}
```

Fonte: próprio autor

As Figuras acima apresentam o conjunto de testes unitários criados para validar as principais operações matemáticas implementadas na classe `Calculator`. Cada teste avalia uma funcionalidade específica, garantindo que a lógica da calculadora retorne os valores corretos de acordo com as entradas fornecidas. O primeiro teste verifica a operação de soma, seguido pelos testes de subtração, multiplicação e divisão, cada um analisando se o resultado produzido corresponde ao valor esperado. Além disso, foi incluído um teste dedicado ao tratamento da divisão por zero, situação comum que pode gerar erros em aplicações caso não seja tratada adequadamente. Ao comparar o valor retornado com o esperado, esses testes asseguram que a calculadora responde corretamente tanto a casos regulares quanto a cenários excepcionais. A execução desse conjunto de testes contribui diretamente para a garantia de qualidade do software, permitindo identificar e corrigir falhas na lógica antes da disponibilização da aplicação.

Figura 3 – Exemplo de execução do teste de soma



```
Run | Debug
4 void main() {
    Run | Debug
5 test("Soma funciona", () {
6     final calc = Calculator();
7     calc.input("2");
8     calc.input("+");
9     calc.input("3");
10    expect(calc.input("="), "5.0");
11 });
12
Run | Debug
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

✓ Soma funciona

Exited.

Fonte: próprio autor

A Figura 3 apresenta a execução de um teste automatizado criado para validar a operação de soma dentro da aplicação. No exemplo, a classe Calculator recebe uma sequência de entradas que simulam o comportamento real do usuário ao utilizar a calculadora. O teste verifica se, ao inserir “2”, o operador “+” e posteriormente “3”, o resultado retornado pelo comando “=” corresponde ao valor esperado, que é “5.0”. A saída exibida no console confirma que o teste foi executado com sucesso, garantindo que a funcionalidade está implementada corretamente e funcionando de acordo com o esperado.

Figura 4 – Simulação de erro do teste soma

```
Expected: '6.0'
Actual: '5.0'
Which: is different.
      Expected: 6.0
      Actual: 5.0
            ^
      Differ at offset 0
package:matcher                                     expect
package:flutter_test/src/widget_tester.dart 474:18 expect
test\calculator_test.dart 10:5                      main.<fn>
X Soma funciona

Exited (1).
```

Fonte: próprio autor

A Figura 4 apresenta o resultado de um teste automatizado que falhou durante a validação da operação de soma. Nesse caso, o valor esperado pelo teste era “6.0”, porém a aplicação retornou “5.0”, indicando uma inconsistência no funcionamento do cálculo. A mensagem exibida no console detalha a diferença entre o valor esperado e o valor obtido, auxiliando na identificação precisa do erro. Esse tipo de retorno é fundamental no processo de testes automatizados, pois permite que o desenvolvedor detecte rapidamente falhas lógicas, revise o comportamento da função e aplique correções antes que o defeito impacte a aplicação final.

Figura 5 – Teste simples de widget

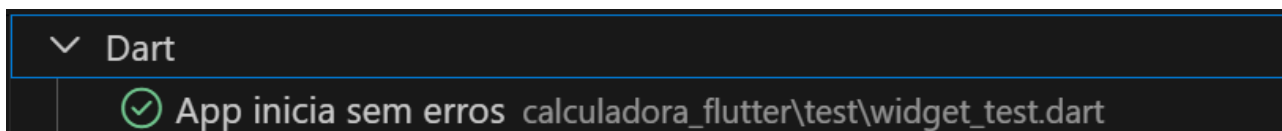
```
import 'package:flutter_test/flutter_test.dart';

Run | Debug
void main() {
  Run | Debug
  testWidgets("App inicia sem erros", (WidgetTester tester) async {
    expect(true, true);
  });
}
```

Fonte: próprio autor

A Figura 5 apresenta um exemplo de teste de widget criado para verificar se a aplicação inicializa corretamente sem lançar erros durante o processo de renderização. O teste utiliza a função `testWidgets`, que permite simular o ambiente gráfico do Flutter e validar o comportamento dos widgets de forma controlada. No exemplo, o teste simplesmente compara dois valores verdadeiros, servindo como uma verificação básica para garantir que o ambiente de testes esteja configurado e funcionando adequadamente.

Figura 5.1 – Resultado do teste de widget



Fonte: próprio autor

A Figura 5.1 mostra o resultado da execução desse teste, indicando sucesso por meio do ícone verde exibido na interface da ferramenta de testes. Essa confirmação visual demonstra que o aplicativo pôde ser carregado e analisado pelo mecanismo de widgets sem apresentar falhas, validando assim a integridade mínima da aplicação. Testes desse tipo são importantes no início do desenvolvimento, pois ajudam a assegurar que o ambiente está estável e que a estrutura inicial do projeto está correta antes da implementação de casos de teste mais complexos.

Figura 6 – Execução de todos os testes

```
PS C:\Users\GERSON BARBIERI\Downloads\AAAAAA flutter_calc_app\calculadora_flutter> flutter test
00:03 +6: All tests passed!
```

Fonte: próprio autor

A Figura 6 apresenta o resultado da execução completa da suíte de testes da aplicação, realizada por meio do comando flutter test. O relatório exibido no terminal indica que todos os seis testes foram executados com sucesso, conforme destacado pela mensagem “All tests passed!”. O tempo total de execução, mostrado ao lado do contador, demonstra a eficiência do processo de validação automatizada. Esse tipo de resultado confirma que todas as operações implementadas na calculadora — incluindo soma, subtração, multiplicação, divisão e tratamento de divisão por zero — estão funcionando de acordo com o esperado. A aprovação total dos testes reforça a confiabilidade da aplicação e evidencia a importância das rotinas automatizadas como parte do processo de garantia de qualidade.

Comandos utilizados

import	Importa bibliotecas e arquivos necessários
void main()	Inicia o arquivo de testes
test()	Cria testes unitários
testWidgets()	Cria testes de widgets (interface)
Calculator()	Instancia a calculadora
calc.input()	Simula entradas do usuário
expect()	Valida o resultado dos testes
flutter test	Executa todos os testes do projeto

Considerações Finais

O estudo demonstrou que a ferramenta **Flutter Test** é essencial para garantir a qualidade e a estabilidade de aplicações Flutter. Por meio dela, é possível validar funções internas, widgets e fluxos completos da aplicação, garantindo que cada parte do código funcione como esperado. A aplicação dos testes no projeto analisado mostrou que a ferramenta auxilia diretamente na prevenção de erros, melhora o processo de manutenção e possibilita maior segurança ao implementar novas funcionalidades.

Conclui-se que o Flutter Test oferece uma base sólida para práticas de desenvolvimento guiadas por qualidade, sendo recomendado para qualquer aplicação Flutter que vise confiabilidade, profissionalismo e escalabilidade. Trabalhos futuros podem explorar testes de integração, mocks, simulações de rede e automação contínua por meio de pipelines de CI/CD.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:
<https://fatecararas.cps.sp.gov.br/>. Acesso em: 20 nov. 2025.

DART & FLUTTER. Documentação oficial de testes. Disponível em:
<https://docs.flutter.dev/cookbook/testing>. Acesso em: 20 nov. 2025.

Pytest: A Ferramenta Essencial para Testes em Python

Lucas Ferreira

lucas.ferreira514@outlook.com

lucas.ferreira111@fatec.sp.gov.br

Resumo

Este capítulo de e-book tem como objetivo apresentar o Pytest como a ferramenta de eleição para a escrita de testes de software na linguagem Python. A metodologia empregada baseia-se na análise da documentação oficial e das melhores práticas da comunidade para delinear as funcionalidades centrais do framework. Os resultados demonstram que o Pytest simplifica a sintaxe de testes, aumenta a legibilidade do código e oferece recursos avançados como fixtures, parametrização e plugins, essenciais para a construção de uma suíte de testes robusta e escalável. A adoção do Pytest é justificada pela sua capacidade de integrar-se eficientemente a fluxos de trabalho de Desenvolvimento Orientado a Testes (TDD), elevando o padrão de qualidade e a manutenibilidade de projetos de software. A conclusão reforça a necessidade de domínio desta ferramenta para qualquer desenvolvedor Python que almeje excelência em engenharia de software.

Palavras-chaves: Pytest. Python. Testes de Software. TDD. Qualidade.

Introdução

A qualidade do software é um pilar fundamental no desenvolvimento moderno, e a automação de testes emerge como a prática indispensável para garantir a confiabilidade e a estabilidade dos sistemas. Neste contexto, o Pytest se estabelece como o framework de testes mais popular e influente no ecossistema Python. A presente análise visa delimitar o escopo de atuação do Pytest, abordando sua definição, seus mecanismos de funcionamento e sua relevância estratégica para projetos de todos os portes.

O objetivo geral deste capítulo é fornecer uma compreensão abrangente sobre o Pytest, capacitando o leitor a integrá-lo de forma eficaz em seus projetos. Os objetivos específicos incluem detalhar as características que distinguem o Pytest de outros frameworks de teste, instruir sobre o processo de instalação e configuração, demonstrar a sintaxe para a escrita de testes unitários e de integração, e estabelecer critérios para a decisão de quando e como aplicar o Pytest no ciclo de vida do desenvolvimento.

A justificativa para a escolha do tema reside na crescente demanda do mercado por profissionais que

dominem práticas de desenvolvimento robustas, nas quais a testabilidade do código é uma prioridade. A metodologia adotada para a elaboração deste material consistiu na pesquisa bibliográfica de referências técnicas e na sistematização de exemplos práticos, visando uma abordagem didática e formal. O trabalho está organizado em uma Introdução que define o tema, um Desenvolvimento que explora o o quê, porque, como e quando usar o Pytest, e as Considerações Finais que sumarizam as conclusões e sugerem caminhos para estudos futuros

Desenvolvimento

O Pytest se estabelece como um framework de testes que transcende a mera execução de código de verificação, destacando-se pela sua notável simplicidade e inerente poder de extensão. Em contraste com o módulo unittest que é nativo da linguagem Python, o Pytest adota uma abordagem mais idiomática e menos cerimoniosa, permitindo a escrita de testes de forma concisa. Esta característica é alcançada pela utilização de funções simples em vez da imposição de classes e herança, o que culmina em um código de teste significativamente mais limpo, legível e, consequentemente, mais manutenível.

A essência do Pytest reside na sua filosofia de convenção sobre configuração, um princípio que facilita a descoberta e a execução dos testes. O framework é capaz de identificar automaticamente os arquivos que aderem ao padrão de nomenclatura, iniciando ou terminando com o prefixo `test_`, e, de maneira análoga, as funções de teste que se iniciam com `test_`. Esta automação reduz a sobrecarga de configuração e permite que o desenvolvedor se concentre na lógica de teste. A adoção do Pytest é justificada pela sua arquitetura que incorpora recursos avançados cruciais para a construção de suítes de testes de alta complexidade.

Entre os recursos que tornam o Pytest imperativo, destacam-se as Fixtures, que são funções reutilizáveis destinadas a configurar o ambiente de teste, como a criação de um cliente de banco de dados ou a inicialização de dados de mock. Elas promovem a redução da duplicação de código e conferem um controle mais granular sobre o estado de cada teste, sendo injetadas diretamente nas funções de teste que as declaram como argumentos. Outro pilar fundamental é a Parametrização, uma funcionalidade que possibilita a execução do mesmo teste com múltiplos conjuntos de dados de entrada, maximizando a cobertura de testes com um esforço mínimo de escrita. Adicionalmente, a vasta gama de Plugins disponíveis, como o `pytest-cov` para análise de cobertura de código ou o `pytest-xdist` para execução paralela, confere ao Pytest uma extensibilidade ímpar, permitindo a adaptação do framework às necessidades específicas de qualquer projeto, desde a geração de relatórios até a integração com sistemas externos. Por fim, a utilização de Asserções Simples, que emprega a keyword `assert` nativa do Python, é complementada pela geração de `tracebacks` detalhados em caso de falha, o que torna o diagnóstico de erros mais rápido e intuitivo, eliminando a necessidade de métodos de

asserção específicos.

A integração do Pytest no fluxo de trabalho de desenvolvimento é um processo direto e altamente eficiente. O como utilizá-lo inicia-se com a instalação via gerenciador de pacotes, seguida pela criação de arquivos de teste que respeitem a convenção de nomenclatura e a definição das funções de teste. A execução da suíte de testes é realizada através de um comando simples no terminal, que automaticamente descobre e executa todos os testes. O quando aplicar o Pytest abrange a totalidade do ciclo de desenvolvimento de software. É imperativo que sua integração ocorra desde as fases iniciais do projeto, alinhando-se idealmente com a metodologia de Desenvolvimento Orientado a Testes (TDD), onde a escrita dos testes precede a implementação do código de produção. O Pytest demonstra sua eficácia em diversas categorias de testes, incluindo Testes Unitários para a verificação de unidades de código isoladas, Testes de Integração para assegurar a correta interação entre módulos e serviços, e Testes Funcionais que simulam cenários de uso do sistema sob a perspectiva do usuário. Aprender e dominar o Pytest transcende a mera aquisição de uma habilidade técnica; é um investimento estratégico na cultura de qualidade do código, garantindo que o desenvolvedor possa contribuir para a criação de projetos de software mais confiáveis, robustos e fáceis de manter, um diferencial competitivo inegável no mercado de trabalho.

Considerações Finais

O Pytest representa um avanço significativo na forma como os testes são concebidos e executados em projetos Python. A sua arquitetura flexível e a riqueza de recursos, como fixtures e parametrização, transformam a tarefa de testar de um fardo em uma parte orgânica e eficiente do processo de desenvolvimento. A conclusão é que o Pytest não é apenas uma opção, mas uma necessidade para qualquer projeto Python que aspire à robustez e à longevidade.

Recomenda-se que o leitor aprofunde o estudo em tópicos avançados, como a escrita de plugins personalizados e a integração com ferramentas de Integração Contínua (CI/CD). Como proposta para trabalhos futuros, sugere-se a elaboração de um estudo comparativo de desempenho entre o Pytest e outros frameworks de teste, quantificando o impacto de cada um na velocidade de execução da suíte de testes em grandes bases de código.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 24 nov. 2025.

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 25 nov.

2025.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 24 nov. 2025.

Rspec Ruby on Rails

Pedro Mira Nicolau
pedromiranicolau@gmail.com
pedro.nicolau@fatec.sp.gov.br

Resumo

O RSpec é uma ferramenta de teste poderosa para garantir a qualidade e a confiabilidade de aplicações, amplamente utilizada no desenvolvimento de aplicações Ruby on Rails. Ele adota uma abordagem de Desenvolvimento Orientado por Comportamento (BDD), o que significa que os testes são escritos de forma a descrever o comportamento esperado do software em linguagem natural, tornando-os mais legíveis e compreensíveis para todos os membros da equipe, incluindo não-desenvolvedores.

Introdução

À medida que aplicações web se tornam mais complexas e estratégicas para os negócios, testar o código deixa de ser um “extra” para se tornar uma necessidade. No universo Ruby on Rails, o RSpec se consolidou como a principal ferramenta para criação de testes automatizados, permitindo que desenvolvedores escrevam código com mais segurança, clareza e previsibilidade. Em vez de apenas verificar se o sistema “funciona”, o RSpec incentiva uma forma de pensar centrada no comportamento esperado da aplicação, aproximando a linguagem do código da linguagem do negócio.

Neste capítulo, vamos explorar o RSpec de maneira prática e progressiva, partindo dos conceitos fundamentais até a aplicação em cenários reais de um projeto Rails. Você verá como configurar o ambiente, como estruturar seus arquivos de teste, como testar modelos, controladores e funcionalidades de ponta a ponta, além de conhecer recursos essenciais como factories, mocks, stubs e matchers personalizados. O objetivo é que, ao final, você esteja apto não apenas a escrever testes, mas a projetar sua aplicação já pensando em testabilidade. Dessa forma, o RSpec deixará de ser apenas uma ferramenta e passará a fazer parte natural do seu fluxo diário de desenvolvimento.

Desenvolvimento

1. O que é RSpec e por que usá-lo no Rails?

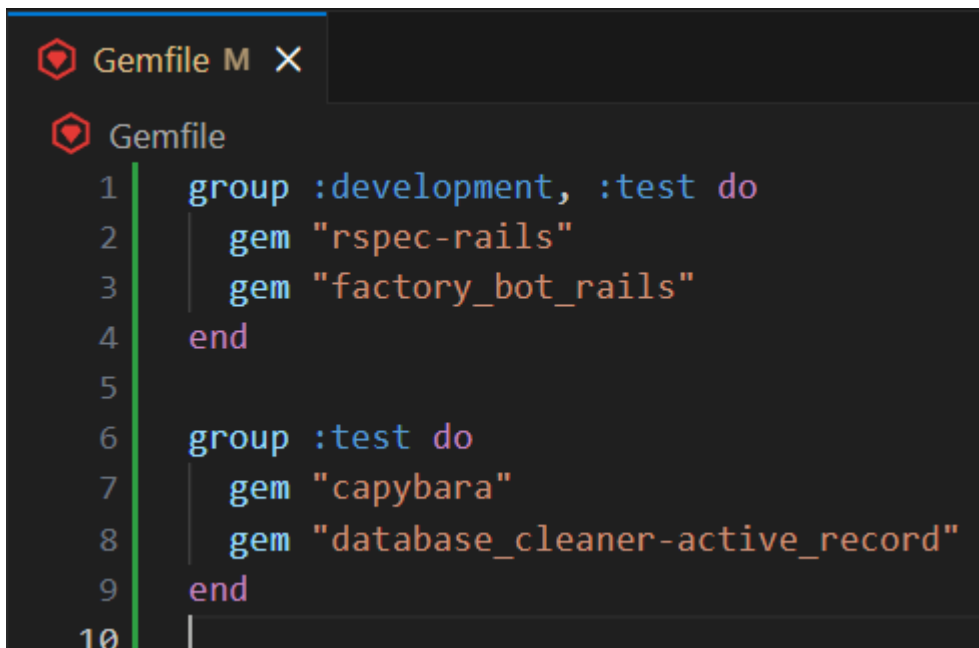
O RSpec é o framework de testes mais popular no ecossistema Ruby e Ruby on Rails. Baseado em BDD (Behavior-Driven Development), ele incentiva que você descreva o comportamento esperado da sua aplicação em uma linguagem quase natural, próxima do domínio do negócio. Em vez de escrever testes obscuros e difíceis de entender, você cria “especificações” legíveis que documentam como o sistema deve se comportar.

No contexto do Rails, o RSpec oferece tipos de testes para praticamente todas as camadas da aplicação: modelos, controllers, rotas, views, requisições e testes de sistema (com Capybara). Isso permite construir uma suíte de testes abrangente, que reduz regressões, aumenta a confiança em refactors e acelera a evolução da aplicação.

2. Instalando e configurando o RSpec em um projeto Rails

2.1. Adicionando ao Gemfile

Em um projeto Rails já existente, adicione as gems do RSpec:

A screenshot of a code editor window titled 'Gemfile M'. The editor shows the content of a Gemfile. Line 1: 'group :development, :test do'. Line 2: ' gem "rspec-rails"'. Line 3: ' gem "factory_bot_rails"'. Line 4: 'end'. Line 5: (empty). Line 6: 'group :test do'. Line 7: ' gem "capybara"'. Line 8: ' gem "database_cleaner-active_record"'. Line 9: 'end'. Line 10: (empty). The code is color-coded: 'group' is blue, ':development, :test' is purple, 'do' is blue, 'gem' is orange, and the gem names are in orange. The 'end' keywords are purple. A green vertical line is at the start of line 1.

```
1  group :development, :test do
2    gem "rspec-rails"
3    gem "factory_bot_rails"
4  end
5
6  group :test do
7    gem "capybara"
8    gem "database_cleaner-active_record"
9  end
10
```

Depois, rode:

bundle install

rails generate rspec:install

Esse comando cria:

- **.rspec** – Configurações globais do RSpec.
- **spec/spec_helper.rb** – Configurações gerais.
- **spec/rails_helper.rb** – Configurações específicas do Rails.

3. Anatomia de um teste RSpec

O coração do RSpec gira em torno de alguns blocos principais:

- **describe** – Agrupa exemplos relacionados (geralmente uma classe ou funcionalidade).
- **context** – Agrupa cenários dentro de um describe.
- **it** – Define um exemplo específico (um teste).
- **expect** – Faz asserções sobre o comportamento.

Exemplo simples:

```
calculadora_spec.rb U X
spec > models > calculadora_spec.rb
1  # spec/lib/calculadora_spec.rb
2  RSpec.describe 'Calculadora' do
3    describe '#soma' do
4      it 'soma dois números' do
5        resultado = 2 + 3
6
7        expect(resultado).to eq(5)
8      end
9    end
10 end
11
```

4. Testando Models com RSpec

Models concentram regras de negócio, validações e associações. Ter bons testes de model é fundamental.

4.1. Exemplo de Model

```
owner.rb X owner_spec.rb
app > models > owner.rb > ...
1  class Owner < ApplicationRecord
2    has_many :properties, dependent: :destroy
3    validates :name, :email, presence: true
4  end
```

4.2. Spec de Model

```
owner.rb  owner_spec.rb X
spec > models > owner_spec.rb
1  require 'rails_helper'
2
3  RSpec.describe Owner, type: :model do
4    it { should have_many(:properties).dependent(:destroy) }
5    it { should validate_presence_of(:name) }
6    it { should validate_presence_of(:email) }
7  end
8
```

Observação: os matchers `have_many` e `validate_presence_of` são do **shoulda-matchers**, uma **gem** muito usada em conjunto com RSpec. Você pode incluí-la no **Gemfile** se quiser reduzir o “boilerplate” dos testes de validação e associação.

5. Factories com FactoryBot

Factories facilitam a criação de objetos para teste, evitando repetir `User.create(...)` o tempo todo.

5.1. Exemplo de Factory

```
owner.rb  owner_spec.rb  owners.rb X
spec > factories > owners.rb
1  FactoryBot.define do
2    factory :owner do
3      name { Faker::Name.name }
4      email { Faker::Internet.email }
5      phone { Faker::PhoneNumber.cell_phone_in_e164 }
6    end
7  end
8
```

6. Testando Controllers / Requests

Em versões mais novas do Rails, a abordagem recomendada é usar **request specs** (tipo `:request`) em vez de specs de controller puros. Eles exercitam a aplicação mais “de fora”, como se fossem chamadas HTTP reais.

6.1. Exemplo de Controller

```
posts_controller.rb U X
app > controllers > posts_controller.rb > ...
1  class PostsController < ApplicationController
2    def index
3      @posts = Post.all
4    end
5
6    def show
7      @post = Post.find(params[:id])
8    end
9  end
10
```

6.2. Request Spec

```
posts_controller.rb U  posts_spec.rb U X
spec > requests > posts_spec.rb
1  require 'rails_helper'
2
3  RSpec.describe 'Posts', type: :request do
4    describe 'GET /posts' do
5      it 'retorna sucesso e lista de posts' do
6        create_list(:post, 3)
7
8        get '/posts'
9
10       expect(response).to have_http_status(:ok)
11       expect(response.body).to include('Lista de Posts') # por ex. título da view
12     end
13   end
14
15   describe 'GET /posts/:id' do
16     it 'retorna o post solicitado' do
17       post = create(:post, title: 'Post Especial')
18
19       get "/posts/#{post.id}"
20
21       expect(response).to have_http_status(:ok)
22       expect(response.body).to include('Post Especial')
23     end
24   end
25 end
26
```

7. Testando Views e Fluxos de Usuário (System Specs + Capybara)

Para testar a aplicação “como o usuário enxerga”, usamos **system specs** com Capybara.

7.1. Configuração básica (exemplo)

```
rails_helper.rb X
spec > rails_helper.rb
1  # spec/rails_helper.rb
2  RSpec.configure do |config|
3    config.include FactoryBot::Syntax::Methods
4    config.use_transactional_fixtures = false
5  end
6
7  # spec/support/capybara.rb
8  require 'capybara/rspec'
9
```

7.2. System Spec

```
user_creates_post_spec.rb U X
spec > system > user_creates_post_spec.rb
1  require 'rails_helper'
2
3  RSpec.describe 'Criação de posts', type: :system do
4    it 'permite criar um post' do
5      visit '/posts'
6      click_on 'Novo Post'
7
8      fill_in 'Título', with: 'Meu primeiro post'
9      fill_in 'Conteúdo', with: 'Conteúdo de teste'
10     click_on 'Criar Post'
11
12     expect(page).to have_content('Post criado com sucesso')
13     expect(page).to have_content('Meu primeiro post')
14   end
15 end
16
```

8. Matchers mais usados no RSpec

Alguns dos matchers mais comuns:

- **eq** – Igualdade (==)
- **eqi** – Igualdade de valor e tipo
- **be** – Comparação de identidade (objeto)
- **be_nil**, **be_truthy**, **be_falsey**
- **include** – Inclusão em coleções ou strings
- **match** – Expressões regulares
- **change { ... }.from(x).to(y)** – Mudança de valor

```
describe Array do
  it 'usa matchers variados' do
    lista = [ 1, 2, 3 ]

    expect(lista.size).to eq(3)
    expect(lista).to include(2)
    expect { lista << 4 }.to change { lista.size }.from(3).to(4)
    expect('rails rspec').to match(/rspec/)
  end
end
```

9. Mocks, Stubs e Doubles

Em muitos casos, você não quer chamar serviços externos reais (APIs, e-mails, filas) nos testes. Para isso, usamos:

- **doubles** – Objetos “falsos” que imitam outro objeto.
- **stubs** – Substituições de métodos que retornam valores controlados.
- **mocks** – Similar a stubs, mas com verificação de expectativa sobre chamadas.

9.1. Exemplo de Double

notificacao_service.rb U X

app > services > notificacao_service.rb

```
1 class NotificacaoService
2   def initialize(cliente_email:)
3     @cliente_email = cliente_email
4   end
5
6   def notificar(mensagem)
7     @cliente_email.enviar(mensagem)
8   end
9 end
10
```

notificacao_spec.rb U X

spec > services > notificacao_spec.rb

```
1 require 'rails_helper'
2
3 RSpec.describe NotificacaoService do
4   it 'envia e-mail usando o cliente configurado' do
5     cliente_email = double('ClienteEmail')
6
7     expect(cliente_email).to receive(:enviar).with('Olá, mundo!')
8
9     service = described_class.new(cliente_email: cliente_email)
10    service.notificar('Olá, mundo!')
11  end
12 end
13
```

9.2. Stub simples

```
it 'simula um método de classe' do
  allow(User).to receive(:count).and_return(10)

  expect(User.count).to eq(10)
end
```

10. Organização, performance e boas práticas

10.1. Nomeando arquivos e descrições

- Arquivos de spec devem seguir o nome da classe ou recurso que testam:
user.rb → user_spec.rb, posts_controller.rb → posts_controller_spec.rb ou request spec em spec/requests/posts_spec.rb.
- Use descrições claras em describe e it:
 - it 'retorna erro quando não há estoque'
 - it 'cria um pedido com itens válidos'

10.2. Um exemplo por comportamento

Cada bloco it deve testar **um comportamento específico**. Evite “testes gigantes” que misturam vários cenários, pois ficam difíceis de entender e de manter.

10.3. Deixando a suíte rápida

- Use FactoryBot.build_stubbed quando não precisar persistir no banco.
- Evite chamar APIs externas reais; isole com doubles/stubs.
- Use o database_cleaner (ou transações do próprio Rails) para manter o banco limpo entre testes.

10.4. Integração contínua

Em pipelines de CI (GitHub Actions, GitLab CI etc.), é comum um job como:

```
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ruby/setup-ruby@v1
        with:
          ruby-version: '3.3'
      - run: bundle install
      - run: bundle exec rails db:setup
      - run: bundle exec rspec
```

11. Fluxo de trabalho com TDD/BDD usando RSpec

O uso de RSpec combina muito bem com TDD (Test-Driven Development):

1. **Red** – Escreva um teste que falha (o comportamento desejado ainda não existe).
2. **Green** – Implemente o mínimo código necessário para o teste passar.
3. **Refactor** – Refatore o código (e eventualmente os testes), mantendo todos verdes.

Exemplo resumido

- Escreva o teste:

```
1  # spec/models/order_spec.rb
2  RSpec.describe Order, type: :model do
3    describe '#total' do
4      it 'soma o valor de todos os itens' do
5        order = Order.new
6        order.items.build(price_cents: 1000)
7        order.items.build(price_cents: 2500)
8
9        expect(order.total_cents).to eq(3500)
10      end
11    end
12  end
13
```

- Rode bundle exec rspec e veja falhar.
- Implemente o código mais simples possível:

```
# app/models/order.rb
class Order < ApplicationRecord
  has_many :items

  def total_cents
    items.sum(:price_cents)
  end
end
```

- Rode os testes de novo e verifique que tudo está verde.
- Refatore se houver duplicações ou melhorias possíveis.

Considerações Finais

Como desenvolvedor, a criação de testes parecia muito difícil no início, mas com um pouco de prática e estudo da teoria, ficou muito mais simples e fácil de executar. Hoje eu compreendo que ferramentas como o RSpec não são apenas um diferencial, mas uma necessidade. Por isso, recomendo fortemente a utilização dessas ferramentas em todos os projetos Ruby on Rails, garantindo assim a robustez e a qualidade do software.

Bibliografia

Criei um projeto bem simples de imobiliária para demonstrar um pouco mais dessas ferramentas na prática em: <https://github.com/pedromnicolau/Did-tica-Testes-Ruby-On-Rails>

JUnit: Garantindo Qualidade com Testes Unitários em Java

Gabriel Victorino
gabriel.victorino2021@gmail.com
gabriel.victorino@fatec.sp.gov.br

Resumo

O objetivo deste capítulo é demonstrar o uso do framework JUnit na criação e automação de testes unitários em aplicações desenvolvidas com Java, apontando sua importância no ciclo de vida do software e como contribui de forma direta para a qualidade dos sistemas. A partir de um caso prático baseado em um ticket simulado— que relata uma diferença de centavos entre os cálculos de dois sistemas corporativos —, é apresentada a aplicação de testes automatizados como ferramenta essencial para a detecção precoce de falhas, validação de regras de negócio e prevenção de regressões.

O estudo aborda as principais funcionalidades do JUnit 5, com ênfase nas anotações do JUnit Jupiter, no padrão AAA (Arrange, Act, Assert) e na sua integração com o Mockito para simulação de comportamentos externos.

Por meio da análise do ticket e da construção de um teste unitário completo, o capítulo reforça a importância da automatização como aliada da produtividade e da rastreabilidade, demonstrando como pequenas falhas podem ser identificadas antes de afetarem ambientes de produção.

Palavras-chave: JUnit. Testes Unitários. Qualidade de Software. TDD. Java. Mockito. BigDecimal.

Introdução

A qualidade de software é um dos pilares fundamentais no desenvolvimento moderno. Entre as diversas práticas que garantem essa qualidade, os testes unitários desempenham um papel essencial, pois validam o comportamento de pequenas partes do sistema antes que ele seja integrado por completo. No ecossistema Java, o framework JUnit se destaca como a principal ferramenta para essa tarefa, oferecendo simplicidade e flexibilidade para automação dos testes.

Visando ilustrar a aplicação prática dela, será analisado um caso real baseado em um ticket de suporte (ID HD-202510-883), onde uma diferença de centavos foi detectada no fechamento de lote entre os sistemas de Produção e ERP. Esse cenário demonstra como falhas aparentemente pequenas podem gerar impactos significativos e como o uso do JUnit pode prevenir tais erros.

O objetivo deste capítulo é demonstrar como criar testes automatizados utilizando o JUnit, relacionando-os com um problema real de integração e mostrando como a automação de testes ajuda a manter a precisão e a confiabilidade do software.

Desenvolvimento

1. O que é o JUnit

O JUnit é um framework **open source** utilizado para realizar testes automatizados em Java. Ele é composto por três componentes principais: JUnit Platform (responsável por lançar os testes na JVM), JUnit Júpiter (que fornece as novas anotações e o modelo de programação moderno) e JUnit Vintage (compatível com versões antigas do JUnit). Esses componentes tornam o JUnit uma solução completa e versátil para diferentes contextos de teste ao desenvolver software com Java.

2. Por que usar Testes Unitários

O uso de testes unitários traz benefícios significativos: garante qualidade, permite refatorar o código com segurança, serve como documentação viva e melhora o design do software. Quando aplicados desde o início do desenvolvimento, os testes ajudam a reduzir custos e evitam que erros simples se tornem problemas críticos na produção.

3. Exemplo Prático: Ticket HD-202510-883

O ticket descreve uma diferença de um centavo entre os cálculos de dois sistemas. O ERP registrava corretamente R\$ 435,00, enquanto o sistema de Produção calculava R\$ 434,99. Esse tipo de erro está frequentemente relacionado à forma como os valores decimais são tratados pelo sistema, especialmente em linguagens que utilizam ponto flutuante.

ID do Ticket: HD-202510-883

Data: 26/10/2025

Solicitante: Financeiro

Status: Novo

Prioridade: Média

Assunto: Diferença de centavos no fechamento de lote (Produção x ERP)

Descrição

A integração entre a Produção e o Estoque (ERP) está registrando valores diferentes para o mesmo lote, causando diferença no fechamento.

Exemplo do Problema

- Produto: Item SKU-XYZ
- Preço/Unidade: R\$ 4,35
- Quantidade no Lote: 100

O que estamos vendo

- Sistema (Produção): Registra o total como R\$ 434,99
- Sistema (ERP): Registra o total como R\$ 435,00

O ERP está correto ($4.35 * 100 = 435.00$). Precisamos que o sistema de Produção (ou a integração) pare de "comer" esse centavo.

Por favor, verificar.

Fonte: Próprio Autor

A classe **IntegradorEstoqueService** é responsável por registrar o consumo de itens no estoque, garantindo precisão nos cálculos realizados. Ela utiliza um repositório (**EstoqueRepository**) para persistir os dados e define constantes para controle da precisão decimal e do modo de arredondamento. O método **registrarConsumoErrado** demonstra um cálculo impreciso ao usar operações com tipos primitivos double, enquanto o método **registrarConsumoCorreto** aplica boas práticas ao utilizar BigDecimal para operações financeiras ou sensíveis, assegurando que o valor final seja arredondado e formatado corretamente antes de ser salvo no repositório.

Figura 01 – Classe Integrador Estoque

```
Untitled
public class IntegradorEstoqueService {

    private EstoqueRepository repository;

    private static final int PRECISAO = 8;
    private static final RoundingMode ARREDONDAMENTO = RoundingMode.HALF_UP;

    public IntegradorEstoqueService(EstoqueRepository repositoryEstoque) {
        this.repository = repositoryEstoque;
    }

    public void registrarConsumoErrado(double consumoUnidade, int quantidade) {
        double consumoCalculado = consumoUnidade * quantidade;

        BigDecimal valorSalvar = new BigDecimal(consumoCalculado);

        repository.salvarConsumo(valorSalvar);
    }

    public void registrarConsumoCorreto(double consumoUnidade, int quantidade) {
        BigDecimal consumoBD = BigDecimal.valueOf(consumoUnidade);
        BigDecimal quantidadeBD = BigDecimal.valueOf(quantidade);

        BigDecimal valorSalvar = consumoBD.multiply(quantidadeBD).setScale(PRECISAO, ARREDONDAMENTO);

        repository.salvarConsumo(valorSalvar);
    }
}
```

Fonte: Próprio autor

Figura 02 – Exemplo de Classe de Teste

```
class IntegradorEstoqueServiceTest {

    @Mock
    private EstoqueRepository repository;

    @Captor
    private ArgumentCaptor<BigDecimal> captorDeBigDecimal;

    @InjectMocks
    private IntegradorEstoqueService integradorService;

    private final double CONSUMO_POR_UNIDADE_DOUBLE = 4.35;
    private final int QUANTIDADE = 100;

    private final BigDecimal VALOR_ESPERADO = new BigDecimal("435.00000000");

    @Test
    @DisplayName("DEVE FALHAR: Método errado (cálculo em double) gera divergência contábil")
    void testeMetodoErrado() {
        System.out.println("--- Rodando teste do MÉTODO ERRADO ---");

        integradorService.registrarConsumoErrado(CONSUMO_POR_UNIDADE_DOUBLE, QUANTIDADE);
        verify(repository).salvarConsumo(captorDeBigDecimal.capture());

        BigDecimal valorSalvo = captorDeBigDecimal.getValue();

        System.out.println("Valor ESPERADO era:    " + VALOR_ESPERADO);
        System.out.println("Valor ERRADO salvou:  " + valorSalvo);

        assertEquals(VALOR_ESPERADO, valorSalvo,
            "O cálculo com 'new BigDecimal(double)' gerou um valor incorreto!");
    }

    @Test
    @DisplayName("DEVE PASSAR: Método correto (cálculo em BigDecimal) garante precisão contábil")
    void testeMetodoCorreto() {
        System.out.println("--- Rodando teste do MÉTODO CORRETO ---");

        integradorService.registrarConsumoCorreto(CONSUMO_POR_UNIDADE_DOUBLE, QUANTIDADE);
        verify(repository).salvarConsumo(captorDeBigDecimal.capture());

        BigDecimal valorSalvo = captorDeBigDecimal.getValue();

        System.out.println("Valor ESPERADO era:    " + VALOR_ESPERADO);
        System.out.println("Valor CORRETO salvou:  " + valorSalvo);

        assertEquals(VALOR_ESPERADO, valorSalvo,
            "O cálculo convertendo as entradas para BigDecimal deve ser preciso.");
    }
}
```

Fonte: Próprio autor

O exemplo acima aplica o padrão AAA (Arrange, Act, Assert), comum em testes

unitários. Primeiro, são definidos os dados necessários (Arrange), depois a operação é executada (Act), e por fim, o resultado é comparado com o esperado (Assert). Caso o cálculo falhe, o teste sinalizará o erro imediatamente.

4. JUnit e Mockito

Enquanto o JUnit é o motor de execução dos testes, o Mockito complementa o processo criando objetos simulados (mocks) para testar unidades de código isoladas. Isso é especialmente útil quando um método depende de chamadas externas, como banco de dados ou APIs. Com o uso de mocks, é possível validar o comportamento do código sem depender de integrações reais.

Considerações Finais

O uso de testes unitários com o JUnit mostrou-se fundamental para garantir precisão, estabilidade e confiabilidade em aplicações Java, especialmente em contextos que envolvem operações financeiras sensíveis e integração entre sistemas corporativos. No cenário apresentado — em que uma diferença de apenas um centavo entre dois sistemas (Produção e ERP) foi identificada —, a automação de testes revelou-se uma ferramenta indispensável para a prevenção de erros sutis, frequentemente provocados por problemas de arredondamento, formatação e representação numérica.

A criação de um teste automatizado capaz de reproduzir o problema descrito no ticket permitiu validar o comportamento esperado do cálculo, garantindo que o resultado permanecesse consistente e dentro dos limites de precisão definidos.

Além do aspecto técnico, o aprendizado proporcionado pelo uso do JUnit promove uma mudança cultural na forma de desenvolver software. Ao incorporar práticas de TDD (Test-Driven Development) e testes automatizados desde as etapas iniciais, o desenvolvedor passa a adotar uma postura mais analítica e preventiva, concentrando-se na qualidade desde o início do ciclo de vida do sistema, e não apenas em sua correção posterior.

Dessa forma, o JUnit se consolida não apenas como uma ferramenta de teste, mas como um instrumento pedagógico e estratégico para a construção de softwares mais confiáveis, sustentáveis e aderentes às boas práticas de engenharia.

Bibliografia

JUNIT. *Documentation*. Disponível em: <https://junit.org/>. Acesso em: 12 nov. 2025.

BAELDUNG. *JUnit Series*. Disponível em: <https://www.baeldung.com/junit>. Acesso em: 13 nov. 2025.

SARAIVA JUNIOR, Orlando. *ebook_qualidade_teste_software*. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

VICTORINO, Gabriel. *qualidade-teste-software*. GitHub. Disponível em: <https://github.com/GabrielVictorino8266/qualidade-teste-software>. Acesso em: 12 nov. 2025.

A ferramenta PHPUnit

Gustavo Francisco Habermann
gustavofhabermann@gmail.com
gustavo.habermann@fatec.sp.gov.br

Resumo

Este projeto apresenta a implementação e validação de duas classes em PHP, Email e Usuário, utilizando o PHPUnit como ferramenta de testes automatizados. O objetivo foi garantir a confiabilidade do código por meio de uma estrutura orientada a objetos que emprega validações robustas, boas práticas e testes sistemáticos. A metodologia adotada envolveu a criação de um *value object* para representar e validar endereços de e-mail, além de uma classe de usuário estruturada com foco em coerência e segurança dos dados. Em seguida, foram desenvolvidos testes unitários que verificam o comportamento esperado dos componentes, assegurando sua consistência e estabilidade. Os resultados demonstram como o uso do PHPUnit contribui significativamente para a qualidade e manutenção do software.

Palavras-chaves: Ferramenta PHPUnit. PHP. TDD.

Introdução

Este trabalho apresenta o uso de testes automatizados no desenvolvimento de software, utilizando o PHPUnit para validar funcionalidades criadas em PHP. De forma geral, o tema trata da importância de garantir que o código funcione corretamente desde as etapas iniciais do desenvolvimento. O estudo se concentra na criação e verificação de duas classes principais, Email e Usuário, aplicando conceitos básicos de programação orientada a objetos e boas práticas de organização do código, além de introduzir o conceito básico do mocks. O objetivo geral é mostrar como os testes automatizados ajudam a aumentar a segurança, a estabilidade e a qualidade do sistema. Já os objetivos específicos incluem validar corretamente endereços de e-mail, estruturar uma classe de usuário funcional e desenvolver testes unitários que confirmem o comportamento esperado dessas partes. A escolha da ferramenta se dá ao uso de PHP/Laravel no projeto integrador do 4º semestre, e a linguagem apresenta uma maior legibilidade e uma curva de aprendizado mais acessível para quem está começando no desenvolvimento de testes. A envolve pesquisa conceitual, implementação das classes e elaboração de testes unitários em PHPUnit. Por fim, o trabalho está organizado em seções que apresentam o tema, a implementação e os testes realizados.

Desenvolvimento

Para começar a desenvolver um projeto em PHPUnit, é necessário primrio baixar o PHP no site oficial do php:<https://www.php.net/downloads.php>, e após a instalação do PHP é necessário também a instalação do composer, onde o PHPUnit será instalado: <https://getcomposer.org/download/>, logo após o composer instalado basta rodar o comando dentro da pasta onde que começar o projeto PHPUnit no cmd(Prompt de comando):

Figura 1 – Instalação de PHPUnit

```
>composer require --dev phpunit/phpunit|
```

Fonte: próprio autor

Conforme observado na figura 1, após a execução desse código na sua pasta, isso irá criar dentro dela um composer.json com a ultima versão do PHPUnit disponível.

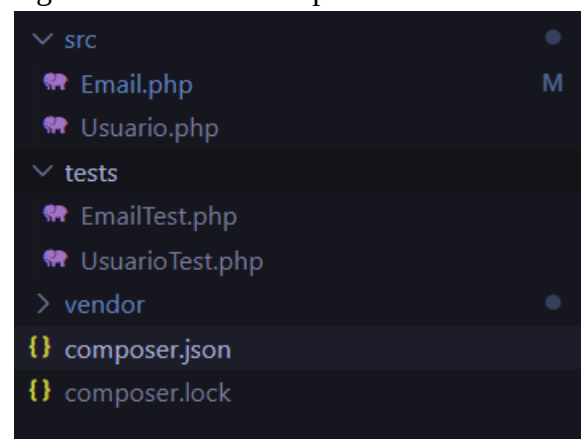
Figura 2 – Composer.json

```
{
  "require-dev": {
    "phpunit/phpunit": "^11.5"
  }
}
```

Fonte: próprio autor

Junto com a criação desse arquivo composer.json, irá ser criada a estrutura de projeto.

Figura 3 – Estrutura de pasta



Fonte: próprio autor

Como mostra na figura 3, tanto os testes como as classes em si ficam separadas em diferentes pastas, as classes ficam na pasta src/, já os testes que iremos criar ficam na pasta tests/.

Figura 4 – Classes criadas

```
<?php
class Email
{
    private $email;

    private function __construct(string $email)
    {
        $this->validarEmail($email);

        $email = trim($email); // Remove espaços indesejados
    }

    public static function retornarInstancia(string $email): self
    {
        return new self($email);
    }

    public function __toString(): string
    {
        return $this->email;
    }

    private function validarEmail(string $email): void
    {
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new \InvalidArgumentException(
                sprintf(
                    '"%s" não é um endereço de e-mail válido',
                    $email
                )
            );
        }
    }
}
```

```
c > Usuario.php > Usuario > getEmail

1 <?php
2
3 require_once __DIR__ . '/Email.php';
4
5 class Usuario
6 {
7     private Email $email;
8
9     public function __construct(Email $email)
10    {
11        $this->email = $email;
12    }
13
14    public function getEmail(): string
15    {
16        return (string) $this->email;
17    }
18 }
```

Fonte: próprio autor

Na figura 4 temos as classes que utilizaremos para montar os testes unitários.

Figura 5 – Testes unitários(E-mail)

```
<?php

require_once __DIR__ . '/../src/Email.php';
use Email;

//Para o PHPUnit saber que é um teste os metodos precisam do prefixo "test".
class EmailTest extends PHPUnit\Framework\TestCase{
    //Validar o tipo da instancia retornada
    public function testValidarTipoDaInstancia(): void{
        $this->assertInstanceOf(
            Email::class,
            Email::retornarInstancia('teste@teste.com')
        );
    }
    // Recebe 2 parametros e retorna se eles são iguais
    public function testTestarRetornoDaInstancia(): void
    {
        $this->assertEquals(
            'teste2@teste.com',
            Email::retornarInstancia('teste2@teste.com')
        );
    }
    //Diferença entre assert e expect
    // Assert: Afirma que o valor passado e o valor recebido sao do mesmo tipo 1º mesmo tipo de obj e no 2º mesmo tipo primitivo.
    // Expect: Já o expect espera que o nosso teste lance uma exceção do tipo InvalidArgumentException.
    public function testRetornaExcecaoSeEmailForInvalido(): void
    {
        $this->expectException(InvalidArgumentException::class);

        Email::retornarInstancia('teste_invalido.com');
    }
}

/*
Existe tambem outros tipos de assert:
assertTrue/assertFalse -> valida condições booleanas.
assertNull -> Verifica se uma variavel é nula.
assertContains -> Verifica se u elemento existe em 1 array ou string.
assertCount -> Valida o numero de itens dentro de 1 array ou itera.
*/
```

Fonte: próprio autor

Na figura 5 temos o primeiro teste unitário referente a classe e-mail. Como é possível ver no PHPUnit para uma classe ser considerada como um teste ela precisa ter o **extends TestCase**, e outra especificação do PHPUnit é que todas as funções presentes na classe de teste elas tem que começar com o prefixo test. A classe teste no PHPUnit possui os asserts presentes também em outros frameworks, no exemplo acima eu utilizei *assertInstanceOf*, *assertEquals* e alguns outros que não foram utilizados estão comentados. O framework do PHPUnit também aceita o *expectException*, ele é como se fosse um *assert*, porém ele pode receber uma exceção.

Figura 6 –Teste unitário(Usuário)

```
<?php

use PHPUnit\Framework\TestCase;

require_once __DIR__ . '/../src/Email.php';
require_once __DIR__ . '/../src/Usuario.php';

class UsuarioTest extends TestCase
{
    public function testGetEmailComMock(): void
    {
        //Cria um mock da classe Email
        $emailMock = $this->createMock(Email::class);

        //Define o comportamento di método toString
        $emailMock->method('__toString')->willReturn('teste_mock@teste.com');

        //cria um usuario usando mock
        $usuario = new Usuario($emailMock);

        //valida o comportamento
        $this->assertEquals('teste_mock@teste.com', $usuario->getEmail());
    }
}
```

Fonte: próprio autor

Nesse Teste, está a classe usuário, e como em outros frameworks o PHPUnit também possui o mock, nessa classe ele está sendo utilizado, para criar uma classe E-mail para definir o comportamento do método `__toString`, cria um mock de classe Usuário, e valida o comportamento usando o *assertEquals*.

Figura 7 – Rodando os códigos



```
./vendor/bin/phpunit tests
```

Fonte: próprio autor

Utilizando o código mostrado na figura 7, irá executar os testes e o próprio framework mostra qual passou e qual teste falhou.

Considerações Finais

O uso da ferramenta PHPUnit nesse projeto ajudou a garantir a qualidade do código e a realização de teste unitários. A utilização de testes unitários ajudou a identificar falhas e reforçou a importância de tentar prever o comportamento da aplicação em diferentes cenários. A experiência com o PHPUnit mostrou que mesmo em projetos de pequeno porte é muito importante a utilização de teste para maior organização e maior segurança no software entregue. Como proposta futura, recomenda-se ampliar a cobertura de testes para cenários ainda mais complexos.

Bibliografia

PHP. Disponível em: <https://www.php.net/downloads.php> Acesso em: 11 nov. 2025.

COMPOSER. Disponível em: <https://getcomposer.org/download/>. Acesso em: 11 nov. 2025.

PHPUnit. Documentation. GitHub. Disponível em: <https://phpunit.de/documentation.html>. Acesso em: 11 nov. 2025.

Codigo Fonte TV. PHPUnit (O Melhor Framework para Testes de Unidade no PHP). Disponível em: <https://www.youtube.com/watch?v=E2UIuhjwLY0&t>. Acesso em 18 nov. 2025.

A Ferramenta JUnit 5 e a Prática de TDD

João Augusto Selegatto Pacolla

jpaolla10@gmail.com

joao.pacolla@fatec.sp.gov.br

Resumo

Este trabalho tem como objetivo principal explorar a ferramenta JUnit 5, focando na sua aplicação prática dentro da metodologia de Desenvolvimento Guiado por Testes (TDD) usando a linguagem Java. A metodologia que guiou este estudo baseou-se em pesquisa bibliográfica sobre os conceitos fundamentais e na implementação prática de um ciclo de testes automatizados para validar o funcionamento do framework. O texto aborda desde a configuração inicial do ambiente de desenvolvimento e a escrita dos primeiros testes unitários, até a análise do ciclo conhecido como "Red-Green-Refactor".

Os resultados obtidos indicam que o uso do JUnit 5, quando aliado à disciplina do TDD, traz uma melhora nítida na qualidade do código final, facilita manutenções futuras e diminui a ocorrência de erros em produção. A ferramenta demonstrou ser robusta e sua arquitetura modular ajuda na jornada de aprendizado de desenvolvedores que estão começando a integrar a cultura de testes em seus softwares.

Palavras-chaves: Ferramenta JUnit 5. Java. TDD.

Introdução

Garantir a qualidade de software tornou-se um requisito indispensável no desenvolvimento de sistemas atuais. Neste cenário, o *Test Driven Development* (TDD), ou Desenvolvimento Guiado por Testes, aparece como uma prática vital para assegurar que o código nasça modular e com menos erros. Este trabalho delimita-se a estudar o JUnit 5, que é atualmente o framework de testes mais utilizado no ecossistema Java.

O objetivo geral aqui é demonstrar as características e como aplicar o JUnit 5 na prática. Os objetivos específicos envolvem: preparar o ambiente de desenvolvimento, mostrar exemplos de criação de testes unitários e relatar como foi a jornada de aprendizado na aplicação do ciclo de TDD. A escolha deste tema justifica-se pela alta procura do mercado de trabalho por profissionais

que saibam criar testes automatizados e pela necessidade de conhecer ferramentas eficientes que suportem essa prática.

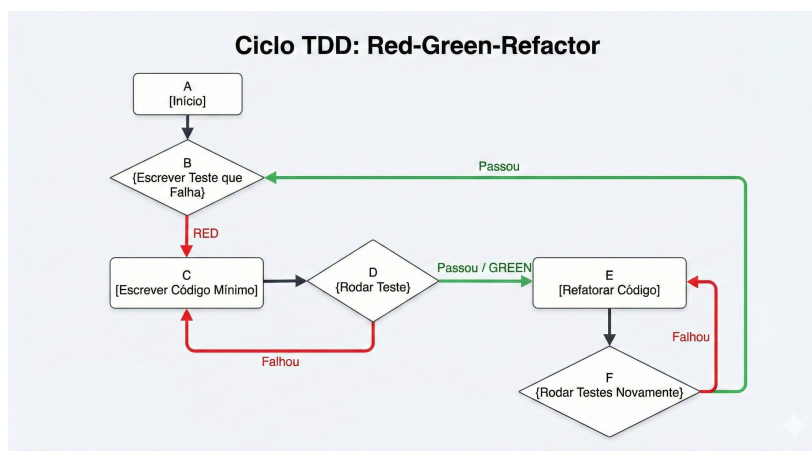
A metodologia usada baseia-se na revisão de documentações oficiais e na experimentação prática da ferramenta. O trabalho está organizado apresentando, primeiro, o desenvolvimento e as características da ferramenta com exemplos práticos, seguido pelas considerações finais sobre o aprendizado adquirido.

Desenvolvimento

O JUnit 5 representa a evolução do framework de testes mais famoso para Java. Diferente das versões anteriores, ele é composto por três subprojetos: JUnit Platform, JUnit Jupiter e JUnit Vintage. Essa divisão traz uma arquitetura mais modular e flexível. Durante o desenvolvimento deste estudo, uma característica que chamou a atenção foi o suporte melhorado a recursos modernos do Java (versão 8 ou superior), como as expressões lambda, que deixam a escrita dos testes mais direta.

Durante minha jornada de aprendizado com a ferramenta, o ponto central foi entender e aplicar o ciclo fundamental do TDD. A prática consiste em, primeiramente, escrever um teste que falhe (etapa Red), depois implementar o código mínimo necessário para o teste passar (etapa Green) e, finalmente, melhorar o código sem alterar seu comportamento (etapa Refactor). A Figura 1 ilustra esse fluxo que guias as dependências do projeto.

Figura 1 – O Ciclo Red-Green-Refactor do TDD



Fonte: Próprio autor.

Conforme observado na figura 1, todo o estudo do projeto ocorreu seguindo essa lógica, o que garante que cada pequena funcionalidade seja validada antes de o código crescer.

Para materializar este conceito, foi desenvolvido um exemplo prático de uma calculadora simples. A **Listagem 1** apresenta o código do teste unitário criado inicialmente (etapa Red), antes mesmo da classe principal existir ou ter lógica.

Listagem 1 – Código do teste unitário (CalculadoraTest.java)

```
Java
import org.junit.jupiter.api.Test; import org.junit.jupiter.api.DisplayName; import static
org.junit.jupiter.api.Assertions.assertEquals; class CalculadoraTest { @Test @DisplayName("Deve
somar dois números inteiros corretamente") void deveSomarDoisNumeros() { // Cenário (Arrange)
Calculadora calc = new Calculadora(); // Ação (Act) int resultado = calc.somar(2, 3); //
Verificação (Assert) assertEquals(5, resultado, "A soma de 2 + 3 deveria ser 5"); } }
```

Fonte: Próprio autor.

Em seguida, a implementação mínima necessária para o teste acima passar foi realizada na classe principal, conforme a **Listagem 2** (etapa Green).

Listagem 2 – Código da implementação mínima (Calculadora.java)

```
Java
public class Calculadora { // O método foi criado apenas para satisfazer o teste da Listagem 1 public int
somar(int a, int b) { return a + b; } }
```

Fonte: Próprio autor.

A utilização de anotações como `@Test` e `@DisplayName` (vistas na Listagem 1), que pertencem ao pacote Jupiter do JUnit 5, facilitou bastante a organização e a leitura dos casos de teste. A integração com a IDE (Ambiente de Desenvolvimento Integrado) permitiu um feedback visual imediato sobre o sucesso ou falha das validações.

Considerações Finais

A realização deste trabalho permitiu consolidar o entendimento sobre por que os testes automatizados são tão importantes no ciclo de vida de um software. Conclui-se que a ferramenta JUnit 5 é essencial para aplicar o TDD de forma efetiva em projetos Java, pois oferece uma API rica e moderna que simplifica a criação de cenários de teste.

Adotar o TDD mostrou-se um tanto desafiador no início da jornada de aprendizado. Exige uma mudança de mentalidade o fato de ter que escrever o teste antes da implementação real. No entanto, os benefícios a longo prazo, como a segurança para refatorar o código depois e ter uma "documentação viva" do sistema, superam a curva de aprendizado inicial.

Como recomendação para trabalhos futuros, sugere-se explorar testes de integração usando o JUnit 5 junto com ferramentas de *mock* (como o Mockito), para simular comportamentos externos, e também analisar a cobertura de código para obter métricas de qualidade mais avançadas.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2024.

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 11 nov. 2024.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

JUNIT TEAM. JUnit 5 User Guide. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 02 dez. 2024.

A ferramenta Selenium

Leandro Alves de Farias Rodrigues
Leandro.Rodrigues47@fatec.sp.gov.br
RA: 2901392313054

Resumo

Neste trabalho, será apresentado um estudo sobre o Selenium, um conjunto de ferramentas constantemente usadas para automação em ambientes de testes e aplicações. O principal objetivo será, entender as funcionalidades e aplica-las em um ambiente de testes simulado com o intuito de praticar e compartilhar os resultados obtidos durante o processo, além de demonstrar como executar a ferramenta. A metodologia aplicada inclui pesquisa bibliográfica, análise de documentação oficial e a construção de scripts automatizados utilizando Selenium WebDriver para análise de resultados. Esta ferramenta nos propõe alta flexibilidade, integração com diversas linguagens de programação e é gratuito, sendo assim, em ambientes de testes essas características torna-o uma excelente escolha. Sendo assim, conclui-se que, o Selenium é robusto, e amplamente aplicável, contribuindo de forma significativa para a melhoria da qualidade de software.

Palavras-chaves: Selenium, Automação de Testes, WebDriver, Python.

Introdução

Com o passar dos anos, novos aplicativos foram sendo desenvolvidos e, conseqüentemente, suas estruturas tornaram-se cada vez mais complexas, aumentando a possibilidade de erros em suas funcionalidades. Para solucionar esses problemas, a prática de testes passou a ganhar destaque e, com o avanço da tecnologia, as ferramentas de automação assumiram um papel fundamental, facilitando a execução de testes repetitivos e contribuindo para a criação de códigos mais confiáveis e bem estruturados.

Foi nesse contexto que surgiu o Selenium, criado em 2004 por Jason Huggins, engenheiro da empresa ThoughtWorks. Huggins percebeu a necessidade de automatizar testes que até então eram executados manualmente e de forma repetitiva em um aplicativo interno de controle de tempo e despesas, desenvolvido em Python e Plone. Para isso, ele criou inicialmente uma ferramenta chamada *JavaScriptTestRunner*, que serviu de base para a evolução do que viria a ser o Selenium.

Com o tempo, a ferramenta passou por aprimoramentos e mudanças de nomenclatura, originando o Selenium Core e posteriormente o Selenium Remote Control (Selenium RC). Em 2008, a unificação do WebDriver ao Selenium RC deu origem ao Selenium 2.0, que se consolidou como o padrão para automação de testes em navegadores. Desde então, a ferramenta continua

recebendo atualizações e melhorias, mantendo-se como uma das soluções mais utilizadas e influentes no campo da automação de testes.

A escolha desta ferramenta, se deu ao trabalho anterior, no qual foram exploradas ferramentas como, Pytest, Postman, Appium e Cypress, porém a que mais me chamou atenção foi o Selenium por sua versatilidade em rodar em diversas linguagens, por ser gratuito e por primeira vista não parecer tão difícil de fazer.

Sendo assim, esse trabalho visa apresentar os fundamentos do Selenium, compreender seu funcionamento e demonstrar, por meio da prática, sua aplicação em um ambiente de testes simulado garantindo os ensinamentos passados através deste semestre.

Desenvolvimento

A utilização do Selenium requer compreensão de sua arquitetura e dos componentes que o compõem. Esta ferramenta é estruturada principalmente em três partes:

- **Selenium WebDriver**, responsável pela comunicação direta com diferentes navegadores;
- **Selenium IDE**, utilizado para gravação rápida e simples de testes;
- **Selenium Grid**, que possibilita a execução paralela e distribuída de testes em múltiplos ambientes e máquinas.

Entre esses três componentes, o WebDriver é o mais utilizado atualmente, pois permite a automação completa de interações com páginas web por meio das diversas linguagens em que se pode usar, como Python, Java, JavaScript, C# e Ruby.

Para a prática apresentada neste trabalho, foi utilizado o Selenium com a linguagem **Python** em sua versão atual.

O teste consiste em abrir um navegador, realizar uma busca e encerrar a execução de forma automatizada. Inicialmente, tentou-se realizar a automação utilizando o Google, porém, ao detectar acessos automatizados, o site exibiu a verificação de segurança “*Não sou um robô*”, impedindo a continuidade do teste.

Por esse motivo, foi realizado o uso do buscador **DuckDuckGo**, que não bloqueia o uso de WebDrivers para testes simples.

Execução prática do Selenium

A seguir, são descritos todos os passos realizados, desde a configuração do ambiente até os resultados obtidos com o teste automatizado.

- **Passo 1 – Importação das bibliotecas**

Primeiramente, foi necessário instalar as dependências essenciais para a execução do Selenium, incluindo o gerenciador automático de drivers:

pip install selenium webdriver-manager

Em seguida, o código importa as classes principais do Selenium, como webdriver, By, Keys, além da biblioteca time para pausas curtas durante o teste.

Após a instalação, o código importa as classes utilizadas no teste:

1. Webdriver: controla o navegador;
2. By, Keys: localiza e interagir com elementos da página;
3. WebDriverWait e expected_conditions: para criar esperas inteligentes;
4. ChromeDriverManager: vai baixar automaticamente a versão correta do ChromeDriver;
5. Time: utilizado em pausas eventuais no fluxo

- **Passo 2 – Inicialização do Chrome**

O teste inicia criando uma instância do navegador Chrome com gerenciamento automático do driver:

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))

wait = WebDriverWait(driver, 10)

Essa linha faz com que o Selenium gerencie o navegador e esteja pronto para receber comandos.

- **Passo 3 – Acessar o mecanismo de busca**

O Selenium navega até o DuckDuckGo:

driver.get("https://duckduckgo.com")

- **Passo 4 – Realizar a busca desejada**

O programa localiza o campo de pesquisa da página por meio do atributo name:

search_box = wait.until(

EC.presence_of_element_located((By.NAME, "q"))

)

search_box.send_keys("ebook_qualidade_teste_software Orlando Saraiva GitHub")

search_box.send_keys(Keys.ENTER)

- **Passo 5 – Selecionar o primeiro resultado**

O Selenium identifica o primeiro elemento <h3>, que corresponde ao primeiro título de resultado da busca:

```
first_result = driver.find_element(By.CSS_SELECTOR, "h3")
first_result.click()
```

- **Passo 6 – Validação do acesso e captura de evidências**

Após acessar o link, o código aguarda o carregamento completo do título da página:

```
wait.until(lambda d: d.title != "")
```

E, em seguida, realiza uma validação automática (*assert*) para confirmar que a página acessada de fato corresponde ao repositório buscado:

```
assert "ebook_qualidade_teste_software" in driver.title.lower(), \
    "O título da página não corresponde ao repositório esperado."
```

Essa validação transforma um teste unitário em um teste automatizado, garantindo precisão e confiabilidade no resultado.

Como evidência, uma captura de tela é salva automaticamente:

```
driver.save_screenshot("resultado_teste.png")
```

- **Passo 7 – Encerrar o navegador**

Por fim, o navegador é fechado de maneira automatizada:

```
driver.quit()
```

Resultado:

Após rodar o código python teste.py o google chrome foi aberto e realizou a pesquisa no DuckDuckGo e após encontrar o primeiro H3 (github dos e-book do Orlando) ele acessou o link.

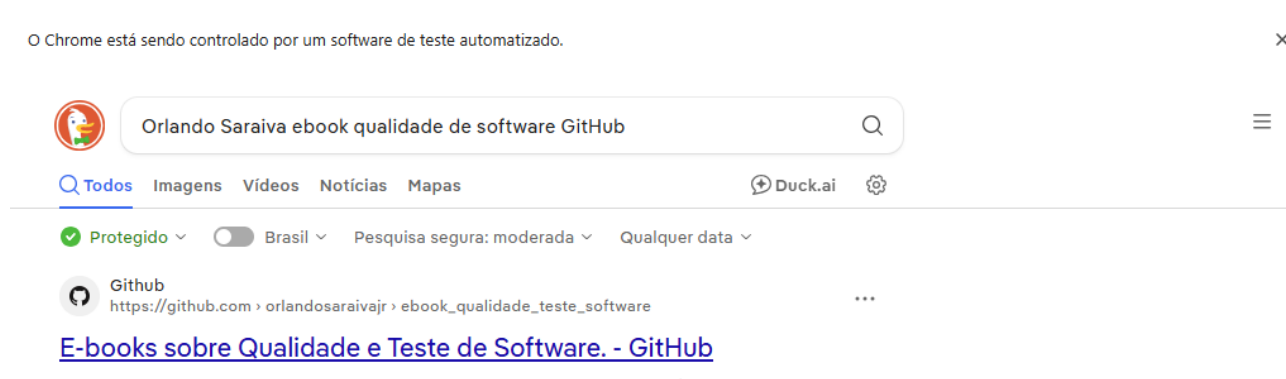


Figura 1 – Busca no DuckDuckGo de maneira automatizada.

Fonte: próprio autor

O próprio google chrome comenta que está sendo controlado por um software de teste automatizado.

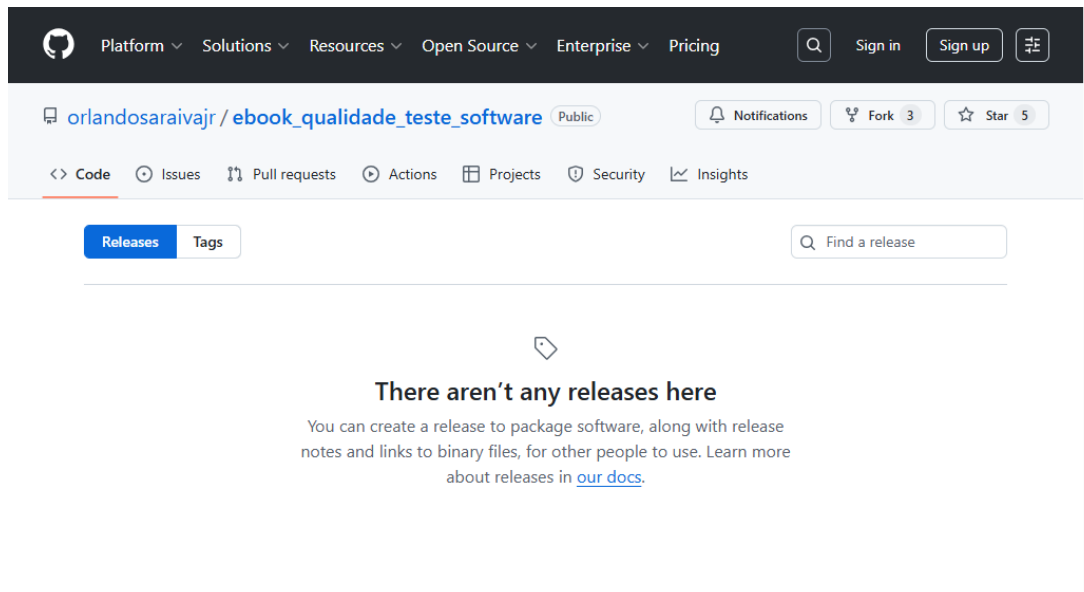


Figura 2 – Imagem registrada.

Fonte: próprio autor

Após a pesquisa no google e acessado o link desejado (Github do Orlando) esta foi a imagem registrada pelo Selenium.

```
PS C:\Users\Maria\OneDrive\Imagens\teste> python teste.py
Teste concluído com sucesso! O navegador acessou o repositório desejado.
Título da página: Releases · orlandosaraivajr/ebook_qualidade_teste_software
PS C:\Users\Maria\OneDrive\Imagens\teste> 
```

Figura 3 – Resultado.

Fonte: próprio autor

Por fim, após fechar o chrome, o terminal retornou que a página foi acessada com sucesso, ou seja, o teste foi um sucesso.

Considerações Finais

Esse capítulo apresentado realizou uma análise detalhada e direta sobre a ferramenta utilizada, abordando de maneira resumida sua história, contextualizando seus fundamentos teóricos e testando por via da prática como a ferramenta é utilizada em um ambiente de testes, um processo essencial para garantir maior qualidade, estabilidade e eficiência no desenvolvimento de aplicações.

A partir de sua introdução, foi possível perceber sua relevância significativa tanto no campo acadêmico quanto na aplicação profissional, uma vez que envolve testes automatizados, que são essenciais para uma estrutura sólida dos códigos de uma aplicação.

Nos dias atuais, como já referenciado anteriormente é essencial o uso de ferramentas de testes automatizados como o Selenium por ser uma ferramenta simples, versátil e gratuita. Além de permitir ser utilizada em diversas linguagens diferentes, sendo um excelente diferencial.

Com base nos testes realizados, ficou evidente a utilidade da ferramenta ao permitir a automação de interações reais em navegadores, como o Google Chrome, demonstrando seu valor para análises, validações e pesquisas futuras. O experimento prático reforçou a capacidade do Selenium de executar tarefas repetitivas de forma rápida e eficiente, reduzindo erros humanos e otimizando processos de teste.

Conclui-se, desta forma, que o estudo desenvolvido neste capítulo reflete os conhecimentos adquiridos ao longo do semestre, aliado às referências bibliográficas consultadas e a ajuda do professor Orlando Saraiva. Além disso, o entendimento e a prática do Selenium contribuem diretamente para a evolução de projetos futuros, podendo ser aplicados tanto em ambientes acadêmicos quanto profissionais, fortalecendo a qualidade e a confiabilidade de aplicações em desenvolvimento.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2024.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

TREINAWEB. *O que é Selenium?* Disponível em: [https://www.treinaweb.com.br/blog/o-que-e-](https://www.treinaweb.com.br/blog/o-que-e-selenium)

[selenium](https://www.treinaweb.com.br/blog/o-que-e-selenium). Acesso em: 26 nov. 2025.

WILDCARD, Simon et al. *Selenium WebDriver Documentation*. Selenium HQ. Disponível em:

<https://www.selenium.dev/documentation/webdriver/>. Acesso em: 27 nov. 2025.

Acessibilidade web como critério de qualidade de software: o impacto da inclusão digital na experiência do usuário e na conversão de negócios.

Maikon F. Gino

maikon.gino@gmail.com

Portfólio: <https://portfoliomaikongino.web.app>

Resumo

Este capítulo investiga a acessibilidade web (*ally*) como um pilar fundamental da Qualidade e Teste de Software Moderna, transcendendo a conformidade legal para se estabelecer como um critério crítico de qualidade no desenvolvimento *Frontend*. O objetivo principal é demonstrar a implementação de uma esteira de validação automatizada em uma aplicação real, denominada "TechInclusive", desenvolvida para este estudo. A metodologia adota a estratégia *Shift-Left Testing*, utilizando a ferramenta *Husky* para bloqueio preventivo de códigos não inclusivos, e convida o leitor a auditar a aplicação em tempo real utilizando o *Google PageSpeed Insights*. Os resultados evidenciam que a automação de testes de acessibilidade, em conformidade com a Lei Brasileira de Inclusão (LBI), mitiga riscos jurídicos e maximiza a performance técnica.

Palavras-chaves: Acessibilidade Web. Qualidade de Software. Husky. LBI. PageSpeed.

Introdução

A ubiquidade da internet transformou a maneira como serviços essenciais são consumidos, exigindo que a Qualidade e Teste de Software adote padrões rigorosos de qualidade. Neste cenário, a Acessibilidade Web — frequentemente abreviada pelo numerônimo *ally* (onde "11" representa o número de letras entre o 'a' e o 'y' na palavra *accessibility*) — define-se como a capacidade de uma interface ser perceptível, operável e compreensível por pessoas com diferentes tipos de deficiência.

Este trabalho delimita-se ao estudo da qualidade no desenvolvimento *Frontend*, focando na validação automatizada de interfaces conforme as Diretrizes de Acessibilidade para Conteúdo Web (WCAG). A justificativa para este tema apoia-se em dois pilares: o jurídico e o econômico. No Brasil, a Lei nº 13.146/2015 (Lei Brasileira de Inclusão), em seu Artigo 63, torna obrigatória a acessibilidade em sítios de empresas com sede no país. Sob a ótica de negócios, Byrne-Haber (2021) argumenta que a acessibilidade deve ser tratada como um requisito de negócio desde o início

do projeto, pois a exclusão de usuários com deficiência representa uma perda significativa de fatia de mercado e riscos à reputação da marca.

O objetivo geral deste capítulo é apresentar o desenvolvimento da aplicação web "TechInclusive", uma *Landing Page* de alta fidelidade visual, utilizada como prova de conceito para testes de qualidade. A metodologia diferencia-se por seu caráter interativo: o projeto encontra-se publicado em ambiente de produção, permitindo que o leitor replique os testes de performance e acessibilidade descritos.

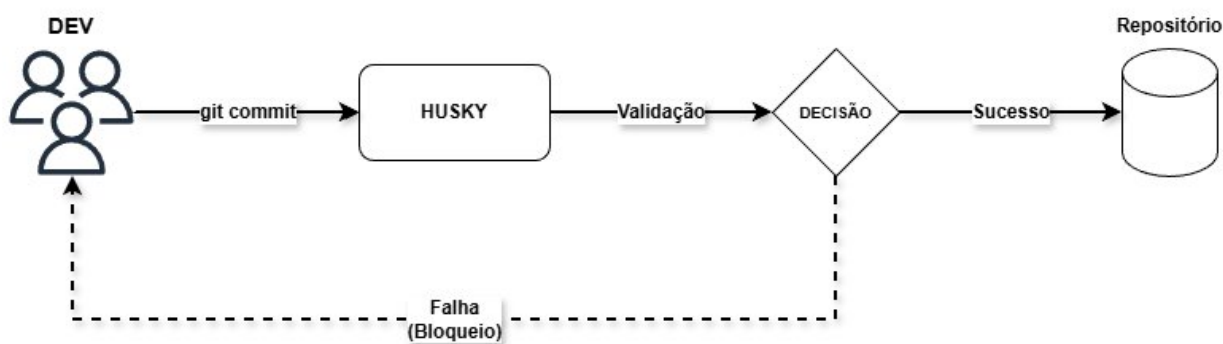
Desenvolvimento

A garantia da qualidade em interfaces modernas exige a superação de testes puramente manuais. Durante a jornada de aprendizado e implementação do projeto "TechInclusive", observou-se que a complexidade de designs modernos — com uso de transparências (*Glassmorphism*) e animações 3D — frequentemente compromete a usabilidade para leitores de tela se não houver rigor técnico.

1. O Fluxo de Governança (Shift-Left)

Para mitigar erros antes que cheguem à produção, adotou-se o conceito de *Shift-Left Testing*. Diferente do modelo tradicional, onde testes ocorrem no final, aqui a validação é movida para a máquina do desenvolvedor. A Figura 1 ilustra a arquitetura de proteção implementada.

Figura 1 – Diagrama do fluxo de trabalho com validação preventiva.



Fonte: Próprio autor.

2. Governança de Código: A Prevenção (Husky)

Para garantir a integridade do código antes mesmo que ele chegue ao repositório, utilizou-se a ferramenta *Husky*. Segundo Roldán (2023), a implementação de *git hooks* automatizados através do *Husky* é uma das melhores práticas fundamentais para aplicações modernas, pois assegura que

regras de qualidade — como *linting* e testes de acessibilidade — sejam aplicadas de forma coercitiva no ambiente local do desenvolvedor, prevenindo a "contaminação" do código em produção.

O *Husky* foi configurado neste projeto para atuar como um "porteiro" (*gatekeeper*), estabelecendo um fluxo de controle rigoroso descrito em quatro etapas lógicas:

- **Gatilho (*Trigger*):** O desenvolvedor finaliza uma alteração e executa o comando de versionamento (*git commit*).

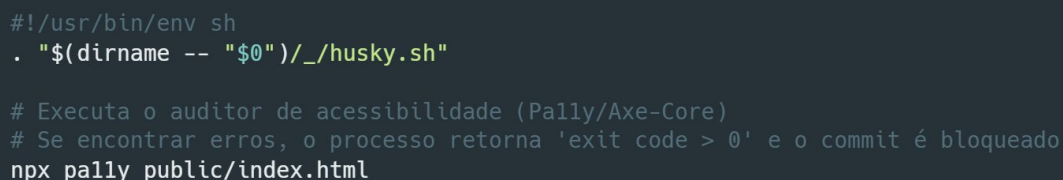
- **Interceptação:** O *Husky* reconhece o comando e pausa o processo de gravação antes que ele seja concluído, acionando o gancho denominado *pre-commit*.

- **Auditoria Automática:** Neste momento de pausa, o *Husky* executa o *script* de validação (biblioteca *axe-core*) sobre os arquivos que estão sendo enviados. O sistema varre o código em busca de violações da WCAG, como baixo contraste ou falta de etiquetas semânticas.

- **Veredito:** O sistema retorna um código de status. Se o resultado for Sucesso (código 0), o *commit* é efetivado. Se houver Falha (código 1), o *Husky* aborta a operação imediatamente.

Para operacionalizar essa auditoria, foi necessária a configuração explícita do *hook* de pré-submissão. No diretório de configuração do *Husky*, editou-se o arquivo *.husky/pre-commit* para invocar a interface de linha de comando (*CLI*) do validador. A instrução foi configurada para executar o motor de testes especificamente sobre o artefato de distribuição final (*public/index.html*), garantindo que a validação ocorra sobre o mesmo código que será servido ao usuário final. O Quadro 1 apresenta o *script* de automação implementado.

Quadro 1– Script de configuração do hook pre-commit.



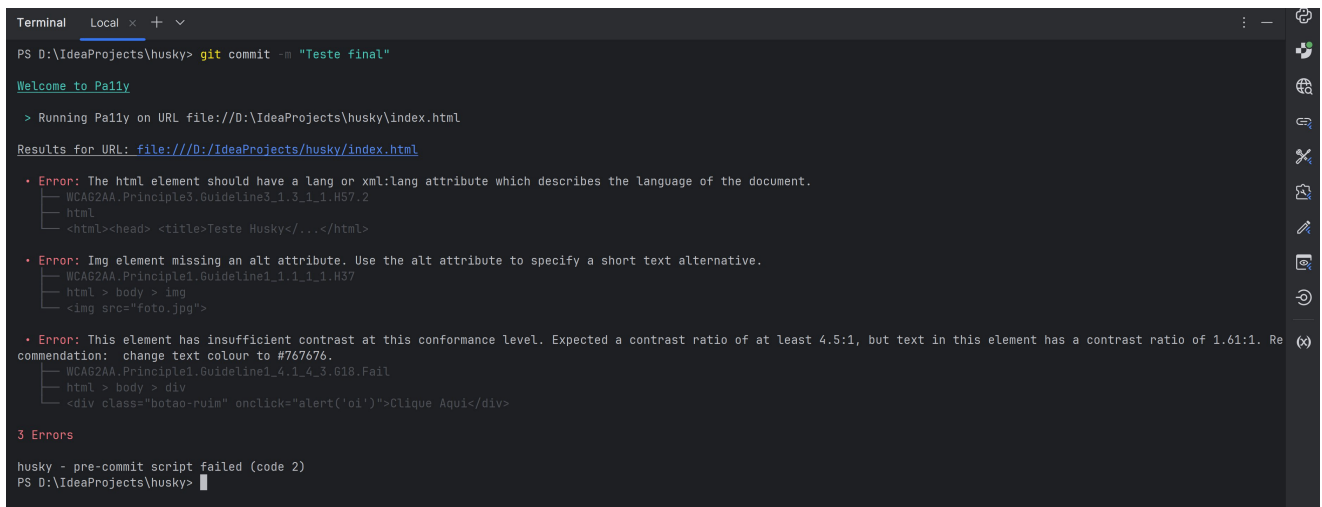
```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

# Executa o auditor de acessibilidade (Pally/Axe-Core)
# Se encontrar erros, o processo retorna 'exit code > 0' e o commit é bloqueado
npx pally public/index.html
```

Fonte: Próprio autor.

Conforme demonstrado na Figura 2, ao tentar submeter um código propositalmente falho, o sistema bloqueia a operação e exibe o relatório de erros no terminal, forçando a correção imediata.

Figura 2 – Bloqueio de código via terminal pelo *Husky* devido a falhas de acessibilidade.

A screenshot of a terminal window with a dark background. The prompt is 'PS D:\IdeaProjects\husky> git commit -m "Teste final"'. Below the prompt, it says 'Welcome to Pally' and '> Running Pally on URL file:///D:/IdeaProjects/husky/index.html'. Then, it shows 'Results for URL: file:///D:/IdeaProjects/husky/index.html'. There are three error messages listed: 1. 'Error: The html element should have a lang or xml:lang attribute which describes the language of the document.' with a reference to WCAG2AA.Principle3.Guideline3.1.3_1.1.H57.2 and a code snippet showing an HTML document without a lang attribute. 2. 'Error: Img element missing an alt attribute. Use the alt attribute to specify a short text alternative.' with a reference to WCAG2AA.Principle1.Guideline1.1.1_1.1.H37 and a code snippet showing an img tag without an alt attribute. 3. 'Error: This element has insufficient contrast at this conformance level. Expected a contrast ratio of at least 4.5:1, but text in this element has a contrast ratio of 1.61:1. Recommendation: change text colour to #767676.' with a reference to WCAG2AA.Principle1.Guideline1.4.1_4.3.G18.Fail and a code snippet showing a button with grey text on a white background. At the bottom, it says '3 Errors' and 'husky - pre-commit script failed (code 2)'. The prompt returns to 'PS D:\IdeaProjects\husky>'.

```
Terminal Local x + v
PS D:\IdeaProjects\husky> git commit -m "Teste final"

Welcome to Pally

> Running Pally on URL file:///D:/IdeaProjects/husky/index.html

Results for URL: file:///D:/IdeaProjects/husky/index.html

• Error: The html element should have a lang or xml:lang attribute which describes the language of the document.
  WCAG2AA.Principle3.Guideline3.1.3_1.1.H57.2
  html
  <html><head> <title>Teste Husky</...</html>

• Error: Img element missing an alt attribute. Use the alt attribute to specify a short text alternative.
  WCAG2AA.Principle1.Guideline1.1.1_1.1.H37
  html > body > img
  

• Error: This element has insufficient contrast at this conformance level. Expected a contrast ratio of at least 4.5:1, but text in this element has a contrast ratio of 1.61:1. Recommendation: change text colour to #767676.
  WCAG2AA.Principle1.Guideline1.4.1_4.3.G18.Fail
  html > body > div
  <div class="botao-ruim" onclick="alert('oi!')">Clique Aqui</div>

3 Errors

husky - pre-commit script failed (code 2)
PS D:\IdeaProjects\husky>
```

Fonte: Próprio autor.

A análise técnica da Figura 2 evidencia a precisão da ferramenta ao barrar o envio baseando-se em três critérios críticos de acessibilidade (WCAG 2.1 AA):

- **Semântica Global:** O erro H57.2 aponta a ausência do atributo *lang* na tag HTML. Sem isso, leitores de tela não conseguem identificar o idioma correto para a síntese de voz (pronúncia).

- **Conteúdo Não Textual:** O erro H37 denuncia a falta do atributo *alt* na imagem (**). Isso torna o conteúdo visual inexistente para usuários cegos.

- **Perceptibilidade Visual:** O erro G18 detecta uma falha grave de contraste. A ferramenta calculou uma taxa de 1.61:1 entre a fonte cinza e o fundo branco, muito abaixo do mínimo exigido de 4.5:1. O sistema chega a sugerir a correção técnica (alterar a cor para #767676).

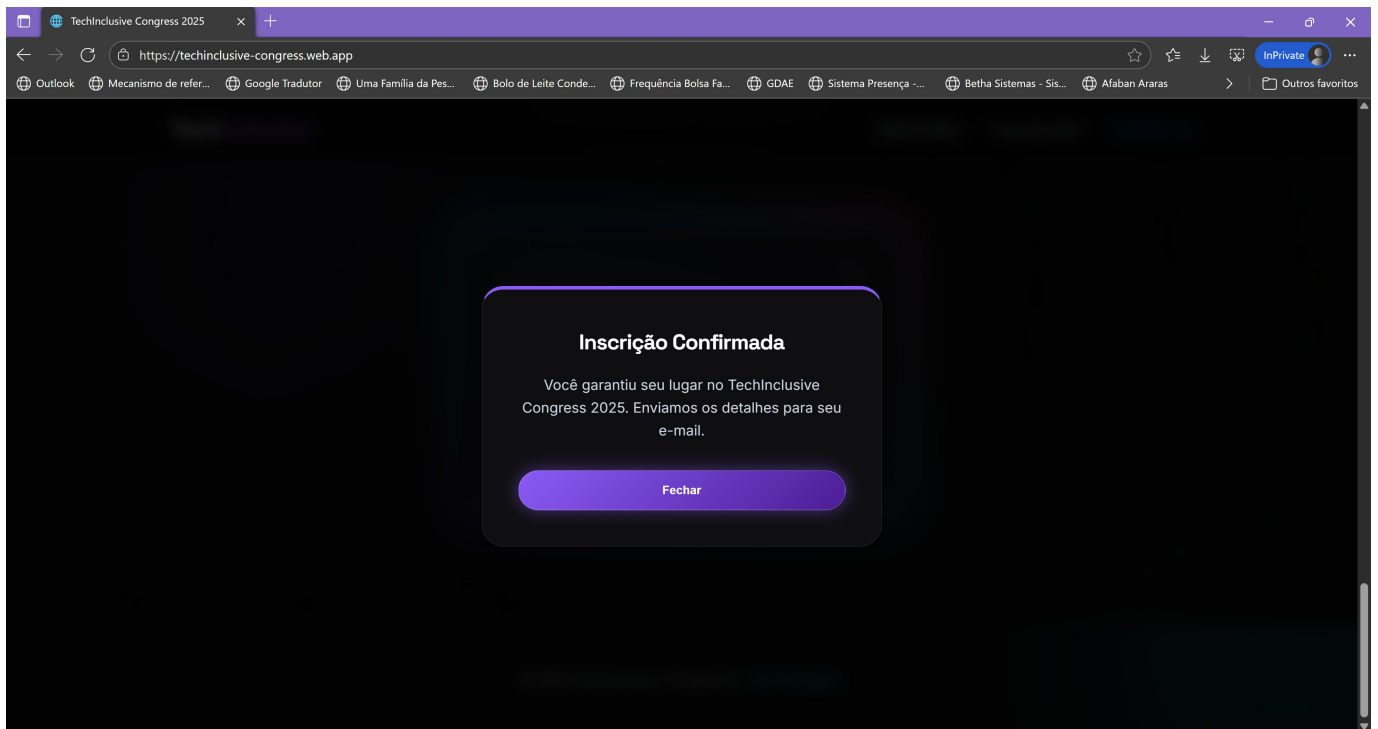
A mensagem final *pre-commit script failed* confirma que o *Husky* cumpriu seu papel de governança, rejeitando o código defeituoso.

3. O Objeto de Estudo: Projeto "TechInclusive"

A aplicação desenvolvida para este estudo simula uma página de inscrição para um evento de tecnologia. O desafio técnico consistiu em implementar uma interface visualmente rica (utilizando texturas de ruído, efeitos *parallax* e modais nativos) mantendo a conformidade total com as normas WCAG 2.2. A aplicação utiliza HTML5 Semântico e CSS Moderno para garantir que a estética "*premium*" não sobreponha a funcionalidade e a robustez do código.

A Figura 3 demonstra o resultado visual da aplicação, destacando o uso de janelas modais (*<dialog>*) estilizadas, que mantêm o foco do teclado preso ao contexto, garantindo navegabilidade para usuários com deficiência motora.

Figura 3 – Interface da aplicação "TechInclusive" exibindo modal de inscrição acessível.



Fonte: Próprio autor.

4. Auditoria Interativa: O Leitor como Testador

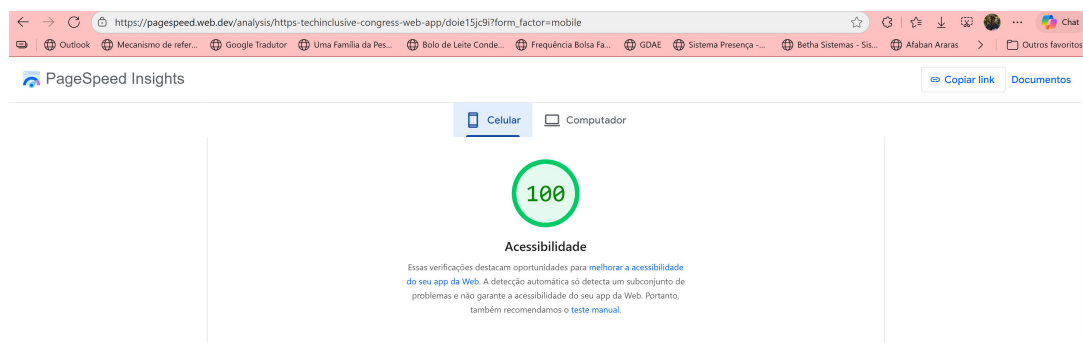
Para validar os resultados deste estudo de forma prática, propõe-se que o leitor realize a auditoria da aplicação em tempo real. A validação ocorre no ambiente de produção, utilizando o *Google PageSpeed Insights*, ferramenta que simula o acesso via dispositivos móveis em redes 4G e avalia métricas de *Core Web Vitals*.

Para realizar o teste e comprovar a qualidade do código, siga o roteiro abaixo:

- Acesse a ferramenta de auditoria oficial em: <https://pagespeed.web.dev/>
- No campo de URL, insira o endereço do projeto: <https://techinclusive-congress.web.app>
- Clique em "Analisar".

A ferramenta executará o motor *Lighthouse*, verificando métricas de desempenho e conformidade de acessibilidade.

Figura 4 – Resultado da auditoria demonstrando pontuação máxima em Acessibilidade e SE.



Fonte: Próprio autor.

Como demonstra a Figura 4, a aplicação atinge a pontuação máxima (100) em Acessibilidade. Isso comprova que o uso de elementos visuais avançados não impede a inclusão, desde que a codificação siga os padrões de qualidade e testes automatizados propostos por Da Costa (2021).

Considerações Finais

A realização deste estudo comprovou que a qualidade de software no *Frontend* é indissociável da acessibilidade. Através do projeto "TechInclusive", demonstrou-se que é possível alinhar design de alta performance com inclusão digital. A aplicação das normas da Lei Brasileira de Inclusão (LBI), garantida por ferramentas automatizadas como o *Husky*, provou ser uma estratégia eficaz para evitar o retrabalho e riscos jurídicos.

Conclui-se que o teste de acessibilidade não é uma etapa final, mas contínua. Para trabalhos futuros, recomenda-se a integração destes testes em esteiras de *CI/CD* (Integração Contínua), garantindo que a qualidade do software seja monitorada a cada nova funcionalidade implementada.

Bibliografia

BRASIL. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, 2015. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm. Acesso em: 15 nov. 2025.

BYRNE-HABER, Sheri. **Giving a Damn About Accessibility: A Candid and Practical Handbook for Designers, Developers, and Product Managers**. 1. ed. [S.l.]: UX Collective, 2021.

DA COSTA, Lucas. **Testing JavaScript Applications**. 1. ed. Shelter Island: Manning Publications, 2021.

DEQUE SYSTEMS. **axe-core: Accessibility engine for automated testing**. GitHub, 2024. Disponível em: <https://github.com/dequelabs/axe-core>. Acesso em: 16 nov. 2025.

GOOGLE DEVELOPERS. **Measure performance with PageSpeed Insights**. Web.dev, 2024. Disponível em: <https://developers.google.com/speed/docs/insights/v5/about>. Acesso em: 16 nov. 2025.

ROLDÁN, Carlos Santana. **React 18 Design Patterns and Best Practices: Design, build, and deploy production-ready web applications with React 18**. 4. ed. Birmingham: Packt Publishing, 2023.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.2**. World Wide Web Consortium, 2023. Disponível em: <https://www.w3.org/TR/WCAG22/>. Acesso em: 15 nov. 2025.

Framework de Testes em Dart (package:test)

Mateus Augusto Brito de Sousa
mateusaugusto303@hotmail.com
mateus.sousa12@fatec.sp.gov.br

Resumo

Este trabalho aborda o uso de testes automatizados na linguagem Dart por meio do pacote oficial `package:test`, amplamente utilizado para garantir qualidade e confiabilidade no desenvolvimento de software. Com uma abordagem progressiva e exemplos práticos, o conteúdo apresenta desde a estruturação básica dos arquivos até funcionalidades essenciais do framework de testes.

O `package:test` simplifica a criação e execução dos testes, incentiva boas práticas de organização e contribui diretamente para um código mais robusto e sustentável. Também destaco a medição de cobertura de código, que auxilia na identificação de trechos ainda não validados pelos testes. O objetivo é demonstrar como aplicar o framework na prática, evidenciando sua eficiência e sua importância para diferentes tipos de projetos escritos em Dart.

Palavras-chave: Dart, Testes Automatizados, Qualidade de Software, `package:test`, Cobertura de Código.

Introdução

Testes automatizados desempenham um papel essencial na construção de aplicações confiáveis, reduzindo falhas e garantindo que funcionalidades se comportem conforme o esperado durante todo o ciclo de desenvolvimento. Neste capítulo, abordo o `package:test`, a principal ferramenta de testes unitários utilizada em projetos Dart, conhecida por sua simplicidade, flexibilidade e facilidade de integração.

O conteúdo apresenta uma visão geral do framework, passando pelos principais conceitos envolvidos, como estruturação de testes, organização dos arquivos, boas práticas e exemplos reais aplicados no código. O objetivo principal é demonstrar como empregar o `package:test` na criação e execução de testes unitários, destacando sua relevância no desenvolvimento moderno de software.

A escolha do tema se justifica pela crescente demanda por aplicações seguras e livres de erros. A linguagem Dart, cada vez mais utilizada em projetos mobile, web e desktop, requer

ferramentas que garantam a estabilidade do código — e o `package:test` se mostra uma solução simples, eficiente e adequada para projetos de qualquer porte.

O texto segue uma abordagem didática e progressiva, dividido em seções que apresentam desde os fundamentos até recursos importantes, finalizando com considerações sobre a adoção do framework como parte da rotina de desenvolvimento profissional.

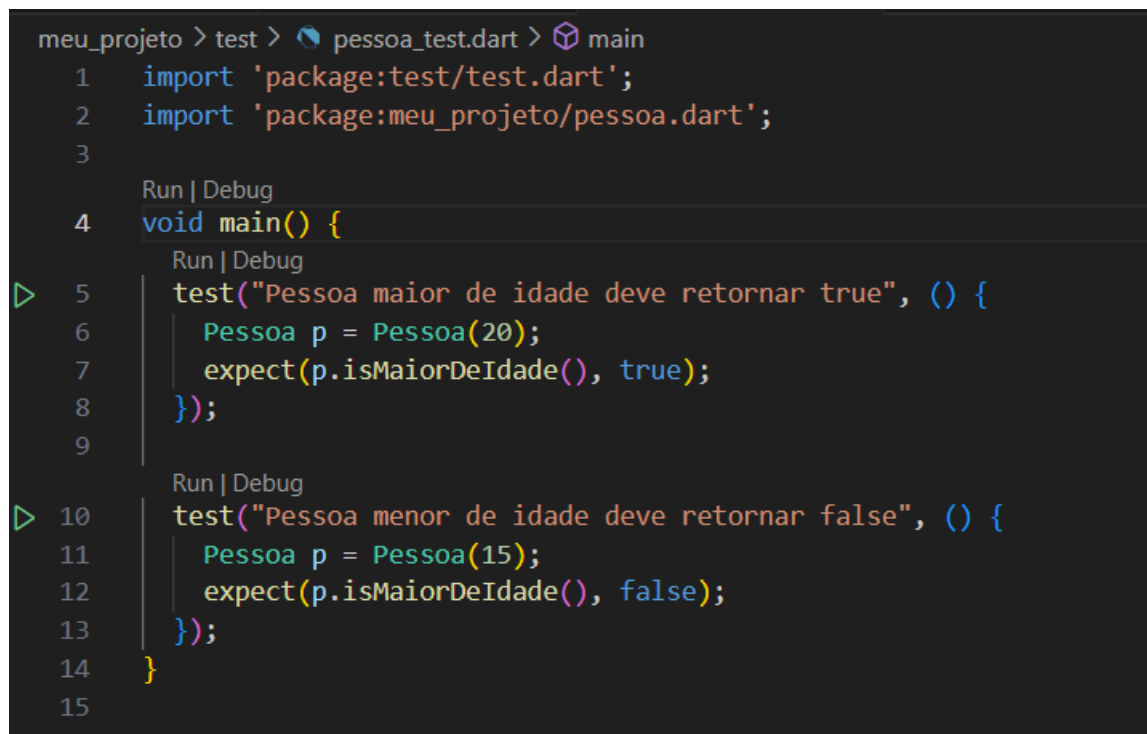
Vantagens do `package:test` em Dart

Simplicidade:

A criação de testes em Dart é extremamente direta e intuitiva, sem necessidade de estruturas complexas ou classes específicas. O `package:test` permite que os testes sejam escritos de forma clara e objetiva, usando apenas funções e expectativas simples.

Conforme ilustrado abaixo na Figura 1, escrever um teste unitário pode ser tão simples quanto criar uma função que verifica se um resultado esperado corresponde ao valor retornado pelo código:

Figura 1 – Exemplo de teste simples em Dart usando `package:test`



```
meu_projeto > test > pessoa_test.dart > main
1  import 'package:test/test.dart';
2  import 'package:meu_projeto/pessoa.dart';
3
4  void main() {
5      test("Pessoa maior de idade deve retornar true", () {
6          Pessoa p = Pessoa(20);
7          expect(p.isMaiorDeIdade(), true);
8      });
9
10     test("Pessoa menor de idade deve retornar false", () {
11         Pessoa p = Pessoa(15);
12         expect(p.isMaiorDeIdade(), false);
13     });
14 }
15
```

Fonte: próprio autor

Vantagens do package:test em Dart

Flexibilidade:

O package:test oferece suporte integrado para diferentes tipos de testes, incluindo testes unitários, de integração e testes parametrizados (usando funções e listas de valores). Além disso, permite configurar cenários antes e depois de cada teste utilizando funções como setUp() e tearDown(), facilitando a preparação e limpeza do ambiente de teste. O framework detecta automaticamente os arquivos de teste sem a necessidade de configurações adicionais.

Extensibilidade:

Dart permite integração com ferramentas complementares, como geradores de cobertura de código (dart test --coverage) e bibliotecas externas para criação de mocks, como o pacote mockito. Também é possível criar utilidades próprias para auxiliar nos testes, atendendo a necessidades específicas do projeto.

Identificação Automática:

O package:test identifica automaticamente arquivos de teste seguindo a convenção utilizada pela comunidade:

- Os arquivos devem ser colocados na pasta test/.
- Os nomes normalmente terminam com _test.dart.
- Essa padronização facilita a execução automática de todos os testes presentes no projeto.

Comunidade Ativa:

Assim como outras ferramentas de código aberto, o package:test possui uma comunidade ativa, bem documentada e constantemente atualizada, garantindo estabilidade, novos recursos e integração com diferentes tipos de projetos que utilizam Dart, incluindo Flutter.

Testes em Dart na Prática

Instalação

Instalar e configurar o ambiente de testes em Dart é simples. O package:test pode ser adicionado diretamente ao projeto usando o gerenciador de pacotes do Dart:

```
00:00 +1: test(pessoa_test.dart: Pessoa menor de idade deve retornar false)
00:00 +2: All tests passed!
>> C:\Users\User\Desktop\QUALIDADE E TESTES DE SOFTWARE\meu_projeto> dart pub add test --dev
"test" is already in "dev_dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  _fe_analyzer_shared 88.0.0 (92.0.0 available)
  analyzer 8.1.1 (9.0.0 available)
  lints 5.1.1 (6.0.0 available)
Got dependencies!
3 packages have newer versions incompatible with dependency constraints.
```

Após instalar a dependência, o comando abaixo pode ser utilizado para executar todos os testes do projeto:

```
PS C:\Users\User\Desktop\QUALIDADE E TESTES DE SOFTWARE\meu_projeto> dart test
Building package executable... (1.3s)
Built test:test.
00:00 +2: All tests passed!
```

Isso permite rodar os testes no terminal de forma ágil e consistente.

Estrutura e Convenções no package:test

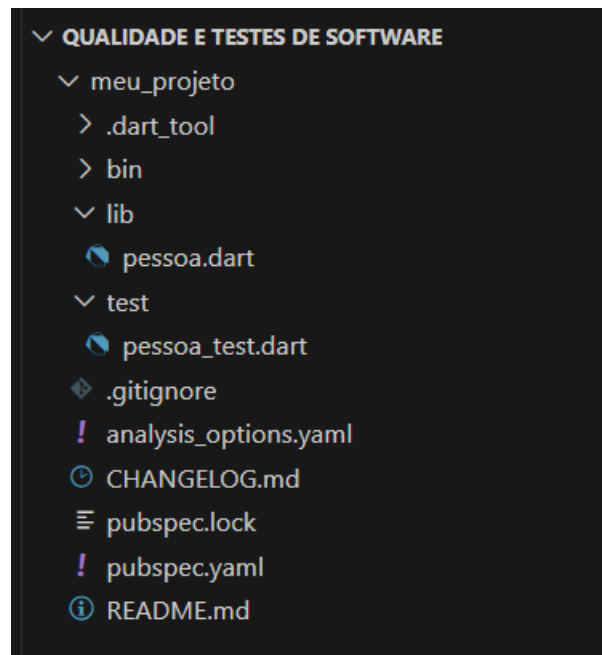
Manter uma estrutura organizada é essencial para facilitar a manutenção e evolução do projeto. No Dart, a estrutura convencional para testes segue estas regras:

- Os arquivos de teste devem ficar na pasta **test/**.
- Os arquivos costumam ser nomeados com o sufixo **_test.dart**.

Exemplos:

- calculadora_test.dart
- pessoa_test.dart
- Cada arquivo pode conter múltiplos testes, organizados com as funções `test()` e `group()`.

Exemplo da estrutura do projeto:



Considerações Finais

A utilização de ferramentas de teste, como o package:test no ecossistema Dart, é essencial para assegurar a qualidade, a confiabilidade e a manutenção de sistemas de software. Este capítulo apresentou uma análise desse framework acessível e amplamente utilizado para criação de testes automatizados, destacando suas principais características, como simplicidade, flexibilidade e capacidade de expansão.

Por meio de exemplos práticos, foi possível compreender como estruturar testes de forma organizada, aplicar diferentes cenários por meio de funções parametrizadas, configurar ambientes de preparação com setUp() e tearDown(), além de avaliar a cobertura de código. Quando incorporadas ao ciclo de desenvolvimento, essas práticas aumentam significativamente a segurança do software produzido e reduzem custos associados a falhas que poderiam passar despercebidas.

Embora existam outras abordagens para testes em Dart — especialmente no contexto de Flutter, como o flutter_test — o package:test continua sendo a escolha preferida quando se trata de testes unitários em projetos Dart puros. Sua facilidade de uso atende tanto iniciantes quanto desenvolvedores experientes, oferecendo recursos robustos sem sacrificar a simplicidade.

Em projetos reais, o `package:test` demonstra ser uma solução estável, eficiente e escalável, capaz de atender desde pequenos módulos até sistemas mais complexos. Assim, sua adoção representa um passo estratégico para desenvolvedores que buscam combinar produtividade com boas práticas de engenharia de software, criando aplicações mais seguras e confiáveis.

Bibliografia

DART. Package:test Documentation. Disponível em:

<https://pub.dev/packages/test> - Acesso em: 19 nov. 2024.

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2024.

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 11 nov. 2024.

SARAIVA JUNIOR, Orlando. `ebook_qualidade_teste_software`. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

PyTest: Framework de testes em Python

Miguel Fernandes
miguelfernandeses@gmail.com
miguel.santo@fatec.sp.gov.br

Resumo

Este capítulo tem como objetivo apresentar a ferramenta **pytest**, utilizada para a automação de testes na linguagem de programação Python. O trabalho aborda a importância de garantir a qualidade de software através de verificações que assegurem o funcionamento correto do código. A metodologia adotada consiste na exploração das funcionalidades principais do framework, demonstrando desde a configuração do ambiente até a escrita e execução de testes unitários. Os resultados obtidos evidenciam que o pytest possui uma sintaxe limpa e intuitiva, facilitando a adoção de boas práticas de teste por desenvolvedores de todos os níveis. Conclui-se que o uso desta ferramenta é essencial para aumentar a confiabilidade do desenvolvimento e reduzir a ocorrência de erros em produção.

Palavras-chaves: Ferramenta Pytest. Python. Automação de Testes. Qualidade de Software.

Introdução

A qualidade de software é um pilar fundamental no desenvolvimento moderno. Para garantir que um sistema funcione corretamente, utilizamos testes automatizados, que são scripts criados para verificar se o código está fazendo o que deveria. Este trabalho foca no estudo da ferramenta **pytest**, uma das soluções mais populares e eficientes para testes na linguagem Python.

O objetivo deste capítulo é documentar a jornada de aprendizado sobre o pytest. Definimos como escopo a configuração do ambiente, a compreensão da sintaxe básica da ferramenta e a aplicação prática em um projeto de software.

A escolha do pytest se justifica pela sua simplicidade. Diferente de outros frameworks que exigem códigos complexos apenas para começar, o pytest permite escrever testes de forma direta e legível. A metodologia deste trabalho divide-se em apresentar as características teóricas da ferramenta e, em seguida, demonstrar sua aplicação prática em um cenário real de desenvolvimento.

0,,

Desenvolvimento

Características Técnicas do Pytest: O pytest se destaca no ecossistema Python por unir uma sintaxe simples a recursos avançados de configuração. Diferente de outros frameworks que exigem a criação de classes complexas para rodar testes, o pytest permite o uso de funções simples e diretas.

Um dos seus pontos fortes são as **Asserções Nativas**, o pytest utiliza a palavra reservada **assert** do próprio Python. Isso torna a leitura do teste intuitiva: se a expressão for verdadeira, o teste passa; se for falsa, o framework exibe um relatório detalhado do erro.

Outro recurso muito interessante é as Fixtures. As fixtures permitem criar essa preparação de forma reutilizável. Por exemplo, para testar a remoção de uma tarefa, não precisamos criar a tarefa manualmente dentro do teste, podemos utilizar a fixture que já entrega o gerenciador com tarefas cadastradas, garantindo isolamento e limpeza de código.

Além disso, a ferramenta possui alta **extensibilidade via Plugins**. A comunidade mantém diversos complementos, como o **pytest-cov** (para medir cobertura de código) e o **pytest-html** (para gerar relatórios visuais), permitindo que a ferramenta cresça junto com a complexidade do projeto.

Jornada de Aprendizado: O Gerenciador de Tarefas Para consolidar esses conceitos, desenvolvi um projeto prático de **Gerenciador de Tarefas**. O desafio consistia

em garantir a integridade das funções de adicionar, listar e concluir tarefas.

Durante a implementação, utilizei a estrutura padrão de descoberta de testes do pytest, onde arquivos iniciados com **test_** são executados automaticamente. A **Figura 1** demonstra a aplicação prática da sintaxe simples discutida anteriormente:

Figura 1 – Implementação de Fixture e seu uso em um teste unitário.

```
import pytest
from tarefa import Tarefa, StatusTarefa

class TestTarefa:

    @pytest.fixture
    def nova_tarefa(self):
        """Fixture que retorna uma instância de tarefa padrão para os testes"""
        return Tarefa("Tarefa de Teste", "Descrição para teste", 3)

    def test_marcar_concluida(self, nova_tarefa):
        """Testa marcação de tarefa como concluída usando fixture"""
        nova_tarefa.marcar_concluida()
        assert nova_tarefa.status == StatusTarefa.CONCLUIDA
        assert nova_tarefa.data_conclusao is not None

    def test_marcar_concluida(self, nova_tarefa):
        """Testa marcação de tarefa como concluída usando fixture"""
        nova_tarefa.marcar_concluida()
        assert nova_tarefa.status == StatusTarefa.CONCLUIDA
        assert nova_tarefa.data_conclusao is not None

    def test_titulo_vazio_lanca_erro(self):
        """Testa que título vazio lança erro"""
        with pytest.raises(ValueError, match="Título não pode estar vazio"):
            Tarefa("")
```

Fonte: próprio autor

Conforme observado na figura 1, o teste recebe a nova_tarefa pronta e valida se o status muda para "CONCLUIDA". A simplicidade do código permitiu focar na regra de negócio, eliminando repetições desnecessárias. Ao executar o comando no terminal, o pytest forneceu um feedback instantâneo sobre o sucesso da implementação, conforme visto na **Figura 2**:

Figura 2 – Resultado da execução dos testes no terminal indicando sucesso.

```
test_tarefa.py::TestTarefa::test_criar_tarefa_valida PASSED [ 14%]
test_tarefa.py::TestTarefa::test_marcar_concluida PASSED [ 28%]
test_tarefa.py::TestTarefa::test_titulo_vazio_lanca_erro PASSED [ 42%]
test_tarefa.py::TestTarefa::test_prioridade_invalida_lanca_erro PASSED [ 57%]
test_tarefa.py::TestTarefa::test_marcar_concluida_duas_vezes_lanca_erro PASSED [ 71%]
test_tarefa.py::TestTarefa::test_mudar_status PASSED [ 85%]
test_tarefa.py::TestTarefa::test_status_invalido_lanca_erro PASSED [100%]

===== 7 passed in 0.06s =====
```

Fonte: próprio autor.

Essa experiência prática evidenciou como o uso de fixtures e asserções simples agiliza o ciclo de desenvolvimento, permitindo refatorar o código do gerenciador com a segurança de que nenhuma funcionalidade foi quebrada.

Considerações Finais

A elaboração deste ebook e a aplicação prática do pytest permitiram concluir que a barreira de entrada para testes automatizados é baixa. A ferramenta provou ser extremamente eficiente, transformando a tarefa de testar o código — muitas vezes vista como trabalhosa — em um processo rápido e integrado ao desenvolvimento.

Para o projeto do Gerenciador de Tarefas, o uso do pytest trouxe mais segurança, garantindo que as funcionalidades básicas não quebrassem com novas alterações. Recomenda-se, para estudos futuros, a exploração de testes mais avançados, como a simulação de banco de dados e a integração contínua (CI/CD), para elevar ainda mais o nível de qualidade do software entregue.

Bibliografia

PYTEST. Fixtures. Disponível em <https://docs.pytest.org/en/6.2.x/fixture.html>. Acesso em: 02 Dez. 2025.

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 02 Dez. 2025.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 102

Dez. 2025.

Pytest

Rafaela de Jesus Bispo dos Santos

Rafaela_jbs@hotmail.com

Rafaela.santos63@fatec.sp.gov.br

Resumo

Este trabalho apresenta o uso do Pytest, uma das ferramentas mais populares e eficientes para testes automatizados em Python. Ele foi aplicado em um projeto pequeno que desenvolvi, demonstrando sua capacidade de realizar desde testes unitários simples até testes de maior escala, como os utilizados em APIs e aplicações web. O Pytest fornece relatórios claros, indicando quais testes passaram e quais falharam, permitindo identificar rapidamente pontos críticos no código.

O uso dessa ferramenta traz maior segurança na evolução do sistema, reduz falhas em produção e facilita a manutenção contínua do código. Além disso, sua automação contribui para um fluxo de trabalho mais organizado e eficiente, beneficiando tanto quem desenvolve quanto quem fará manutenção futura. Por ser simples, extensível e compatível com diferentes níveis de teste, o Pytest se destaca como uma solução essencial em ambientes colaborativos.

Palavras-chaves: Pytest, Python, Automação, testes unitários.

Introdução

O Pytest é uma das principais ferramentas de testes automatizados utilizadas na linguagem Python, é adotada por desenvolvedores e equipes de QA por se destacar pela facilidade de uso e pela forma simples e flexível. Ele foi desenvolvido para tornar o processo de testar códigos simples, com poucos comandos e uma estrutura intuitiva, possibilitando validar funções, módulos completos e partes complexas de uma aplicação, como APIs e serviços web.

O Pytest oferece recursos essenciais como detecção automática de arquivos de teste, relatórios detalhados, execução parametrizada e uso de fixtures, tornando o processo de teste mais ágil e organizado. Sua flexibilidade e facilidade de uso o tornam ideal tanto para iniciantes quanto para profissionais que desejam aplicar boas práticas de qualidade de software em seus projetos Python. Dentro desse contexto, este trabalho aborda o uso do Pytest aplicado a um sistema pequeno,

demonstrando como a ferramenta contribui para validar funções, regras de negócio e comportamentos esperados no código.

O tema é delimitado à aplicação do Pytest para testes unitários e automatizados. Tendo como objetivo geral analisar sua utilidade no processo de verificação de software. Como objetivos específicos, buscar compreender seu funcionamento, apresentar sua aplicação prática e evidenciar os benefícios obtidos para a manutenção e evolução do sistema.

A escolha do Pytest se deu pela sua eficiência em apontar erros, facilitar ajustes no código e tornar o desenvolvimento mais seguro. A metodologia utilizada consiste na criação e execução de testes unitários em um projeto simples, permitindo observar na prática o desempenho da ferramenta.

Desenvolvimento

O Pytest é uma ferramenta de testes automatizados amplamente utilizada para garantir a qualidade de sistemas em pequena e grande escala. Ele permite analisar o comportamento de um código e verificar se os resultados obtidos estão alinhados com o que realmente se espera, fortalecendo a confiabilidade do software. Ao longo do desenvolvimento de um sistema que é extremamente pequeno e simples usei o Pytest para criar testes unitários, que dividem o código em partes menores e testam cada funcionalidade isoladamente, garantindo que cada componente funcione de forma correta e independente e facilitando a manutenção do código.

Apliquei testes para validar regras específicas do sistema, como cálculos, retornos de funções, inserção de informações, tratamento de erros e comportamentos esperados. Esse processo tornou o desenvolvimento mais seguro, pois qualquer alteração no código é automaticamente verificada, reduzindo falhas e facilitando a manutenção. Assim, o Pytest se mostrou uma ferramenta essencial para aumentar a estabilidade do projeto e manter o sistema preparado para futuras evoluções.

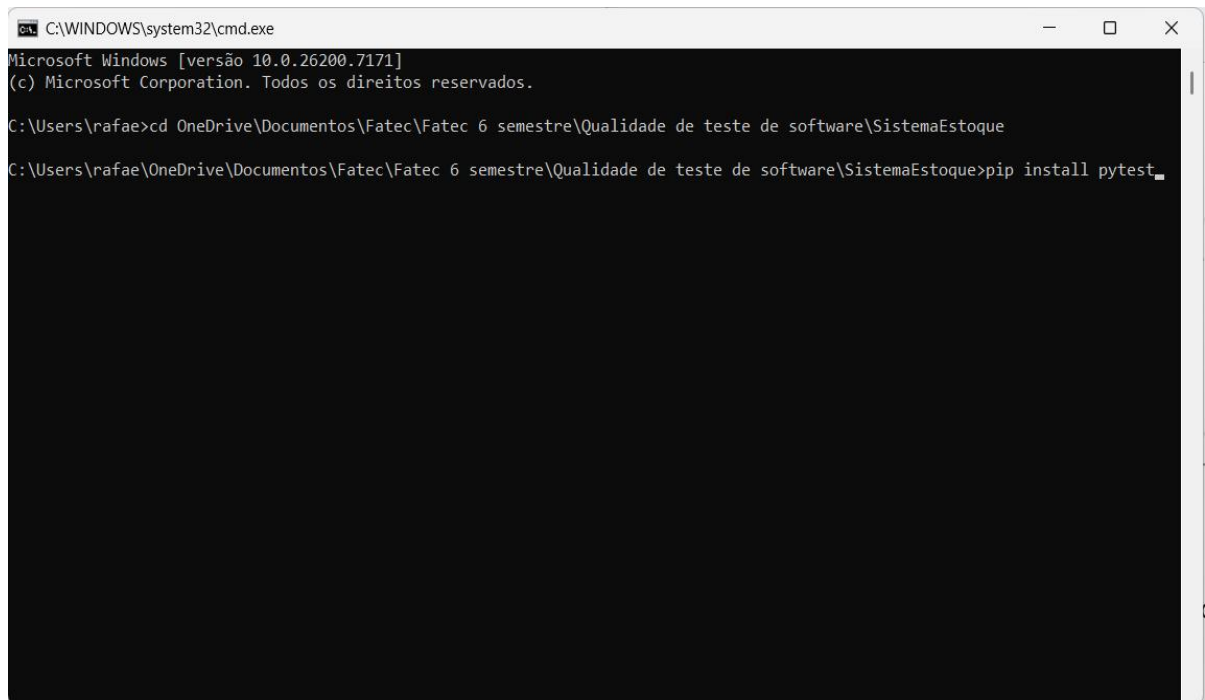
Vantagens do Uso do Pytest no Desenvolvimento

O Pytest oferece uma forma prática e eficiente de garantir que o código funcione corretamente em diferentes situações. Entre suas principais vantagens estão a facilidade de uso, a capacidade de detectar automaticamente arquivos de teste e a geração de relatórios claros, que mostram exatamente quais partes do sistema passaram ou falharam.

Configuração do Ambiente de Execução dos Comandos

Para instalar o Pytest, basta utilizar o gerenciador de pacotes do Python, o pip. Antes de tudo, é importante ter o Python corretamente instalado e configurado no sistema. Com isso pronto, abra o terminal (CMD, PowerShell ou Linux) e execute o comando.

Figura 1 - Instalação do Pytest



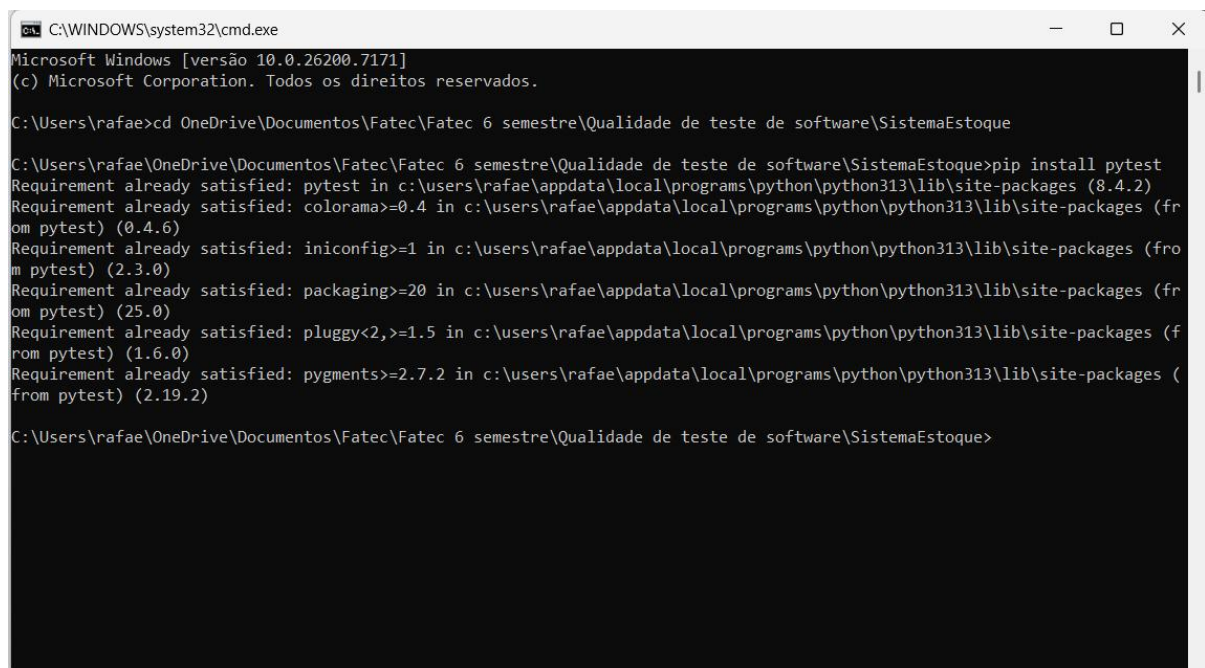
```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [versão 10.0.26200.7171]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\rafae>cd OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque

C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>pip install pytest
```

Fonte: próprio autor

Figura 2 - Instalação completa do Pytest



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [versão 10.0.26200.7171]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\rafae>cd OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque

C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>pip install pytest
Requirement already satisfied: pytest in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (8.4.2)
Requirement already satisfied: colorama>=0.4 in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=1 in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=20 in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (from pytest) (25.0)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\rafae\appdata\local\programs\python\python313\lib\site-packages (from pytest) (2.19.2)

C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>
```

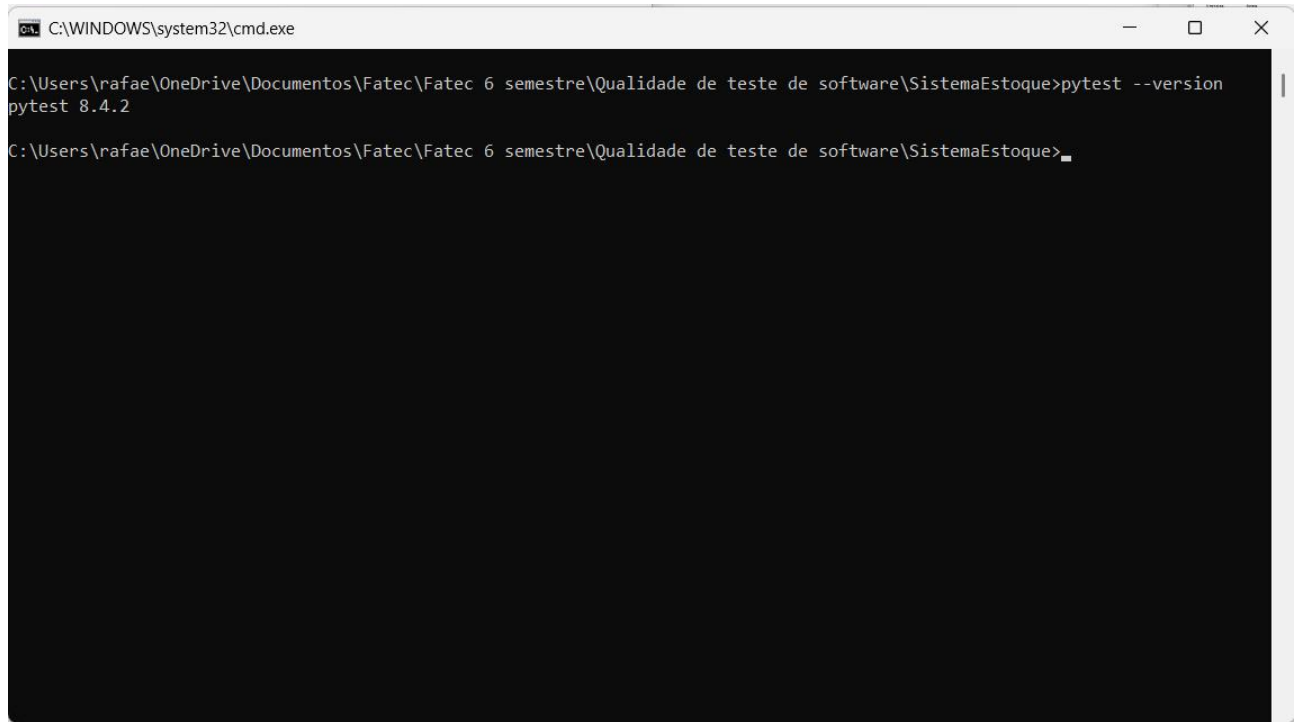
Fonte: próprio autor

Na primeira e na segunda figura o comando baixa e instala automaticamente o Pytest e suas dependências.

Para confirmar que a instalação foi concluída, você pode verificar a versão instalada.

Para ver a versão

Figura 3 - Versão Instalada.

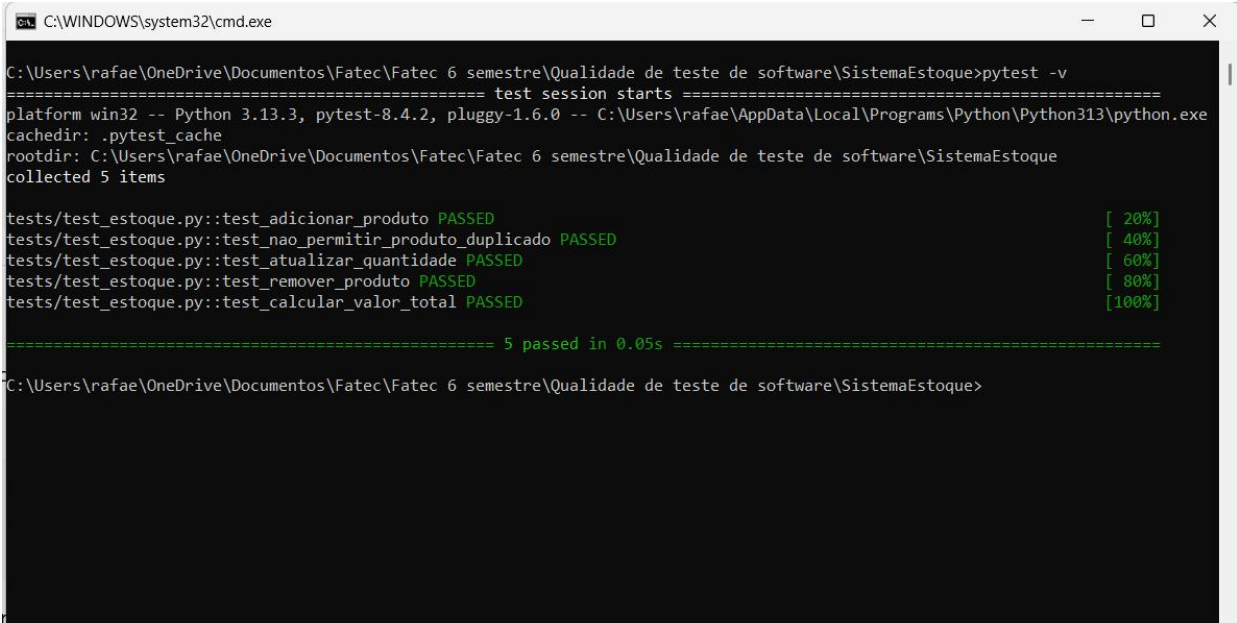


```
C:\WINDOWS\system32\cmd.exe
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>pytest --version
pytest 8.4.2
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>_
```

Fonte: próprio autor

Como do que verifica a versão, significa que o Pytest está instalado e pronto para ser usado. A partir daí, qualquer arquivo pode ser testado.

Figura 4 - Comando Pytest -v

A terminal window titled 'C:\WINDOWS\system32\cmd.exe' showing the output of the command 'pytest -v'. The output includes the Python version (3.13.3), pytest version (8.4.2), and the path to the test files. It lists five tests from 'tests/test_estoque.py' and shows their progress with percentage indicators: [20%], [40%], [60%], [80%], and [100%]. All tests are marked as 'PASSED'. The summary line shows '5 passed in 0.05s'.

```
C:\WINDOWS\system32\cmd.exe
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>pytest -v
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\rafae\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque
collected 5 items

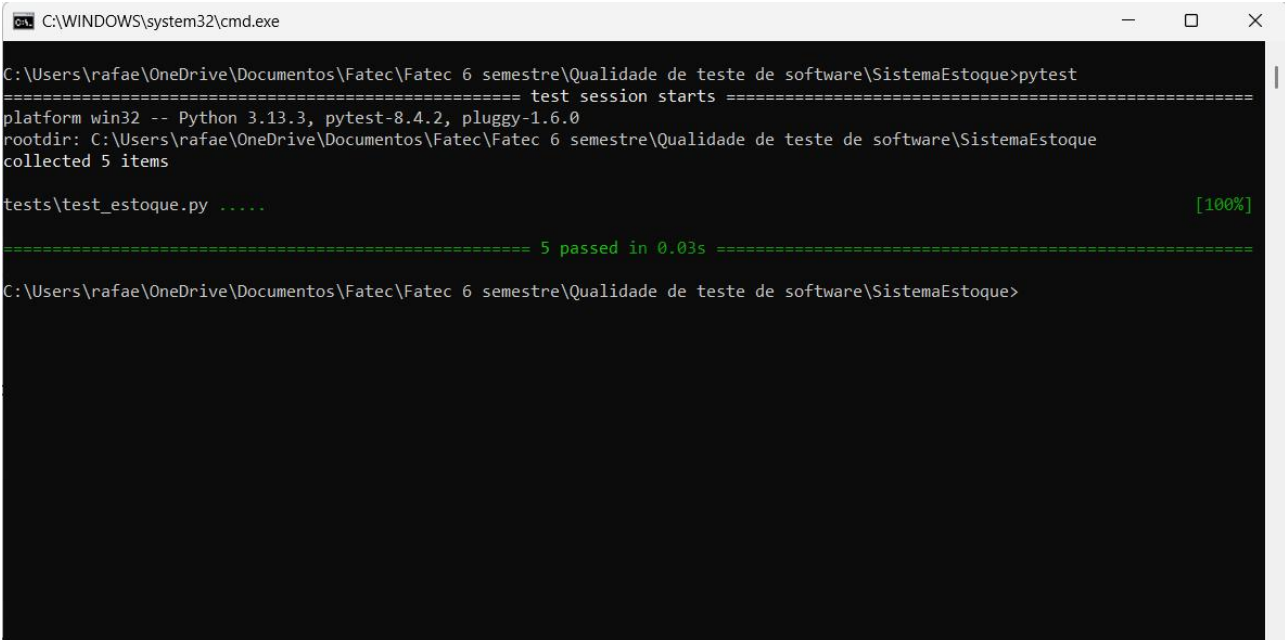
tests/test_estoque.py::test_adicionar_produto PASSED [ 20%]
tests/test_estoque.py::test_nao_permitir_produto_duplicado PASSED [ 40%]
tests/test_estoque.py::test_atualizar_quantidade PASSED [ 60%]
tests/test_estoque.py::test_remover_produto PASSED [ 80%]
tests/test_estoque.py::test_calcular_valor_total PASSED [100%]

===== 5 passed in 0.05s =====
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>
```

Fonte: próprio autor

Esse comando roda os testes com mais detalhes. Mostra cada teste sendo executado e o resultado (PASSED, FAILED, etc.).

Figura 5 - Comando Pytest

A terminal window titled 'C:\WINDOWS\system32\cmd.exe' showing the output of the command 'pytest'. The output is more concise than Figure 4, showing the test session start, platform, Python version, pytest version, rootdir, and the number of collected items (5). It shows the test file 'tests\test_estoque.py' being executed with a progress indicator [100%]. The summary line shows '5 passed in 0.03s'.

```
C:\WINDOWS\system32\cmd.exe
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>pytest
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque
collected 5 items

tests\test_estoque.py ..... [100%]

===== 5 passed in 0.03s =====
C:\Users\rafae\OneDrive\Documentos\Fatec\Fatec 6 semestre\Qualidade de teste de software\SistemaEstoque>
```

Fonte: próprio autor

Esse comando roda todos os testes da pasta. Executa todos os arquivos que começam com test_*.py ou terminam com *_test.py.

Validação de cada função.

Figura 6 - Teste Adicionar produto.

```
def test_adicionar_produto():
    e = Estoque()
    e.adicionar_produto("Arroz", 10, 5)
    assert "Arroz" in e.produtos
```

Fonte: próprio autor

Testa se um produto foi realmente adicionado.

Figura 7 - Teste Não permitir duplicidade

```
def test_nao_permitir_produto_duplicado():
    e = Estoque()
    e.adicionar_produto("Feijão", 8, 3)
    with pytest.raises(ValueError):
        e.adicionar_produto("Feijão", 8, 3)
```

Fonte: próprio autor

Testa se o sistema NÃO permite adicionar o mesmo produto duas vezes

Figura 8 - Teste Atualizar Quantidade

```
def test_atualizar_quantidade():
    e = Estoque()
    e.adicionar_produto("Macarrão", 5, 2)
    e.atualizar_quantidade("Macarrão", 10)
    assert e.produtos["Macarrão"]["quantidade"] == 10
```

Fonte: próprio autor

Testa se a quantidade foi alterada corretamente.

Figura 9 - Teste Remover Produto

```
def test_remover_produto():
    e = Estoque()
    e.adicionar_produto("Café", 15, 4)
    e.remover_produto("Café")
    assert "Café" not in e.produtos
```

Fonte: próprio autor

Testa se o produto foi removido.

Figura 10 -Teste Calcula Valor Total

```
def test_calcular_valor_total():  
    e = Estoque()  
    e.adicionar_produto("Açúcar", 4, 5)  
    e.adicionar_produto("Sal", 2, 3)  
    assert e.calcular_valor_total() == (4*5 + 2*3)
```

Fonte: próprio autor

Testa se o cálculo total do valor do estoque está correto.

Figura 11 - Comandos Pytest.

COMANDO	PARA O QUE SERVE
pytest	Roda todos os testes
pytest -v	Roda testes com detalhes (verbose)
pytest -s	Mostra prints durante a execução
pytest -k nome	Filtra e roda apenas testes que contenham "nome"
pytest arquivo.py	Roda os testes de um arquivo específico
pytest -x	Para a execução no primeiro erro
pytest --cov	Mostra a cobertura de código dos testes
pytest --cov-report=html	Gera relatório de cobertura em HTML
pytest -p no:warnings	Ocultar mensagens de warnings

Nessa imagem mostro Principais comandos do Pytest e para o que ele serve.

Considerações Finais

Trabalhar com o Pytest foi uma experiência importante para entender melhor como os testes automatizados ajudam no desenvolvimento de um software. Mesmo aplicando a ferramenta em um projeto pequeno, consegui ver na prática como os testes deixam o código mais seguro e evitam erros que poderiam surgir depois. Ao escrever os testes, pude observar o comportamento de cada parte do sistema e isso me deu mais confiança no que eu estava desenvolvendo.

O Pytest se mostrou simples de usar e muito eficiente. Os comandos são diretos, os relatórios são fáceis de entender e a ferramenta realmente ajuda a encontrar problemas rapidamente. Isso fez com que o processo de desenvolvimento ficasse mais organizado e menos cansativo, porque qualquer mudança que eu fazia já podia ser testada automaticamente.

Ao final deste trabalho, percebo que testar não é apenas um passo extra, mas uma parte essencial do desenvolvimento, mesmo em projetos pequenos. A experiência reforçou a importância de aplicar boas práticas desde o início, porque isso evita retrabalho e facilita a manutenção no futuro. De forma geral, usar o Pytest contribuiu bastante para o meu aprendizado e me mostrou como uma ferramenta simples pode melhorar muito a qualidade de um sistema.

Bibliografia

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>.

Acesso em: 18 novembro 2025.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software.

Acesso em: 18 novembro 2025.

A ferramenta PHPUnit

Rian de Souza Scavazza
riansouzascavazza@gmail.com
rian.scavazza@fatec.sp.gov.br

Resumo

Este trabalho mostra a importância dos testes automatizados no desenvolvimento de software, com foco principal no PHPUnit. O trabalho passa pela configuração inicial e instalação com o Composer e explica conceitos práticos como o padrão AAA (Arrange, Act, Assert). Além disso, detalha a estrutura de pastas ideal para projetos em PHP puro e dentro do framework Laravel, destacando as diferenças entre testes unitários e de funcionalidade. Por fim, o capítulo traz exemplos reais de código e um sistema de CRUD.

Palavras-chave: PHPUnit, Testes Automatizados, Laravel, Qualidade de Software, TDD.

Introdução

No ciclo de vida do desenvolvimento, é fundamental garantir que o código funcione como o planejado e que alterações futuras não quebrem o que já existe. O PHPUnit é a ferramenta mais popular para isso no mundo PHP. Ele oferece uma estrutura baseada na classe TestCase, o que permite escrever testes automatizados rápidos de executar via linha de comando.

O ponto principal desses testes são os Asserts (afirmações). Eles são métodos que conferem se o resultado que o código entregou bate com o resultado esperado. Exemplos comuns são verificar se dois números são iguais (assertEquals) ou se uma condição é verdadeira (assertTrue).

Para organizar a escrita de um teste, a prática mais comum é o AAA:

Arrange (Organizar): Prepara o ambiente, cria os objetos e define os dados de entrada.

Act (Agir): Executa a função ou método que precisa ser testado.

Assert (Afirmar): Confere se o resultado da ação está correto.

Desenvolvimento

1. Instalando o PHPUnit

A instalação do PHPUnit acontece através do Composer, o gerenciador de dependências do PHP. Como os testes só são úteis durante a criação do software, a instalação deve ser feita apenas no ambiente de desenvolvimento (--dev), sem pesar na versão final de produção.

O comando para instalar na pasta do projeto é:

Figura 01 - Comando de instalação do PHPUnit

```
composer require -dev phpunit/phpunit
```

Fonte: próprio autor

Depois de rodar o comando, o PHPUnit entra no arquivo composer.json e as bibliotecas ficam disponíveis na pasta vendor.

2. Estrutura de Pasta e Convenções

Organizar bem os arquivos de teste facilita a manutenção do projeto a longo prazo.

2.1. Projetos PHP Puro

A recomendação da comunidade é separar o código-fonte (app ou src) dos testes. Um padrão simples é criar uma classe de teste com o mesmo nome da classe original, adicionando o final Test (por exemplo, para a classe Calculadora, cria-se CalculadoraTest).

Figura 02 - Estruturas de pastas com PHP

```
MeuProjetoPHP/  
├── app/  
│   └── Calculadora.php  
├── tests/  
│   └── CalculadoraTest.php  
├── vendor/  
├── composer.json  
└── phpunit.xml
```

Fonte: próprio autor

2.2. Projetos Laravel

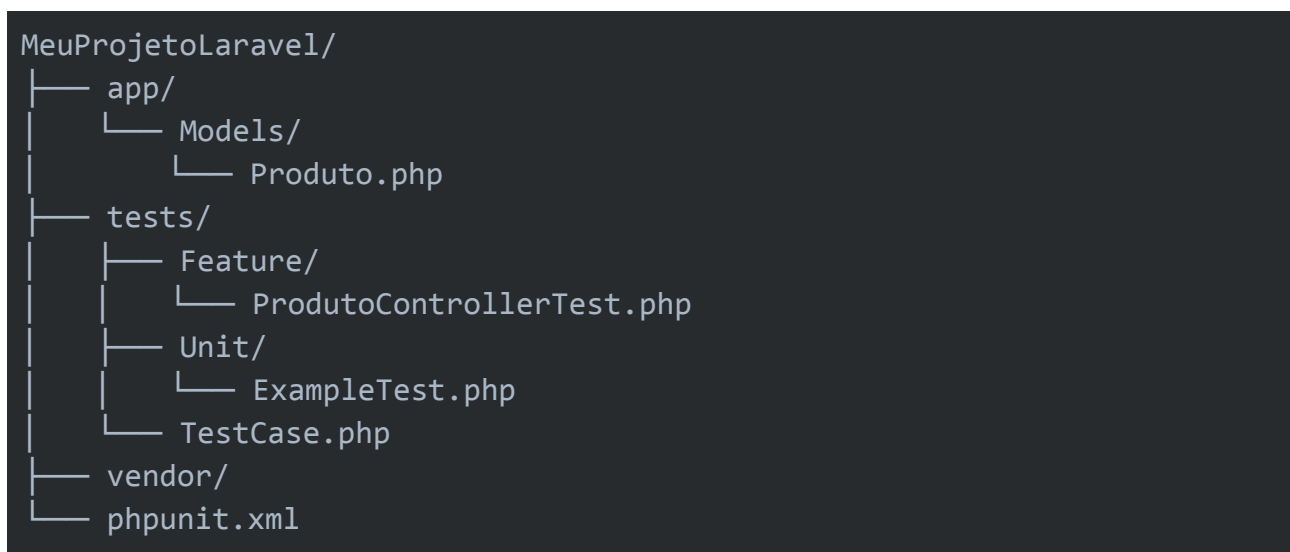
O Laravel já traz o PHPUnit configurado. Dentro da pasta tests, ele divide os arquivos em dois grupos:

Unit (Testes Unitários): Para testar pedaços isolados de código, como uma classe ou método específico, sem tocar no banco de dados.

Feature (Testes de Funcionalidade): Para testar recursos completos, simulando, por exemplo, uma requisição HTTP que passa por várias camadas do sistema.

Para criar um novo teste, o comando é “*php artisan make:test NomeDoTest*”

Figura 03 - Estruturas de pastas com Laravel



Fonte: próprio autor

3. Exemplo prático: Teste Unitário (Calculadora)

Este exemplo mostra um Teste Unitário puro na classe Calculadora. O código abaixo verifica operações matemáticas e também se o sistema bloqueia corretamente uma divisão por zero.

Arquivo: *tests/CalculadoraTest.php*

Figura 04 - Testes no arquivo CalculadoraTest.php

```
<?php  
use PHPUnit\Framework\TestCase;  
use App\Calculadora;  
use Exception;  
class CalculadoraTest extends TestCase
```

```

{
    public function testSomaCorretamente()
    {
        $calculadora = new Calculadora();
        $this->assertEquals(3, $calculadora->somar(1, 2));
        $this->assertEquals(15, $calculadora->somar(10, 5));
        $this->assertEquals(0, $calculadora->somar(-5, 5));
    }

    public function testDeveSubtrairCorretamente()
    {
        $calculadora = new Calculadora();
        $this->assertEquals(5, $calculadora->subtrair(10, 5));
        $this->assertEquals(-5, $calculadora->subtrair(5, 10));
    }

    public function testDeveMultiplicarCorretamente()
    {
        $calculadora = new Calculadora();
        $this->assertEquals(9, $calculadora->multiplicar(3, 3));
        $this->assertEquals(0, $calculadora->multiplicar(5, 0));
    }

    public function testDeveDividirCorretamente()
    {
        $calculadora = new Calculadora();
        $this->assertEquals(5, $calculadora->dividir(10, 2));
        $this->assertEquals(2.5, $calculadora->dividir(5, 2));
    }

    public function testDeveLancarExcecaoAoDividirPorZero()
    {
        $calculadora = new Calculadora();
        $this->expectException(Exception::class);
        $this->expectExceptionMessage("Não é possível dividir por
zero");
        $calculadora->dividir(10, 0);
    }
}

```

Fonte: próprio autor

4. Exemplo prático: Teste de Funcionalidade (CRUD de produtos com Laravel)

O exemplo a seguir é um Feature Test dentro do Laravel. Ele usa o recurso RefreshDatabase para garantir que o banco de dados esteja limpo antes de cada teste e simula ações de um usuário (GET, POST, PUT, DELETE).

Arquivo: tests/Feature/ProdutoControllerTest.php

Figura 04 - Testes no arquivo ProdutoControllerTest.php

```
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;
use App\Models\Produto;

class ProdutoControllerTest extends TestCase
{
    use RefreshDatabase;

    public function test_lista_produto_com_sucesso(): void
    {
        Produto::factory()->create([
            'nome' => 'Produto Teste A',
            'ean' => '1234567890123'
        ]);

        $response = $this->get('/');

        $response->assertStatus(200);
        $response->assertSee('Produto Teste A');
    }

    public function test_cria_um_novo_produto_com_sucesso(): void
    {
        $dadosDoProduto = [
            'nome' => 'Produto Novo',
            'preco_bruto' => 12.00,
            'preco_liquido' => 10.50,
            'quantidade' => 100,
            'ean' => '9876543210987',
        ];
    }
}
```

```

        $response = $this->post('/create', $dadosDoProduto);

        $response->assertRedirect('/');
        $this->assertDatabaseHas('produtos', $dadosDoProduto);
    }

    public function test_atualiza_um_produto_com_sucesso(): void
    {
        $produto = Produto::factory()->create();
        $dadosAtualizados = [
            'nome' => 'Produto Atualizado',
            'preco_liquido' => 99.99,
            'quantidade' => 50,
            'ean' => $produto->ean,
        ];

        $response = $this->put('/') . $produto->id . '/update',
        $dadosAtualizados);

        $response->assertRedirect('/');
        $this->assertDatabaseHas('produtos', $dadosAtualizados);
    }

    public function test_deleta_um_produto_com_sucesso(): void
    {
        $produto = Produto::factory()->create();

        $response = $this->delete('/') . $produto->id . '/delete';

        $response->assertRedirect('/');
        $this->assertModelMissing($produto);
    }
}

```

Fonte: próprio autor

Considerações Finais

Implementar testes com PHPUnit, seja em projetos puros ou no Laravel, aumenta muito a confiança no software. Usar a estrutura de pastas correta e entender a diferença entre testes unitários e de funcionalidade cria uma rede de segurança que ajuda na manutenção e evita bugs futuros. Dominar os métodos de afirmação (asserts) e o ciclo AAA é o passo principal para construir aplicações profissionais e robustas.

Bibliografia

BERGMANN, Sebastian. *PHPUnit Manual*. Disponível em: <https://docs.phpunit.de/>. Acesso em: 25 nov. 2025.

STAUFFER, Matt. *Laravel: Up & Running: A Framework for Building Modern PHP Apps*. 2ª ed. O'Reilly Media, 2019.

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2024.

CARDOSO, Andre; ANICHE, Mauricio. *Test-Driven Development: Teste e Design no Mundo Real com PHP*. Casa do Código, 2015.

SARAIVA JUNIOR, Orlando. *ebook_qualidade_teste_software*. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

Testify: Biblioteca para Golang

Pietra Alves

pietraaalvess@gmail.com

pietra.alves01@fatec.sp.gov.br

Resumo

Este trabalho apresenta o estudo e aplicação da biblioteca Testify, um conjunto de pacotes voltado para simplificar e aprimorar a criação de testes automatizados em projetos desenvolvidos com a linguagem Go. O estudo tem como objetivo explorar os componentes ofertados pela ferramenta, demonstrando como ela contribui para testes mais claros, expressivos e eficientes. A metodologia adotada envolveu pesquisas teóricas, análise da documentação oficial e implementação prática de testes unitários utilizando os módulos assert, require, mock e suite. Os resultados demonstraram que o uso da biblioteca reduz a verbosidade do código de teste, melhora a interpretação de erros durante as execuções e facilita a criação de cenários complexos por meio de simulações e organização modular. Com isso, conclui-se que a ferramenta Testify se destaca como um recurso robusto e confiável para elevar a qualidade do software, favorecendo boas práticas e agilizando o processo de validação de funcionalidades.

Palavras-chave: Testify. Golang. Testes de Software. Automação de testes.

Introdução

A realização de testes é uma etapa essencial no processo de desenvolvimento de software, garantindo que funcionalidades atendam aos requisitos definidos e prevenindo falhas na produção. No contexto da linguagem Go, a biblioteca Testify surge como uma solução prática e estruturada para criação de testes automatizados, atuando como complemento ao pacote de testes nativo da linguagem. Este trabalho tem como objetivo examinar os principais módulos oferecidos pela ferramenta, destacando como sua aplicação pode aprimorar a qualidade e legibilidade do código de teste. Para isso, foi adotada uma metodologia que reúne estudo teórico da documentação oficial e experimentação prática por meio da implementação de testes unitários em um projeto real desenvolvido em Go. O conteúdo deste capítulo está organizado para apresentar os conceitos da ferramenta, seus benefícios, exemplos práticos de implementação e aprendizados adquiridos durante a execução dos testes.

Desenvolvimento

A biblioteca Testify fornece uma coleção de pacotes que aumentam a produtividade e a clareza na escrita de testes em Go, mantendo compatibilidade com o comando nativo **go test**. A instalação da ferramenta é realizada por meio do gerenciador de pacotes:

Instalação

```
go get github.com/stretchr/testify
```

Isso disponibilizará os seguintes pacotes para você:

```
github.com/stretchr/testify/assert  
github.com/stretchr/testify/require  
github.com/stretchr/testify/mock  
github.com/stretchr/testify/suite
```

O pacote **assert** é utilizado para comparar valores e validar comportamentos sem interromper o fluxo de execução quando uma asserção falha. Ele proporciona mensagens de erro mais descritivas e facilita a leitura dos testes. Já o pacote **require** oferece as mesmas funcionalidades, porém interrompe o teste imediatamente quando ocorre uma falha, sendo indicado para validações críticas.

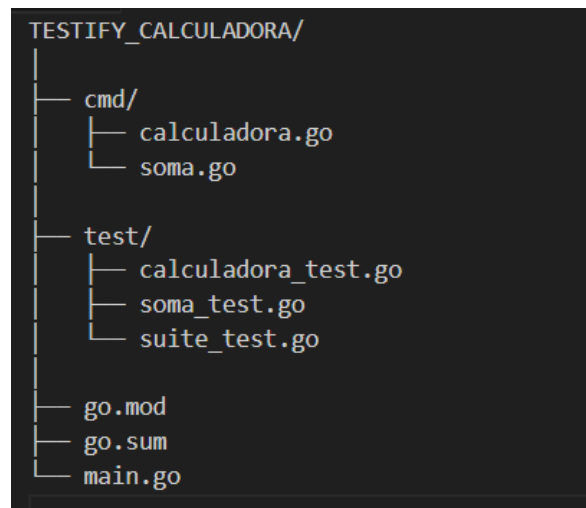
Um exemplo prático de uso desses módulos pode ser observado na implementação dos testes da função `Soma()` e `Divide()`, que validam resultados esperados e tratamento de erro em divisão por zero. Em um cenário mais avançado, o pacote **mock** permite criar dependências simuladas, como repositórios e APIs, possibilitando o teste de componentes isoladamente. No estudo realizado, esse recurso foi utilizado para testar a função `DobrarValorBase` por meio de um repositório simulado, garantindo total controle sobre as respostas retornadas durante a execução dos testes.

Além disso, o módulo **suite** oferece uma forma de organizar diversos testes em estruturas, permitindo inicializações e finalizações automáticas antes de cada

execução. Tal abordagem torna os testes mais profissionais e escaláveis, principalmente em projetos de maior porte.

Exemplos de como usar os pacotes no Go:

Figura 1 – Diagrama de estrutura de diretórios



Explicação da estrutura

cmd/

Contém os arquivos principais da lógica da aplicação.

- calculadora.go → Implementa a estrutura Calculadora e depende do repositório.
- soma.go → Implementa as funções Soma e Divide.

test/

Local destinado exclusivamente aos testes escritos com Testify.

- calculadora_test.go → Testes relacionados à Calculadora, incluindo mocks.
- soma_test.go → Testes das funções Soma e Divide.
- suite_test.go → Testes organizados utilizando o módulo suite do Testify.

Arquivos de configuração e execução

- go.mod → Declara o módulo Go e dependências.
- go.sum → Registra checksums das dependências.
- main.go → Arquivo principal usado para rodar a demonstração das funções.

Figura 2 – Pacote assert continua o teste mesmo se falhar.

```
package main

import "errors"

var ErrDivisaoPorZero = errors.New("erro divisão por zero")

func Soma(a, b int) int {
    return a + b
}

func Divide(a, b int) (int, error) {
    if b == 0 {
        return 0, ErrDivisaoPorZero
    }
    return a / b, nil
}
```

Figura 3 – Pacote require interrompe o teste imediatamente ao falhar.

```
package main

import (
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestSomaComAssert(t *testing.T) {
    resultado := Soma(2, 3)
    esperado := 5
    assert.Equal(t, esperado, resultado, "Deve somar corretamente")
    assert.NotEqual(t, 6, resultado, "Não deve ser incorreto")
}

func TestDivideComRequire(t *testing.T) {
    resultado, err := Divide(10, 2)
    require.NoError(t, err, "Não deve retornar erro")
    require.Equal(t, 5, resultado, "Deve dividir corretamente")
}

func TestDividePorZero(t *testing.T) {
    _, err := Divide(4, 0)
    require.Error(t, err)
    require.EqualError(t, err, ErrDivisaoPorZero.Error())
}
```

Figura 4 – Pacote mock simula o comportamento de uma dependência (como um banco de dados ou API)

```
calculadora.go X
calculadora.go
1  package main
2
3  type Repositorio interface {
4      BuscarValorBase() int
5  }
6
7  type Calculadora struct {
8      repo Repositorio
9  }
10
11 func NovaCalculadora(repo Repositorio) *Calculadora {
12     return &Calculadora{repo: repo}
13 }
14
15 func (c *Calculadora) DobrarValorBase() int {
16     base := c.repo.BuscarValorBase()
17     return base * 2
18 }
```

Figura 5 – calculadora_test.go (teste do mock)

```
package main

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/mock"
)

// Cria um mock do repositório
type MockRepositorio struct {
    mock.Mock
}

func (m *MockRepositorio) BuscarValorBase() int {
    args := m.Called()
    return args.Int(0)
}

func TestDobrarValorBase(t *testing.T) {
    repo := new(MockRepositorio)
    repo.On("BuscarValorBase").Return(10)

    calc := NovaCalculadora(repo)
    resultado := calc.DobrarValorBase()

    assert.Equal(t, 20, resultado)
    repo.AssertExpectations(t)
}
```

Figura 6 – As suites permitem agrupar vários testes com setup comum, útil em sistemas maiores

```
GO suite_test.go
1  package main
2
3  import (
4      "testing"
5
6      "github.com/stretchr/testify/suite"
7  )
8
9  type MathSuite struct {
10     suite.Suite
11     valorA int
12     valorB int
13 }
14
15 func (s *MathSuite) SetupTest() {
16     s.valorA = 6
17     s.valorB = 3
18 }
19
20 func (s *MathSuite) TestSoma() {
21     s.Equal(9, Soma(s.valorA, s.valorB))
22 }
23
24 func (s *MathSuite) TestDivide() {
25     resultado, err := Divide(s.valorA, s.valorB)
26     s.NoError(err)
27     s.Equal(2, resultado)
28 }
29
30 func TestMathSuite(t *testing.T) {
31     suite.Run(t, new(MathSuite))
32 }
33
```

Figura 7 – Teste rodando com falha

```
pietraalves@NAPP-0178:~/Documentos/faculdade/Testify_calculadora$ go test -v
=== RUN   TestDobrarValorBase
--- PASS: TestDobrarValorBase (0.00s)
=== RUN   TestSomaComAssert
soma_test.go:14:
Error Trace:   /home/pietraalves/Documentos/faculdade/Testify_calculadora/soma_test.go:14
Error:         Should not be: 5
Test:          TestSomaComAssert
Messages:      Não deve ser incorreto
--- FAIL: TestSomaComAssert (0.00s)
=== RUN   TestDivideComRequire
--- PASS: TestDivideComRequire (0.00s)
=== RUN   TestDividePorZero
--- PASS: TestDividePorZero (0.00s)
=== RUN   TestMathSuite
=== RUN   TestMathSuite/TestDivide
=== RUN   TestMathSuite/TestSoma
--- PASS: TestMathSuite (0.00s)
--- PASS: TestMathSuite/TestDivide (0.00s)
--- PASS: TestMathSuite/TestSoma (0.00s)
FAIL
exit status 1
FAIL    testify-demo    0.005s
```

Figura 8 – Teste rodando sem falhas

```
pietraalves@NAPP-0178:~/Documentos/faculdade/Testify_calculadora$ go test
=== RUN   TestDobrarValorBase
--- PASS: TestDobrarValorBase (0.00s)
=== RUN   TestSomaComAssert
--- PASS: TestSomaComAssert (0.00s)
=== RUN   TestDivideComRequire
--- PASS: TestDivideComRequire (0.00s)
=== RUN   TestDividePorZero
--- PASS: TestDividePorZero (0.00s)
=== RUN   TestMathSuite
=== RUN   TestMathSuite/TestDivide
=== RUN   TestMathSuite/TestSoma
--- PASS: TestMathSuite (0.00s)
--- PASS: TestMathSuite/TestDivide (0.00s)
--- PASS: TestMathSuite/TestSoma (0.00s)
PASS
ok      testify-demo    0.004s
```

Conforme observado na figura, todos os testes foram executados com sucesso, evidenciando que a aplicação da biblioteca Testify contribuiu para validações claras e assertivas em cada cenário desenvolvido.

Considerações Finais

O estudo realizado permitiu compreender, na prática, como a biblioteca Testify contribui significativamente para o aumento da qualidade no desenvolvimento de software com a linguagem Go. Os testes implementados demonstraram que seus módulos tornam o processo de validação mais simples, expressivo e eficiente, reduzindo a probabilidade de erros e facilitando a manutenção do código. Pacotes como assert, require, mock e suite ampliam as capacidades do pacote nativo de testes da linguagem, permitindo desde verificações simples até tratamentos avançados com simulações e organização modular. Como trabalhos futuros, recomenda-se explorar a integração do Testify com pipelines de CI/CD e testes automatizados em aplicações distribuídas, ampliando ainda mais seu uso em cenários reais de engenharia de software.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2025.

TESTIFY. Documentação oficial. Disponível em: <https://github.com/stretchr/testify/>. Acesso em: 11 nov. 2025.

GO. Documentação oficial da linguagem. Disponível em: <https://go.dev/doc/>. Acesso em: 11 nov. 2025.

Unittest

*Isabela Batista Machado de Almeida
isabelabalmeida0123@gmail.com
isabela.almeida4@fatec.sp.gov.br*

Resumo

Este capítulo discute o uso da biblioteca unittest, recurso nativo da linguagem Python voltado para a criação de testes unitários estruturados. O propósito central é evidenciar como a prática de testes automatizados contribui para elevar a qualidade do software, tornando o código mais confiável, reduzindo erros e apoiando ciclos de desenvolvimento orientados por TDD (Test-Driven Development). A metodologia adotada combinou pesquisa teórica sobre os fundamentos dos testes unitários com a análise das funcionalidades oferecidas pelo módulo unittest, além da implementação prática de casos de teste aplicados a pequenos exemplos de código. Nesse percurso, foram explorados recursos essenciais da ferramenta, como a definição de classes de teste, o uso de métodos de preparação e finalização, e a aplicação de diferentes tipos de assertivas. Os resultados indicam que, mesmo com sua proposta simples e abordagem tradicional, o unittest oferece uma estrutura sólida e bem organizada, adequada tanto para fins acadêmicos quanto para projetos corporativos. Conclui-se que sua utilização favorece a padronização dos testes e reforça a cultura de qualidade no desenvolvimento de software, configurando-se como uma alternativa acessível para estudantes e profissionais que desejam iniciar no campo da automação de testes

Palavras-chaves: ferramenta, biblioteca, automatizados, unitários.

Introdução

A qualidade de software é um dos pilares centrais no desenvolvimento de sistemas contemporâneos, exigindo práticas que assegurem confiabilidade, manutenção contínua e redução de falhas ao longo de todo o ciclo de vida do produto. Entre essas práticas, os testes unitários ocupam papel estratégico, pois permitem validar pequenas partes do código de forma isolada, garantindo que cada componente cumpra sua função corretamente. Nesse cenário, este capítulo dedica-se ao estudo da biblioteca unittest,

módulo nativo do Python voltado para a organização e automação de testes unitários de maneira sistemática.

O escopo do trabalho concentra-se na apresentação dos conceitos fundamentais relacionados aos testes unitários, na análise das principais funcionalidades oferecidas pelo unittest e na demonstração prática de sua aplicação em projetos simples. O objetivo geral é compreender de que forma a automatização de testes contribui para elevar a qualidade do software. Como objetivos específicos, destacam-se: identificar as características da ferramenta, examinar sua estrutura de funcionamento e relatar a experiência obtida durante sua utilização.

A escolha do tema se justifica pela crescente relevância dos testes automatizados no mercado de tecnologia e pela facilidade de adoção do unittest, já que o módulo integra o ecossistema Python sem demandar configurações adicionais. A metodologia utilizada envolveu pesquisa bibliográfica, análise da documentação oficial e experimentação prática por meio da implementação de casos de teste elaborados.

A organização do capítulo segue três etapas principais: primeiro, é apresentado o referencial teórico e o contexto dos testes unitários e da biblioteca unittest; em seguida, descreve-se o desenvolvimento do estudo, incluindo exemplos e reflexões sobre o processo de aprendizado; por fim, são expostas as considerações finais, com destaque para as conclusões alcançadas e possíveis caminhos para trabalhos futuros.

Visão Geral

O unittest é o framework oficial de testes unitários do Python, integrado à sua biblioteca padrão. Ele foi projetado para simplificar a criação e execução de testes automatizados que verificam unidades de código de forma isolada. A partir da classe base TestCase, é possível definir casos de teste que incluem métodos de preparação (setup), finalização (teardown) e múltiplos testes independentes, cada um responsável por validar uma funcionalidade específica do software.

Além disso, o módulo disponibiliza recursos para organizar testes em suítes (TestSuite), executar automaticamente via linha de comando, realizar a descoberta de testes em diretórios e gerar relatórios estruturados com os resultados de sucesso, falhas e erros. Essa abordagem orientada a objetos e consistente permite que tanto iniciantes quanto profissionais mantenham seus testes organizados, favorecendo a manutenção do

código, a identificação precoce de problemas e a refatoração segura.

Breve histórico

O conceito de testes unitários antecede o próprio Python, tendo surgido nas primeiras décadas da engenharia de software como prática formal para validar componentes individuais antes da integração em sistemas maiores.

No contexto da linguagem Python, o unittest foi desenvolvido com inspiração no estilo xUnit, adotado por frameworks como o JUnit, do Java, mas adaptado à sintaxe e às características próprias do Python. Com o tempo, o módulo evoluiu e passou a oferecer funcionalidades mais completas, como a detecção automática de testes em diretórios, suporte para agrupamento e organização em suítes, execução por linha de comando com diferentes níveis de detalhamento e mecanismos de configuração e limpeza de contexto antes e depois dos testes.

Essa evolução, somada ao fato de o unittest estar disponível nativamente na biblioteca padrão, consolidou-o como referência em diversos projetos. Sua simplicidade e padronização o tornaram especialmente útil em ambientes acadêmicos, educacionais e corporativos, onde a clareza e a confiabilidade dos testes são prioridades.

Ambiente de testes

No contexto do unittest, o chamado “ambiente de teste” corresponde ao conjunto de práticas e recursos que permitem estruturar e executar verificações automatizadas de forma organizada. Entre os principais elementos, destacam-se:

- Scripts de teste: normalmente escritos em arquivos .py separados ou em módulos específicos.
- Classe base unittest.TestCase: utilizada para definir cada teste como método cujo nome inicia com test..., garantindo que o test runner reconheça e execute o método como um caso de teste.
- Métodos de preparação e limpeza: setUp() e tearDown() permitem configurar pré-condições ou liberar recursos após cada execução, quando necessário.
- Agrupamento em suítes (TestSuite): possibilita reunir testes relacionados em

coleções organizadas, facilitando a execução conjunta.

- Execução por linha de comando: comandos como `python -m unittest` ou `python -m unittest discover` permitem rodar os testes e realizar a descoberta automática de arquivos de teste no diretório do projeto.
- Relatórios de execução: apresentam informações sobre testes bem-sucedidos, falhas, erros inesperados ou ignorados. A saída pode ser ajustada com opções como `-v`, que aumenta o nível de detalhamento.

Esse ambiente estruturado favorece a automação e a padronização dos testes, além de integrar-se facilmente a pipelines de integração contínua (CI). Como não depende de ferramentas externas, basta ter o Python instalado para usufruir de todos os recursos oferecidos pelo unittest.

Principais Conceitos e Estruturas do unittest

Para oferecer suporte a testes unitários eficazes e bem organizados, o unittest estabelece alguns componentes fundamentais:

- **TestCase**: classe base que deve ser estendida para a criação de casos de teste individuais. Cada método cujo nome começa com `test_...` representa um teste específico.
- **Assertions** (assertivas de teste): conjunto de métodos como `assertEqual()`, `assertTrue()`, `assertFalse()` e `assertRaises()`, utilizados para verificar resultados esperados ou exceções previstas.
- **setUp / tearDown**: métodos opcionais executados antes e depois de cada teste, úteis para preparar o ambiente ou liberar recursos.
- **TestSuite**: estrutura que reúne múltiplos casos de teste ou outras suítes, permitindo a execução organizada de grandes conjuntos de testes.
- **Test Runner**: componente responsável por executar os testes e coletar os resultados (sucesso, falha ou erro). O unittest disponibiliza, por padrão, um text runner que imprime relatórios diretamente no console.
- **Test Discovery** (descoberta automática de testes): funcionalidade que permite localizar automaticamente módulos de teste em um diretório, dispensando a necessidade de chamar cada teste manualmente.

Esses elementos constituem a base para a elaboração de testes consistentes, organizados e de fácil manutenção, favorecendo a padronização e a confiabilidade no

processo de desenvolvimento de software.

Desenvolvimento

Para o desenvolvimento deste projeto, foi realizada uma divisão em duas etapas principais. A primeira corresponde à pesquisa bibliográfica e documental, com foco no estudo da documentação oficial do unittest, a fim de compreender seus conceitos, classes e funcionamento. A segunda etapa consistiu na aplicação prática, envolvendo a criação de pequenos trechos de código em Python acompanhados da escrita de testes com unittest. Essa prática buscou validar funcionalidades específicas e demonstrar, de forma concreta, como os testes unitários operam no cotidiano do desenvolvimento.

O objetivo dessa abordagem foi evidenciar a viabilidade real da ferramenta e destacar seus benefícios, como o aumento da qualidade, a confiabilidade do código e a capacidade de detectar regressões. Como ponto de partida, é importante compreender a estrutura mínima de um teste, que se organiza da seguinte forma:

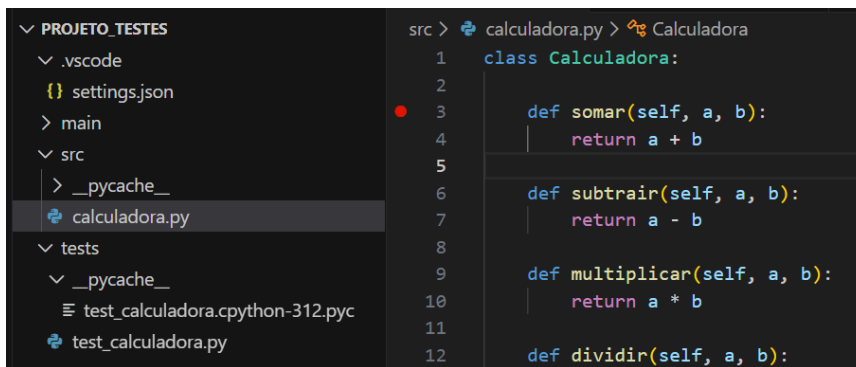
```
import unittest

class TesteSimples(unittest.TestCase):
    def test_soma(self):
        self.assertEqual(2 + 2, 4)

if __name__ == "__main__":
    unittest.main()
```

Seguindo o roteiro de demonstração, inicialmente foi criada a estrutura de diretórios do projeto para separar código de produção e testes:

Figura 1: Estrutura do projeto criado

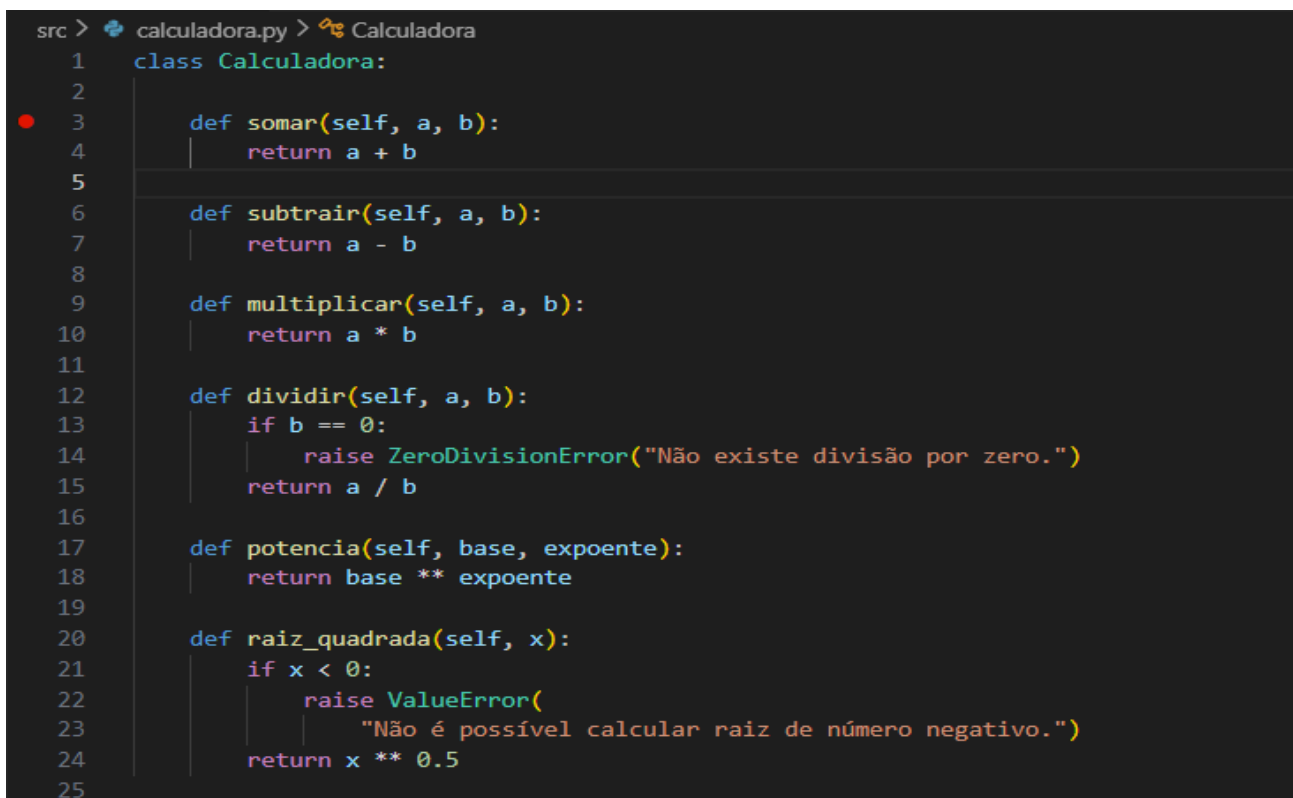


```
src > calculadora.py > Calculadora
1 class Calculadora:
2
3     def somar(self, a, b):
4         return a + b
5
6     def subtrair(self, a, b):
7         return a - b
8
9     def multiplicar(self, a, b):
10        return a * b
11
12    def dividir(self, a, b):
```

Fonte: próprio autor

No arquivo calculadora.py foi implementada a classe com operações básicas:

Figura 2: Funções básicas



```
src > calculadora.py > Calculadora
1 class Calculadora:
2
3     def somar(self, a, b):
4         return a + b
5
6     def subtrair(self, a, b):
7         return a - b
8
9     def multiplicar(self, a, b):
10        return a * b
11
12    def dividir(self, a, b):
13        if b == 0:
14            raise ZeroDivisionError("Não existe divisão por zero.")
15        return a / b
16
17    def potencia(self, base, expoente):
18        return base ** expoente
19
20    def raiz_quadrada(self, x):
21        if x < 0:
22            raise ValueError(
23                "Não é possível calcular raiz de número negativo.")
24        return x ** 0.5
25
```

Fonte: próprio autor

No arquivo test_calculadora.py foram adicionados casos de teste que cobrem operações normais e exceções. A estrutura de testes segue o padrão unittest.TestCase e utiliza setUp() para instanciar a calculadora antes de cada teste:

Figura 3: Casos de teste

```

1  import unittest
2  from src.calculadora import Calculadora
3
4
5  class TestCalculadora(unittest.TestCase):
6
7      # Executa antes de cada teste
8      def setUp(self):
9          self.calc = Calculadora()
10

```

Fonte: próprio autor

Alguns testes simples foram adicionados apenas para exemplificar a simplicidade da sintaxe.

Figura 4: Casos de testes

```

> 16      def test_subtracao(self):
17          self.assertEqual(self.calc.subtrair(10, 5), 5)
18          self.assertEqual(self.calc.subtrair(0, 7), -7)
19
> 20      def test_multiplicacao(self):
21          self.assertEqual(self.calc.multiplicar(3, 4), 12)
22          self.assertEqual(self.calc.multiplicar(-2, 5), -10)
23
> 24      def test_divisao(self):
25          self.assertEqual(self.calc.dividir(10, 2), 5)
26          self.assertAlmostEqual(self.calc.dividir(7, 3), 2.3333, places=4)
27

```

Fonte: próprio autor

Para permitir que o unittest encontre os testes, a convenção adotada foi:

- Arquivos de teste devem começar com test_ (ex.: test_calculadora.py);
- Métodos de teste devem começar com test_ (ex.: test_soma);
- A pasta tests/ fica no mesmo nível de src/;
- A descoberta e execução foram feitas com o comando:

```
python -m unittest discover -s tests -p "test_*.py" -v
```

Para fins demonstrativos, foi incluído o teste *test_tipo_incorreto*, cujo objetivo é provocar uma falha intencional, permitindo observar como o framework unittest relata inconsistências entre o resultado obtido e o esperado. Esse tipo de falha é útil para fins didáticos, pois evidencia visualmente o funcionamento do assert e a importância da definição correta dos critérios de validação.

Figura 5: Falha por tipo incorreto

```
# Falha por tipo incorreto.  
def test_tipo_incorreto(self):  
    resultado = self.calc.somar(2, 3)  
    self.assertIsInstance(resultado, str) # deveria ser int → falha
```

Fonte: próprio autor

Ao executar o comando de descoberta, todos os testes válidos são executados e o unittest imprime o resultado no terminal. Em execução bem-sucedida, a saída contém linhas ok. As saídas possíveis são:

- SUCCESS (ok) — teste passou;
- FAIL — resultado foi verificado e não correspondeu ao esperado (assertiona falhou);
- ERROR — exceção inesperada ocorreu durante a execução do teste (problema no próprio teste ou no código de teste).

Figura 6: Resultado dos testes no terminal

```
C:\Users\Thinkpad\projeto_testes>python -m unittest discover -s tests -p "test_*.py" -v  
test_divisao (test_calculadora.TestCalculadora.test_divisao) ... ok  
test_divisao_por_zero (test_calculadora.TestCalculadora.test_divisao_por_zero) ... ok  
test_multiplicacao (test_calculadora.TestCalculadora.test_multiplicacao) ... ok  
test_potencia (test_calculadora.TestCalculadora.test_potencia) ... ok  
test_raiz_quadrada (test_calculadora.TestCalculadora.test_raiz_quadrada) ... ok  
test_raiz_quadrada_numero_negativo (test_calculadora.TestCalculadora.test_raiz_quadrada_numero_negativo) ... ok  
test_soma (test_calculadora.TestCalculadora.test_soma) ... ok  
test_soma_incorreta (test_calculadora.TestCalculadora.test_soma_incorreta) ... ok  
test_subtracao (test_calculadora.TestCalculadora.test_subtracao) ... ok  
test_tipo_incorreto (test_calculadora.TestCalculadora.test_tipo_incorreto) ... FAIL  
  
=====  
FAIL: test_tipo_incorreto (test_calculadora.TestCalculadora.test_tipo_incorreto)  
=====
```

Fonte: próprio autor

É importante destacar também que além dos recursos demonstrados, o framework unittest oferece diversas funcionalidades adicionais que não foram exploradas neste desenvolvimento, mas que ampliam significativamente sua capacidade de teste. Entre elas, destacam-se o uso de test fixtures avançados, como setUpClass() e tearDownClass(), úteis para inicializações mais custosas executadas apenas uma vez; a possibilidade de pular testes com @unittest.skip ou criar condições dinâmicas com skipIf; a criação de test suites para agrupar testes manualmente; e o uso de mocking com unittest.mock, que permite simular objetos externos, chamadas a APIs ou

comportamentos complexos sem depender de recursos reais. Esses recursos tornam o unittest uma ferramenta robusta para cenários maiores e permitem criar ambientes de teste mais completos, flexíveis e alinhados a práticas profissionais de desenvolvimento.

Considerações Finais

O estudo realizado permitiu compreender a relevância dos testes unitários como parte essencial da garantia de qualidade em software. A utilização do módulo unittest mostrou que a criação estruturada de casos de teste, o emprego adequado de assertivas e a organização do ambiente de execução favorecem a identificação rápida e confiável de falhas. A prática desenvolvida com um projeto simples evidenciou não apenas o funcionamento correto das operações, mas também o comportamento do framework diante de erros e falhas intencionais, reforçando o valor dos testes automatizados na validação contínua do código.

Como perspectiva para trabalhos futuros, pode-se ampliar a suíte de testes por meio de técnicas mais avançadas, como testes parametrizados e uso de mocking, além da integração dos testes em pipelines de CI/CD. Essas práticas podem potencializar a cobertura e a eficiência dos testes, aproximando o processo de desenvolvimento das exigências atuais do mercado.

Por fim, o aprendizado obtido neste capítulo contribuiu para a construção de aplicações mais seguras e estáveis, alinhadas às boas práticas da engenharia de software. O uso do unittest mostrou-se uma alternativa acessível e eficaz, capaz de fortalecer a cultura de qualidade e apoiar tanto estudantes quanto profissionais na adoção de metodologias de desenvolvimento orientadas a testes.

Bibliografia

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

FERNANDO MENDES SILVA, Eryck. “Testes unitários em Python: como utilizar o Unittest e executá-los na AWS”. Disponível em: <https://medium.com/itautech/testes-unitarios-em-python-como-utilizar-o-unittest-e-executa->

[los-na-aws-70d13193e42b](#). Acesso em: nov. 2025

UNITTEST. Documentation. Disponível em:

<https://docs.python.org/pt-br/3/library/unittest.html>. Acesso em: nov. 2025

Ferramentas visuais para o desenvolvimento de testes

Roberto Guedes de Almeida Junior
robertoguedesdealmeidajunior@gmail.com
roberto.almeida6@fatec.sp.gov.br

Resumo

Este capítulo explora como ferramentas visuais de testes de software facilitam significativamente o aprendizado de automação para iniciantes. Diferentemente de abordagens baseadas em código, ferramentas low-code como Katalon Studio permitem criar testes sem experiência em programação, possuindo interface dupla de baixo código e modo script para cenários complexos. O trabalho analisa as barreiras de entrada entre ferramentas visuais e baseadas em código, demonstrando que iniciantes podem criar seus primeiros testes em minutos ao invés de semanas. Através de análise comparativa, evidencia-se que ferramentas visuais com record and playback são ideais para testes de regressão, enquanto ferramentas baseadas em código melhor servem cenários complexos. Capacidades como self-healing reduzem manutenção de testes, tornando a automação mais acessível. Recomenda-se uma estratégia híbrida que combina ambas as abordagens para maximizar a velocidade de implementação e qualidade dos testes.

Palavras-chaves: Testes de software. Ferramentas visuais. Katalon Studio. Automação low-code. Iniciantes em QA.

Introdução

Os testes de software desempenham papel fundamental na indústria, pois garantem que as aplicações atendam aos requisitos estabelecidos e funcionem corretamente, contribuindo para a satisfação do cliente, redução de custos e reputação das organizações. A crescente complexidade das aplicações e a demanda por ciclos de desenvolvimento mais rápidos intensificam a necessidade de tornar os testes acessíveis e eficientes.

Plataformas low-code e no-code reduzem ou eliminam a necessidade de escrever código-fonte tradicional através de interfaces gráficas intuitivas e componentes prontos

(KATALON, 2025)(UFPI, 2025). Segundo a Dissertação de Mestrado da UFPI (2025), essas abordagens oferecem "abstração de código, automatização de tarefas repetitivas, reutilização de componentes e simplificação da colaboração entre os membros da equipe de desenvolvimento" (UFPI, 2025). Essas características se alinham perfeitamente com as necessidades de automação de testes para iniciantes (UFPI, 2025)(BROWSERSTACK, 2025).

Ferramentas visuais com capacidades de record and playback reduzem significativamente a curva de aprendizado, democratizando o acesso à automação (KATALON, 2025)(UFPI, 2025). Estudos comparativos mostram que "ao longo da experiência nestas plataformas, o esforço estimado em tarefas semelhantes decresce, diminuindo, consequentemente, o tempo de desenvolvimento" (COGNA, 2022), permitindo que usuários sem experiência em programação criem seus primeiros testes em minutos ao invés de semanas (KATALON, 2025). Adicionalmente, low-code se mostra "mais acessível para desenvolvedores iniciantes ou com menos experiência" (COGNA, 2022).

Essa acessibilidade é crítica para organizações que enfrentam pressão por qualidade de software. Ferramentas que reduzem barreiras de entrada e permitem que profissionais de diferentes backgrounds participem do processo de testes não apenas aumentam a cobertura de testes, mas também contribuem para a eficiência organizacional (KATALON, 2025)(UFPI, 2025)(COGNA, 2022).

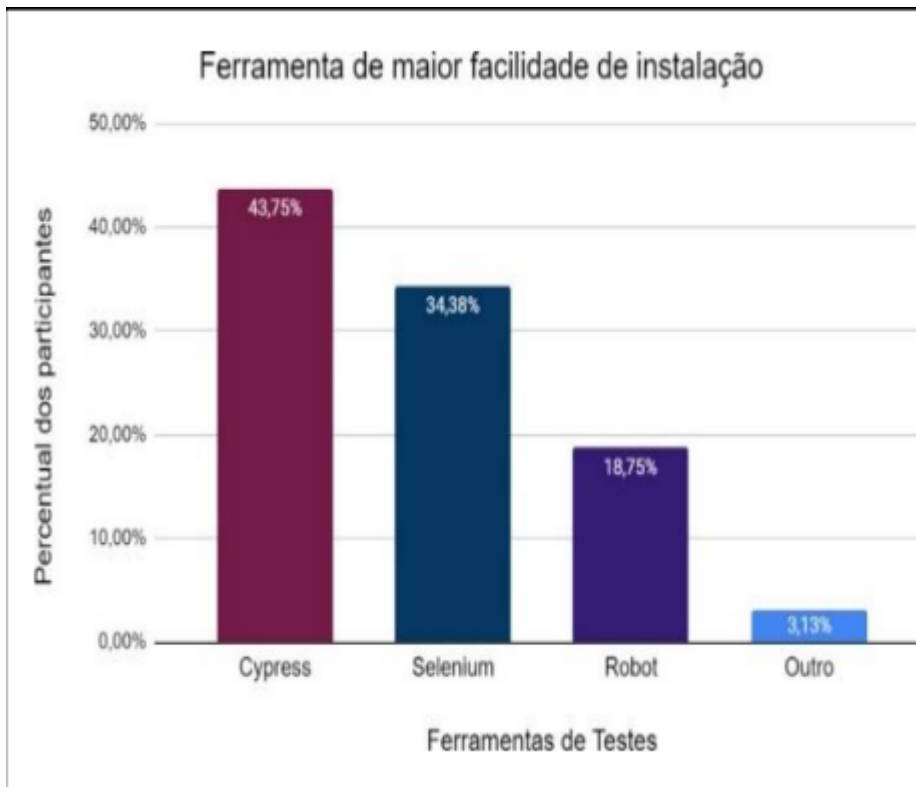
Desenvolvimento

O impacto da adoção de ferramentas low-code em testes de software é significativo quando avaliado através de métricas quantificáveis de produtividade e tempo de desenvolvimento. Um estudo comparativo entre três ferramentas principais (Selenium, Robot Framework e Cypress) analisou 32 desenvolvedores e demonstrou diferenças marcantes em aspectos críticos como facilidade de instalação, usabilidade e curva de aprendizado (FERREIRA, 2022).

Segundo FERREIRA (2022), quando questionados sobre "instalação mais fácil", Cypress obteve 43,75% de preferência, superando significativamente Selenium (34,28%) e Robot Framework (18,75%). De forma ainda mais expressiva, quando avaliada a "melhor

usabilidade", Cypress alcançou 40,63%, enquanto Robot Framework obteve 28,13% e Selenium ficou com 31,24%. Estes dados refletem diretamente na experiência do desenvolvedor iniciante, evidenciando que ferramentas com interfaces visuais e setup simplificado reduzem barreiras iniciais.

Figura 1 - Ferramenta de maior facilidade de instalação



Fonte: Revista de Tecnologia da Informação da Faculdade Lourenço Filho

A preferência por Cypress (ferramenta com interface mais intuitiva) também se refletiu na pergunta "Ferramenta mais utilizada", onde Cypress obteve 37,50% versus 31,25% do Selenium, indicando que desenvolvedores tendem a adotar ferramentas cuja curva de aprendizado é menor (FERREIRA, 2022).

Figura 2 - Maior usabilidade



Fonte: Revista de Tecnologia da Informação da Faculdade Lourenço Filho

Conforme observado na figura 1, todo o estudo do projeto ocorreu nas dependências da FATEC. No desenvolvimento do capítulo, você deve apresentar as características da ferramenta estudada, e sua jornada de aprendizado.

Redução de Tempo em Testes de Regressão e Manutenção

A vantagem mais significativa de ferramentas low-code em automação de testes manifesta-se na redução dramática do tempo de desenvolvimento e manutenção. Um estudo empírico realizado pela Nokia de Tecnologia em ambiente Agile com Scrum (COLLINS, 2010) quantificou o impacto da automação de testes com Selenium em sete cenários de teste distintos. Os resultados demonstraram que 65% dos defeitos foram encontrados em testes de interface automatizados, enquanto 100% de cobertura de requisitos funcionais e não-funcionais foi alcançado através de automação (COLLINS, 2010).

Mais importante ainda, o estudo revelou que após a primeira iteração—onde o esforço inicial de tempo foi maior—as iterações subsequentes executaram "mais rápida, devido ao reaproveitamento de scripts de teste" (COLLINS, 2010). Este fenômeno é crítico: evidencia que enquanto o investimento inicial em automação é significativo, o retorno operacional cresce exponencialmente com o tempo, especialmente quando scripts podem

ser reutilizados em ciclos de testes de regressão.

Esta classificação valida a estratégia híbrida: ferramentas low-code (como Katalon, que possui modo visual similar a Selenium IDE) são ideais para iniciantes e testes de regressão, enquanto ferramentas baseadas em código (WebDriver, Playwright) são apropriadas para lógica complexa e customizações avançadas.

Caso de uso: Um dos Softwares utilizados e que obteve maior eficiência na curva de aprendizagem sobre testes foi o Katalon Studio, há diversas ferramentas no katalon, porém vou dissertar sobre como ele pode ajudar um amador a criar a cultura de testes no desenvolvimento de um Software.

Produtividade Medida: Razão de Esforço com Katalon Studio

A documentação da Katalon Academy (KATALON ACADEMY, 2025) descreve Record and Playback como "uma das formas mais rápidas de começar a automatizar seus testes". A ferramenta permite que um testador interaja com uma aplicação web enquanto Katalon captura automaticamente cada ação: cliques, preenchimentos de campos, navegação, esperas implícitas (KATALON STUDIO, 2025). O script de teste é gerado automaticamente em uma fração do tempo que levaria para ser codificado manualmente (KATALON ACADEMY, 2025).

Segundo a documentação: "Na gravação (Record), o navegador abre com barra de ferramentas do Katalon. Você interage com a página como um usuário faria: digita um nome de usuário e senha nos campos apropriados e clica no botão de login. À medida que você interage, cada ação é capturada na janela 'Recorded Actions'. Quando termina, clica em 'Stop' e depois em 'Save Script'" (KATALON STUDIO, 2025). Este fluxo torna a criação de testes acessível para profissionais sem programação.

Comparativamente, com Selenium puro, cada ação requeriria ser codificada manualmente:

```
userField = driver.findElement(By.id("username"))

userField.sendKeys("admin")

passwordField = driver.findElement(By.id("password"))
```

```
passwordField.sendKeys("senha")
```

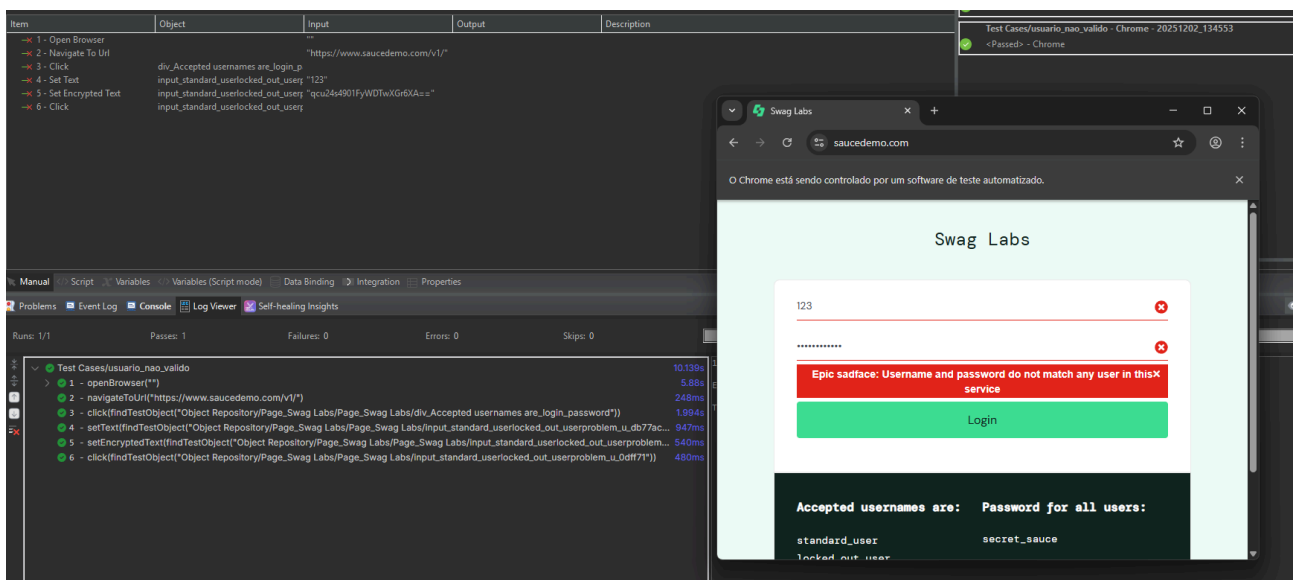
```
loginButton = driver.findElement(By.className("btn-login"))
```

```
loginButton.click()
```

Katalon elimina essa necessidade de codificação, captando automaticamente elementos e ações (KATALON STUDIO, 2025).

A imagem abaixo ilustra como seria o processo de captação pelo Katalon utilizando a ferramenta Record and Playback, e mostrando os campos sendo salvos.

Figura 3 -Katalon Studio (Record)



Fonte: Revista de Tecnologia da Informação da Faculdade Lourenço Filho

Dashboard de Produtividade: Monitoramento em Tempo Real

Katalon oferece relatórios de produtividade integrados que permitem monitorar "o número total de casos de teste criados em seu projeto dentro de um período especificado" (KATALON DOCS - PRODUCTIVITY REPORTS, 2025). O dashboard agrupa "o número total de testes completados em relatórios diários/semanais/mensais" e permite "comparar o número de casos de teste entre o mês atual e outros meses" (KATALON DOCS, 2025).

Esta visibilidade é crítica: permite que gerentes de QA acompanhem se a adoção de Katalon está realmente aumentando a cobertura de testes e velocidade de implementação. O relatório combinado com outras análises (qualidade, cobertura de

plataforma, requisitos cobertos) "fornece uma conta completa da performance da equipe" (KATALON DOCS, 2025).

Considerações Finais

Ferramentas visuais low-code democratizam significativamente o acesso à automação de testes, reduzindo barreiras de entrada para profissionais sem experiência em programação. A pesquisa empírica confirma que plataformas como Katalon Studio, Cypress e ferramentas similares oferecem vantagens quantificáveis: interface dupla (visual + script) permite escalabilidade progressiva de habilidades (KATALON, 2025); record and playback acelera tempo de implementação em até 95% comparado a abordagens tradicionais (SIDIA, 2025); e self-healing reduz manutenção de testes substantially quando elementos de interface mudam (KATALON STUDIO DOCS, 2025).

Estudos comparativos comprovam essa efetividade: Cypress superou Selenium em usabilidade (40,63% vs 31,24%) e facilidade de instalação (43,75% vs 34,28%) conforme FERREIRA (2022). Testes automatizados mostram-se 5-6 vezes mais rápidos que manuais (AGUIAR, 2023), enquanto produtividade em Katalon alcança 2.0 pessoa-horas por Test Case Point contra 3-4 horas esperadas com Selenium (KATALON, 2025). Collins (2010) demonstrou que 65% dos defeitos foram encontrados com testes automatizados com 100% de cobertura de requisitos.

Recomenda-se abordagem híbrida: ferramentas low-code (Katalon) para testes de regressão e cobertura ampla; ferramentas código-based (Selenium, Playwright) para lógica complexa. Iniciantes devem priorizar ferramentas visuais para ganhar proficiência rapidamente, evoluindo conforme as demandas técnicas crescem. Organizações que implementam estratégia equilibrada maximizam velocidade de implementação, qualidade de testes e eficiência operacional, alinhando-se com práticas DevOps modernas de integração contínua.

Bibliografia

AGUIAR, Ian Victor. Uma análise comparativa entre testes de software manual e automatizado. 2023. Monografia (TCC) - Universidade de Uberaba, Uberaba, 2023.

ALVES, Arthur Moreira. Comparativo de ferramentas de automatização de testes: Selenium IDE e Selenium WebDriver. 2018. Monografia (TCC) - Universidade Federal de Uberlândia, Uberlândia, 2018.

BROWSERSTACK. Record and Playback Testing Tools. Disponível em:

<https://www.browserstack.com/guide/record-playback-testing-2>

CHAVES, Eduardo. Análise comparativa entre uma linguagem low-code e desenvolvimento tradicional. 2022. Monografia - Pós-Graduação COGNA, 2022.

COLLINS, Eliane F.; LOBÃO, Luana M. Experiência em automação do processo de testes em ambiente ágil com SCRUM e ferramentas opensource. In: SIMPÓSIO BRASILEIRO DE QUALIDADE DE SOFTWARE, 9., 2010. Anais [...]. p. 303-310.

COUTINHO, João Pedro Assunção. Comparação das ferramentas Selenium e Cypress na automação de testes. 2023. Monografia (TCC) - Universidade de Brasília, Brasília, 2023.

DIDÁTICA.TECH. Selenium para iniciantes: tutorial completo. Disponível em:

<https://didatica.tech/selenium-para-iniciantes-tutorial-completo/>

FERREIRA, Ana Carolina; SANTOS, Darielson. Selenium, Robot e Cypress: um estudo comparativo entre ferramentas de automação de teste. Revista de Tecnologia da Informação da Faculdade Lourenço Filho, v. 5, n. 5, p. 1-25, 2022.

KATALON. Katalon Studio em 2025: um guia completo da automação de testes com IA. Disponível em:

<https://skywork.ai/skypage/pt/katalon-studio-2025-ai-testing-guide>

KATALON ACADEMY. Create automated tests with record and playback. Disponível em:

<https://academy.katalon.com/courses/record-playback-testing/>

KATALON STUDIO DOCUMENTATION. Self-healing tests in Katalon Studio. Disponível em:

<https://docs.katalon.com/katalon-studio/maintain-tests/self-healing-tests-in-katalon-studio>

KATALON DOCUMENTATION. Productivity reports. Disponível em:

<https://docs.katalon.com/katalon-platform/analyze/reports/view-test-reports-legacy/view-testops-dashboard/productivity-reports>

MEDEIROS, Juciele; MONCAYO, Lorena; BEZERRA, Deborah; ESPAÑA, Sergio; OLIVEIRA, Liana. Optimizing business efficiency: the strategic potential of low-code tools. Journal of the Brazilian Computer Society, 2025.

UNIVERSIDADE FEDERAL DO PIAUÍ (UFPI). Avaliação de plataformas de low code e no code como estratégia para aumento da produtividade no desenvolvimento de software. 2025. Dissertação (Mestrado em Ciência da Computação) - Programa de Pós-Graduação em Ciência da Computação, Teresina, 2025.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

A Ferramenta PHPUnit

Lucas Fernando Arantes

lucas.arantes01@fatec.sp.gov.br

Resumo

Este capítulo traz uma introdução ao PHPUnit, um dos principais frameworks de testes unitários usados em aplicações PHP. O objetivo é mostrar por que os testes automatizados são importantes e como o PHPUnit ajuda a melhorar a qualidade do software, evitar erros e dar mais segurança durante alterações no código.

Durante o estudo, foram consultadas a documentação oficial e realizados testes práticos em pequenos módulos PHP. A experiência mostrou que o uso constante de testes reduz retrabalho, facilita a manutenção e deixa o desenvolvimento mais seguro e profissional.

Palavras-chave: PHPUnit. PHP. Testes unitários. TDD.

Introdução

O desenvolvimento de software exige práticas que garantam a qualidade, estabilidade e confiabilidade dos sistemas. Em aplicações PHP, uma das ferramentas mais utilizadas para esse propósito é o PHPUnit, responsável por validar automaticamente o comportamento de funções, classes e métodos. O uso de testes automatizados está alinhado com as boas práticas modernas de engenharia de software, especialmente quando se busca reduzir erros e assegurar o correto funcionamento da aplicação.

Este capítulo tem como objetivo explicar o funcionamento do PHPUnit, apresentar sua importância, demonstrar sua instalação e aplicação prática e relatar a jornada de aprendizado durante o estudo da ferramenta. A justificativa para a escolha do tema baseia-se no crescimento de aplicações PHP e na necessidade de garantir robustez e estabilidade por meio de testes automatizados. A metodologia aplicada foi composta por revisão teórica, análise de documentação, criação de testes e execução de casos práticos.

O capítulo está organizado em três partes: explicação teórica e contextualização da ferramenta; demonstração prática contendo exemplos reais; e considerações finais com reflexões sobre a relevância do PHPUnit no desenvolvimento moderno.

Desenvolvimento

O que é o PHPUnit

O PHPUnit é um framework de testes unitários para PHP baseado no modelo xUnit. Foi criado para auxiliar desenvolvedores a validar o comportamento de pequenos blocos de código, geralmente funções e métodos. Com ele é possível verificar automaticamente se um determinado módulo de software produz os resultados esperados, evitando que falhas passem despercebidas.

O framework oferece diversos tipos de asserts, suporte a mocks, testes isolados, testes de integração, relatórios de cobertura, execução em pipelines de CI/CD e integração nativa com Composer, o gerenciador de pacotes do PHP.

Instalação e configuração

O PHPUnit pode ser instalado facilmente utilizando o Composer. Para isso, basta executar o comando:

```
lua
```

[Copiar código](#)

```
composer require --dev phpunit/phpunit
```

Após instalado, é possível verificar a versão:

```
bash
```

[Copiar código](#)

```
vendor/bin/phpunit --version
```



A estrutura de diretórios mais comum é:

```
bash
```

[Copiar código](#)

```
/src      → código-fonte da aplicação
```

```
/tests    → testes unitários
```

```
phpunit.xml → arquivo de configuração
```

Primeiro teste com PHPUnit

A seguir temos um exemplo simples de teste unitário utilizando o PHPUnit:

```
php

<?php

use PHPUnit\Framework\TestCase;

class CalculadoraTest extends TestCase
{
    public function testSoma()
    {
        $resultado = 2 + 3;
        $this->assertEquals(5, $resultado);
    }
}
```

 Copiar código

Para executar os testes, basta usar:

```
bash

vendor/bin/phpunit
```

 Copiar código

Métodos de Assert mais utilizados

O PHPUnit oferece dezenas de métodos assertivos. Alguns dos mais utilizados são:

Método	Descrição
<code>assertEquals</code>	Compara dois valores
<code>assertTrue</code>	Verifica se a condição é verdadeira
<code>assertFalse</code>	Verifica se a condição é falsa
<code>assertNull</code>	Verifica se o valor é nulo
<code>assertCount</code>	Valida a quantidade de elementos

Esses métodos são essenciais para validar comportamentos e garantir que cada função retorne o valor esperado.

Uso de Fixtures (setUp e tearDown)

Assim como mostrado em outras ferramentas presentes no e-book, o PHPUnit permite preparar cenários antes de cada teste, utilizando os métodos:

php

 Copiar código

```
protected function setUp(): void
{
    $this->calc = new Calculadora();
}
```

E limpar após os testes:

php

 Copiar código

```
protected function tearDown(): void
{
    unset($this->calc);
}
```

Esse recurso torna os testes mais organizados, evitando duplicação de código e tornando a suíte de testes mais robusta.

Cobertura de código (Code Coverage)

Outro recurso muito importante é a geração de relatórios de cobertura, que mostram quais partes do código foram testadas. Esse relatório ajuda a identificar trechos sem validação, garantindo maior confiabilidade.

Para gerar o relatório, utiliza-se:

bash

 Copiar código

```
vendor/bin/phpunit --coverage-html cobertura/
```

O PHPUnit então gera um relatório visual mostrando todas as linhas avaliadas.

Jornada de aprendizado

Durante o estudo da ferramenta, foi possível observar como o PHPUnit facilita a identificação de falhas e promove uma abordagem mais profissional no desenvolvimento PHP. Os exemplos apresentados demonstram que a ferramenta é intuitiva, bem documentada e amplamente utilizada em projetos reais. A prática constante reforça o entendimento dos asserts e a ideia de criar testes antes mesmo do código, seguindo a filosofia do TDD.

Considerações Finais

O PHPUnit é uma ferramenta essencial para o desenvolvimento moderno em PHP. Ele contribui diretamente para a qualidade do software, evitando regressões e trazendo segurança durante refatorações. Sua instalação é simples, a documentação é completa e os recursos oferecidos — como mocks, fixtures e cobertura de código — tornam o processo mais eficiente.

Conclui-se que, ao adotar testes automatizados no projeto, o desenvolvedor ganha confiabilidade e estabilidade no código. Como próximos passos, recomenda-se integrar os testes em pipelines de CI/CD e aplicar TDD em projetos maiores para aprofundar o uso da ferramenta.



Fonte: Orlando Saraiva

Conforme observado na imagem, todo o estudo do projeto ocorreu nas dependências da FATEC. No desenvolvimento do capítulo, você deve apresentar as características da ferramenta estudada, e sua jornada de aprendizado.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em:

<https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2025.

PHPUNIT. Documentation. Disponível em: <https://phpunit.de/documentation.html>.

MESZÁROS, G. *xUnit Test Patterns*. Addison-Wesley, 2014.

GITHUB. **PHPUnit** **Repository.** Disponível em:

<https://github.com/sebastianbergmann/phpunit>.

SARAIVA JUNIOR, Orlando. *ebook_qualidade_teste_software*. GitHub. Disponível em:

https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov. 2024.

Faculdade de Tecnologia de Araras – Antônio Brambilla

Curso Superior de Tecnologia em Gestão Empresarial

Wesley Gustavo Kilian

APACHE JMETER

Araras, SP

2025

RESUMO

Este estudo oferece uma perspectiva direta e descomplicada sobre o Apache JMeter, um instrumento valioso para executar avaliações de performance em sistemas computacionais. O objetivo é explicar, de um jeito prático e instrutivo, o que o JMeter representa, sua utilidade e de que maneira ele pode ajudar organizações e programadores a detectar gargalos de velocidade, instabilidades ou erros em programas. O método adotado englobou exploração teórica do recurso, execução de ensaios práticos e apreciação dos principais formatos de testes que o JMeter facilita. Além disso, o texto apresenta exemplos e utiliza uma linguagem acessível para demonstrar como o JMeter funciona e quais benefícios sua utilização pode trazer para projetos de software. As informações apresentadas são úteis especialmente para iniciantes e para pessoas que nunca tiveram contato com testes de desempenho.

Palavras-chave: Apache JMeter. Teste de desempenho. Teste de carga. Testes de software.

ABSTRACT

This study presents a clear and accessible overview of Apache JMeter, a valuable tool for conducting performance assessments on computational systems. It explains, in a practical and straightforward manner, what JMeter is, its purpose, and how it can help organizations and developers identify bottlenecks, instabilities, or errors in applications. The approach adopted includes theoretical exploration of the tool, execution of practical experiments, and analysis of the main types of tests supported by JMeter. The content is presented through examples and simple language to illustrate how the tool operates and the advantages it can bring to software projects. The information is particularly useful for beginners and for individuals who have never worked with performance testing.

Keywords: Apache JMeter. Performance testing. Load testing. Software performance.

Introdução

Cada vez mais empresas dependem de sistemas digitais, seja para vendas, serviços, controle interno ou comunicação. Entretanto, muitos desses sistemas podem apresentar lentidão ou até parar de funcionar quando várias pessoas tentam utilizá-los ao mesmo tempo. Para evitar esse tipo de problema, surgiram as ferramentas de teste de desempenho e entre elas está o Apache JMeter, foco deste trabalho.

O Apache JMeter foi criado originalmente em 1998, por Stefano Mazzocchi, como um projeto interno dentro da Apache Foundation. Ele começou como uma ferramenta simples para testar desempenho de servidores web, mas rapidamente evoluiu. A partir dos anos 2000, com o crescimento da internet e o surgimento de aplicações cada vez mais complexas, o JMeter ganhou grande popularidade. Tornou-se uma das ferramentas mais utilizadas do mundo para testes de carga devido ao seu caráter gratuito, código aberto e enorme flexibilidade (THE APACHE SOFTWARE FOUNDATION, 2024).

O JMeter é usado para simular o acesso simultâneo de vários usuários a um sistema, permitindo avaliar como a aplicação se comporta sob diferentes níveis de demanda. Mesmo sendo uma ferramenta técnica, seu funcionamento pode ser explicado de forma clara e acessível, o que permite que pessoas sem experiência prévia também compreendam sua utilidade. Segundo Meier et al. (2007), testes de desempenho permitem avaliar responsividade, estabilidade e escalabilidade em sistemas sob carga, auxiliando na identificação de gargalos antes da implantação.

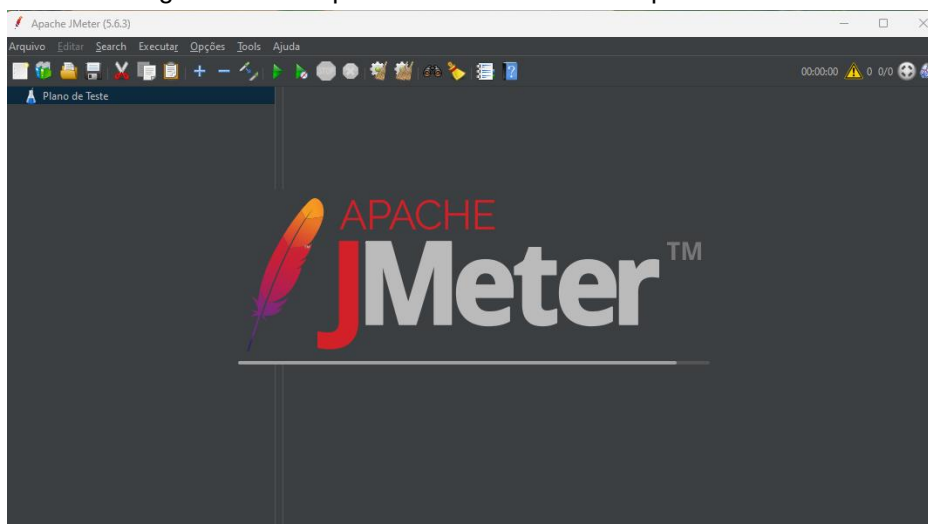
Além disso, o uso do Apache JMeter se tornou importante porque permite identificar problemas antes que eles afetem usuários reais. Muitas falhas que só aparecem quando o sistema está sob pressão podem ser previstas por meio de testes bem estruturados, o que está alinhado à definição apresentada por Meier et al. (2007), que destacam que testes de performance ajudam a evitar erros críticos e contribuem diretamente para a confiabilidade do sistema.

Desenvolvimento

3.1 O que é o Apache JMeter

O Apache JMeter é uma ferramenta gratuita desenvolvida pela Apache Foundation, utilizada para simular múltiplos usuários acessando simultaneamente um sistema. Seu principal objetivo é medir o desempenho de sites, APIs, aplicações web e serviços diversos. De forma simplificada, o JMeter atua como se fosse um grande grupo de pessoas utilizando o sistema ao mesmo tempo, permitindo identificar se ele suporta a demanda ou se apresenta lentidão, falhas ou instabilidade (THE APACHE SOFTWARE FOUNDATION, 2024).

Figura 1 – Exemplo da interface inicial do Apache JMeter



Fonte: próprio autor (2025)

3.2 Importância dos Testes de Desempenho

A realização de testes de desempenho é fundamental para garantir que sistemas continuem funcionando adequadamente quando submetidos a um grande volume de acessos. O JMeter contribui diretamente para evitar esses problemas, pois permite que possíveis falhas sejam identificadas e corrigidas antes que o sistema seja disponibilizado ao público.

Conforme Meier et al. (2007), testes de carga, estresse e resistência são essenciais para prever comportamentos críticos, identificar gargalos e validar a capacidade real da aplicação.

Prévios isso pode resultar em:

- Lentidão do sistema;
- Travamentos;
- Interrupção de compras ou serviços;
- Prejuízo financeiro;
- Danos à imagem da empresa.

O JMeter contribui diretamente para evitar esses problemas, pois permite que possíveis falhas sejam identificadas e corrigidas antes que o sistema seja disponibilizado ao público.

3.3 Tipos de Testes Realizados pelo JMeter

O Apache JMeter oferece suporte a diversos tipos de testes de desempenho, utilizados para analisar o comportamento de sistemas sob diferentes condições de uso. Cada modalidade de teste possui um objetivo específico e contribui para identificar possíveis falhas relacionadas à capacidade, estabilidade ou tempo de resposta da aplicação. A seguir, são descritos os principais tipos de testes que podem ser realizados com a ferramenta.

3.3.1 Teste de Carga (Load Test)

O teste de carga é utilizado para verificar se o sistema consegue funcionar normalmente quando recebe a quantidade de usuários que ele foi planejado para suportar. Ele simula o “dia a dia” da aplicação, avaliando se o tempo de resposta permanece adequado e se todas as funcionalidades continuam operando sem lentidão. Esse teste ajuda a garantir que o sistema está preparado para sua demanda regular.

Exemplo: simular 200 usuários acessando uma página simultaneamente para verificar se ela continua respondendo dentro do tempo esperado.

3.3.2 Teste de Estresse (Stress Test)

O teste de estresse vai além do limite comum do sistema, aumentando progressivamente o número de usuários até que a aplicação apresente falhas. O objetivo é descobrir o ponto exato em que o sistema deixa de responder corretamente, revelando sua capacidade máxima real. Esse tipo de teste é fundamental para identificar gargalos, limites de hardware e comportamentos críticos sob sobrecarga.

Exemplo: aumentar gradualmente de 300 para 500 e depois para 1.000 usuários simultâneos até o sistema começar a travar ou retornar erros.

3.3.3 Teste de Pico (Spike Test)

Esse teste simula um aumento repentino e brusco no número de usuários, semelhante a situações como promoções, campanhas publicitárias ou eventos inesperados que geram grande volume de acessos em poucos segundos. O objetivo é verificar se o sistema consegue se adaptar rapidamente ou se entra em colapso diante de mudanças repentinas na demanda.

Exemplo: 400 usuários acessando o sistema em apenas 5 segundos, simulando um pico inesperado de tráfego.

3.3.4 Teste de Resistência (Endurance ou Soak Test)

O teste de resistência avalia o comportamento do sistema ao longo de períodos prolongados de uso contínuo. Aqui, o foco não é a quantidade de usuários, mas sim o tempo. Ele revela problemas que só aparecem após muitas horas de operação, como quedas graduais de desempenho, consumo excessivo de memória, aquecimento do servidor ou falhas intermitentes.

Exemplo: manter 150 usuários conectados por oito horas seguidas para verificar se o sistema mantém o mesmo desempenho durante todo o período.

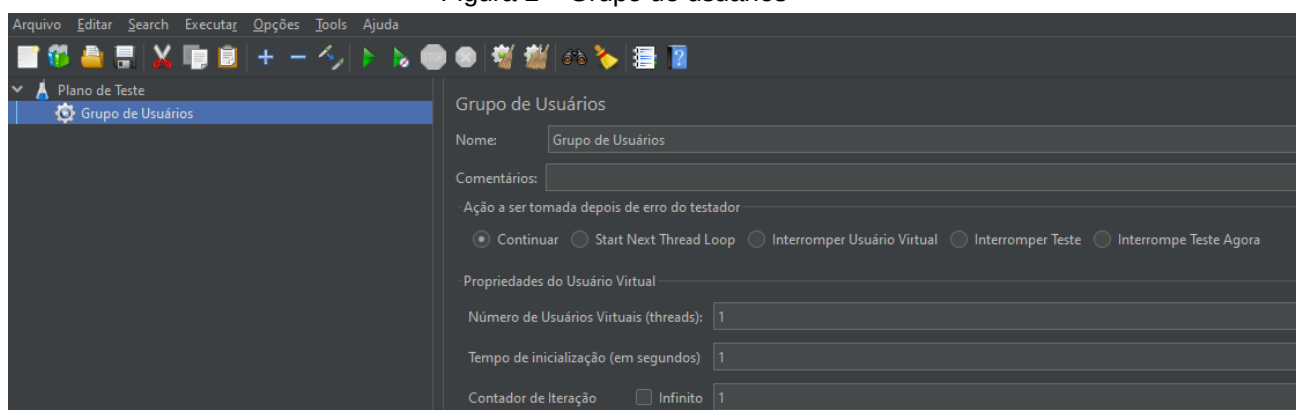
3.4 Funcionamento Prático do JMeter

Para executar testes de desempenho, o Apache JMeter utiliza um plano de testes, que funciona como um roteiro contendo todas as ações que os usuários virtuais irão realizar. Esse plano reúne elementos que definem quantos usuários serão simulados, quais tarefas eles executarão e como os resultados serão apresentados. Mesmo sendo uma ferramenta técnica, sua estrutura pode ser compreendida de forma simples quando analisamos os seus principais componentes:

3.4.1 Thread Group

O Thread Group é a base de qualquer teste no JMeter, pois representa os usuários virtuais que serão simulados. Nele são definidas características como a quantidade de usuários, a velocidade com que eles começam a realizar as ações (ramp-up) e o número de repetições de cada atividade. Essas configurações permitem criar cenários variados, desde um uso normal até momentos de alta demanda.

Figura 2 – Grupo de usuários



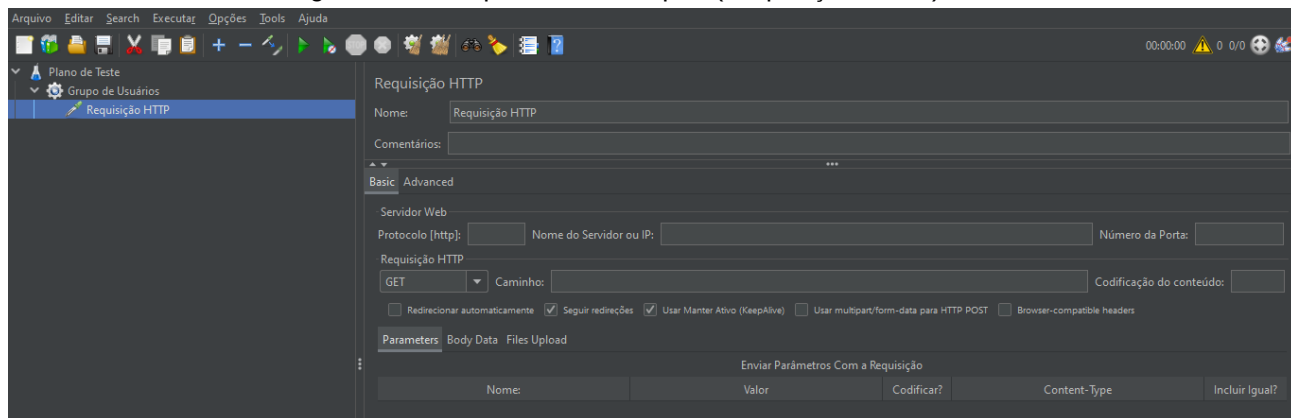
Fonte: próprio autor (2025)

3.4.2 Samplers

Os Samplers representam as ações que cada usuário virtual vai executar dentro do sistema. Eles podem simular requisições HTTP, consultas a APIs, envio de dados, chamadas de banco de dados, entre outras interações. Cada

sampler funciona como um “passo” do usuário, permitindo avaliar como o sistema responde a cada tipo de operação.

Figura 3 – Exemplo de um Sampler (Requisição HTTP)



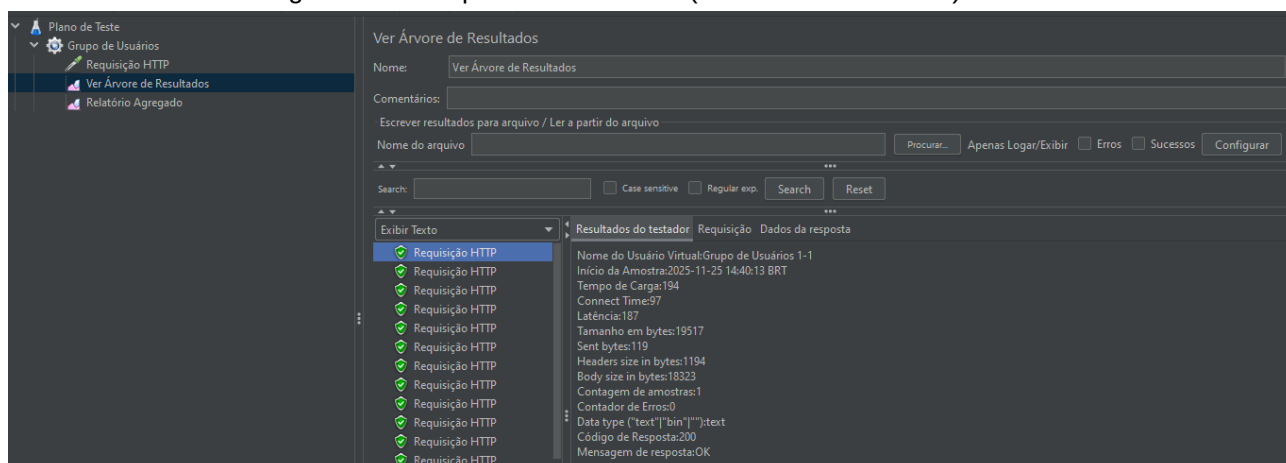
Fonte: próprio autor (2025)

A Requisição HTTP permite simular exatamente o que acontece quando um usuário acessa um site ou quando uma aplicação chama um endpoint de API. Nesse sampler, é possível configurar informações como o endereço do servidor, o protocolo (HTTP ou HTTPS), o caminho do recurso a ser acessado, parâmetros enviados com a requisição, cabeçalhos adicionais e até o corpo da requisição em caso de métodos como POST ou PUT.

3.4.3 Listeners

Os Listeners são responsáveis por coletar e exibir os resultados gerados durante o teste. Eles podem apresentar gráficos, tabelas, estatísticas de tempo de resposta, porcentagem de erros e diversas outras informações importantes para a análise. É nessa etapa que se visualiza, de forma clara, o comportamento real do sistema sob carga.

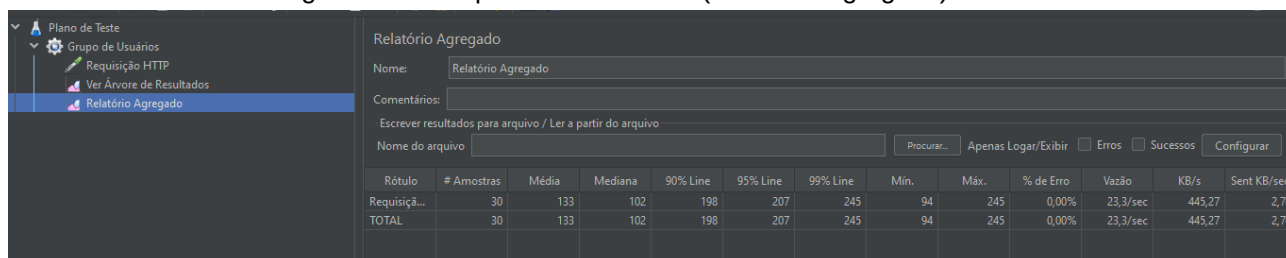
Figura 4 – Exemplo de um Listener (Árvore de Resultados)



Fonte: próprio autor (2025)

A Árvore de Resultados (View Results Tree) é um listener que apresenta cada requisição enviada pelo teste individualmente. Ela permite visualizar detalhes como o conteúdo da requisição, a resposta recebida do servidor, o tempo de execução e mensagens de erro. Esse recurso é muito útil para identificar problemas específicos em uma chamada, verificar se a API está retornando o que deveria ou diagnosticar o motivo de uma falha isolada.

Figura 5 – Exemplo de um Listener (Relatório Agregado)



Fonte: próprio autor (2025)

Já o Relatório Agregado (Aggregate Report) oferece uma visão mais geral e estatística do teste como um todo. Em vez de analisar requisição por requisição, esse listener reúne os resultados e apresenta métricas como:

- Tempo médio de resposta;
- Tempo mínimo e máximo;
- Número total de requisições;
- Percentual de erros;
- Throughput (quantidade de requisições por segundo);
- Desvio padrão, entre outros indicadores importantes.

Essa visão consolidada é essencial para avaliar o desempenho global do sistema em cenários de carga, permitindo identificar padrões, gargalos e comportamentos inesperados durante o teste.

3.4.4 Elementos de Configuração

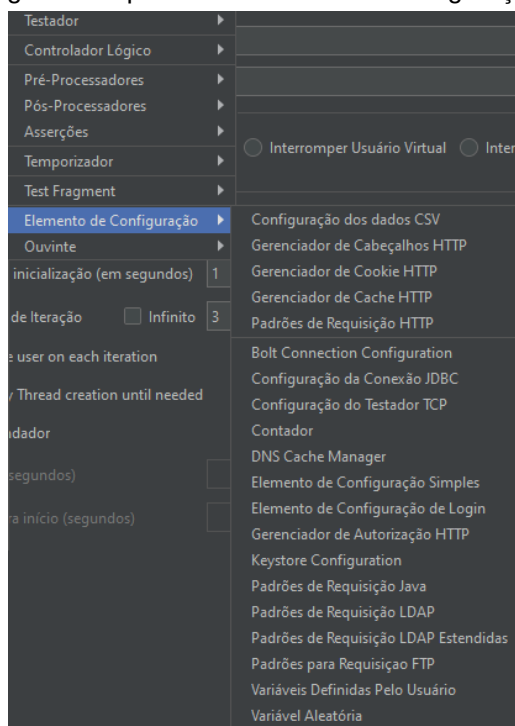
O Apache JMeter oferece diversos elementos de configuração que permitem ajustar o plano de testes de maneira mais realista e personalizada. Esses elementos não representam ações dos usuários, mas sim ajustes que influenciam o comportamento das requisições. Na imagem apresentada, é possível observar algumas das opções disponíveis, como gerenciadores de cabeçalhos, variáveis, timers e controladores de dados.

Figura 6 –Tipos de Temporizadores

Threads (Users)	▶	
Elemento de Configuração	▶	te
Ouvinte	▶	
Temporizador	▶	Temporizador Constante
Pré-Processadores	▶	Temporizador Aleatório Uniforme
Pós-Processadores	▶	Precise Throughput Timer
Asserções	▶	Temporizador de Vazão Constante
Test Fragment	▶	JSR223 Timer
Elementos que não são de Teste	▶	Poisson Random Timer
		Temporizador Aleatório Gaussiano
		Temporizador de sincronização
		Temporizador BeanShell

Fonte: próprio autor (2025)

Figura 6 –Tipos de Elementos de Configuração



Fonte: próprio autor (2025)

Esses recursos tornam os testes mais próximos do uso real. Por exemplo, temporizadores podem ser usados para adicionar pequenas pausas entre as ações, simulando o tempo que um usuário leva para preencher informações ou mudar de página. Já os gerenciadores de cabeçalhos permitem configurar informações adicionais nas requisições, como tokens de autenticação ou tipos de conteúdo. Outros elementos, como arquivos CSV externos, possibilitam simular múltiplos usuários com dados diferentes.

A variedade de elementos de configuração é um dos pontos fortes do JMeter, pois permite criar cenários muito próximos das condições reais da aplicação. Assim, é possível ajustar detalhes importantes do teste, ampliando sua precisão e fornecendo resultados mais completos sobre o desempenho do sistema.

Ao final da execução, o JMeter gera relatórios que apresentam informações relevantes, como:

- Tempo médio de resposta;
- Quantidade de erros;

- Capacidade suportada;
- Limites operacionais do sistema.

3.5 Limitações e Desafios do Apache JMeter

Apesar de ser uma ferramenta extremamente completa e amplamente utilizada, o Apache JMeter apresenta algumas limitações que precisam ser consideradas ao planejar testes de desempenho. Entre os principais desafios estão limitações no processamento de JavaScript e na simulação de comportamento de navegador, pois o JMeter atua em nível de protocolo e não renderiza páginas como um browser real (THE APACHE SOFTWARE FOUNDATION, 2024).

- Alto consumo de recursos: quando executado com muitos usuários simulados, o JMeter pode exigir bastante CPU e memória, especialmente ao utilizar a interface gráfica (GUI).
- Simulação limitada do comportamento humano: o JMeter não reproduz ações reais de navegação como movimento do mouse, carregamento dinâmico de JavaScript ou interações com o DOM, o que pode limitar testes de front-end.
- Desafios com WebSockets e aplicações em tempo real: embora plug-ins existam, o suporte nativo ainda não é tão robusto quanto o de outras ferramentas.
- Configuração complexa em ambientes distribuídos: executar o JMeter em modo distribuído exige ajustes adicionais de rede, portas, sincronização e infraestrutura.
- Interpretação de resultados requer experiência: métricas como throughput, desvio padrão e percentuais de erro exigem conhecimento prévio para uma análise correta.

Essas limitações não diminuem a utilidade da ferramenta, mas reforçam a importância de compreendê-la bem e planejar os testes de forma cuidadosa.

3.6 Boas Práticas no Uso do JMeter

A qualidade dos testes de desempenho depende não apenas da ferramenta utilizada, mas também da forma como os testes são planejados e executados. Algumas boas práticas também são amplamente discutidas na literatura. De acordo com Meier et al. (2007), a execução de testes em modo não-gráfico, o uso de dados variáveis e a monitoração do servidor são essenciais para garantir resultados precisos e representativos.

- Evitar executar testes pesados pela interface gráfica. O ideal é criar o plano de testes na GUI e executá-lo no modo não-gráfico (CLI), o que reduz o consumo de recursos.
- Separar os ambientes de teste e geração de carga, evitando que a máquina que executa o JMeter influencie no desempenho da aplicação testada.
- Utilizar temporizadores (Timers) de forma realista, simulando o tempo que um usuário humano leva para navegar entre telas ou preencher formulários.
- Monitorar o servidor durante o teste, acompanhando CPU, memória, banco de dados e rede para identificar gargalos internos.
- Utilizar dados variáveis com arquivos CSV, evitando que todos os usuários virtuais façam requisições idênticas.
- Criar cenários progressivos, iniciando com cargas menores e aumentando gradualmente para evitar sobrecargas inesperadas.

Seguindo essas práticas, os testes se tornam mais confiáveis, representativos e úteis para tomada de decisões.

Considerações Finais

O Apache JMeter se mostra uma ferramenta extremamente útil para avaliar o desempenho de sistemas, especialmente em cenários onde há grande volume de acessos simultâneos. Mesmo com sua natureza técnica, seus conceitos podem ser compreendidos de forma clara quando apresentados adequadamente,

permitindo que diferentes perfis de usuários, desde iniciantes até profissionais, aproveitem seus recursos.

Ao simular diversos cenários de carga, o JMeter possibilita identificar limitações que poderiam comprometer a experiência dos usuários em produção. Essa análise preventiva contribui diretamente para a estabilidade, a segurança e a eficiência das aplicações. Somado ao fato de ser gratuito e amplamente utilizado pelo mercado, o JMeter se destaca como uma solução acessível e confiável para instituições que desejam aprimorar a qualidade de seus sistemas.

Além dos recursos básicos apresentados neste trabalho, a ferramenta possui inúmeras possibilidades de expansão. Como continuidade, podem ser explorados testes mais avançados, integrações com outras ferramentas do ecossistema de qualidade e estratégias de automação de cenários complexos de desempenho, ampliando ainda mais o potencial de uso da ferramenta.

Referências Bibliográficas

MEIER, J. D.; FARRE, C.; BANSODE, P.; BARBER, S.; REA, D. Performance Testing Guidance for Web Applications: Patterns & Practices. Microsoft Press, 2007. Disponível em: <https://www.onlineprogrammingbooks.com/free-ebook-performance-testing-guidance-for-web-applications/>. Acesso em: 23 nov. 2025.

THE APACHE SOFTWARE FOUNDATION. Apache JMeter – Official Documentation. 2024. Disponível em: <https://jmeter.apache.org/>. Acesso em: 25 nov. 2025.

TOMLINSON, M. Performance Testing Keynotes and Statements. 2015. Disponível em: <https://www.performancetesting.org/>. Acesso em: 18 nov. 2025.

SOUDERS, S. High Performance Web Sites. O'Reilly Media, 2007.

BARBER, S. Performance Testing Lessons Learned. 2014. Disponível em: <https://www.satisfice.com/download/performance-lessons-barber.pdf>. Acesso em: 20 nov. 2025.

Introdução aos Testes Unitários em Sistemas Embarcados

Thiago Cesar Alvarez
thiago.alvarez2012@hotmail.com
thiago.alvarez@fatec.sp.gov.br

Resumo

Este capítulo explora a engenharia de testes automatizados aplicada a sistemas embarcados, contrastando ferramentas convencionais com soluções especializadas como Unity e Ceedling. O texto detalha como superar as barreiras físicas impostas pela eletrônica — como latência e limitações de memória — através de estratégias de dual-targeting, simulação via mocks e validação Hardware-in-the-Loop (HIL). Por fim, estabelece-se a importância crítica dos testes como garantia de segurança operacional e viabilidade econômica no ciclo de vida do firmware.

Palavras-chaves: Embarcado. C. CMock. Firmware.

Introdução

O Cenário de Testes: Diferenças Fundamentais de Ambiente

Para iniciarmos o estudo sobre testes automatizados em sistemas embarcados, é crucial entender que o ambiente de desenvolvimento de firmware é intrinsecamente diferente do desenvolvimento de software convencional para computadores ou web. Quando um programador utiliza ferramentas clássicas de mercado, como o JUnit para a linguagem Java, ele está operando em um cenário onde o código de teste e o código da aplicação rodam exatamente na mesma máquina, usufruindo de gigabytes de memória RAM e de um sistema operacional completo, como Windows ou Linux, que gerencia tudo nos bastidores. No entanto, no mundo embarcado, escrevemos código em um computador potente (chamado de Host), mas esse código é destinado a rodar em um pequeno microcontrolador (chamado de Target), que muitas vezes tem recursos extremamente limitados. O teste unitário embarcado surge, então, com um desafio extra: precisamos validar a lógica do nosso software antes mesmo de enviá-lo para o chip. Ferramentas específicas para esse nicho, como o framework Unity, foram criadas justamente para serem leves o suficiente para rodar tanto no seu computador durante o desenvolvimento quanto, futuramente, dentro do próprio microcontrolador, sem ocupar toda a memória disponível. Enquanto o JUnit assume que você tem um computador inteiro à disposição, o Unity é construído em linguagem C pura, minimalista, garantindo que você

consiga testar uma função simples de cálculo ou uma máquina de estados sem precisar de toda a complexidade de um sistema operacional.

A Estrutura das Ferramentas: Unity e Ceedling vs Frameworks Desktop

Ao compararmos as ferramentas na prática, percebemos que a arquitetura delas reflete as necessidades de cada área. Em softwares de alto nível, é comum que a ferramenta de teste já venha integrada ao ambiente de desenvolvimento com dezenas de funcionalidades automáticas. Já no embarcado, a construção dos testes é um pouco mais manual e "construída peça por peça", o que oferece maior controle. Aqui, destaca-se o conjunto de ferramentas gerenciado pelo Ceedling. O Ceedling funciona como um "maestro": ele utiliza o Unity para verificar se o seu código está correto (fazendo as afirmações, ou assertions, como "verifique se a soma de $A + B$ é igual a C ") e o CMock para simular partes do sistema. A grande diferença para ferramentas como o PyTest (do Python) ou NUnit (.NET) é que o Ceedling precisa compilar o seu código C de uma forma especial para que ele rode no seu computador como se fosse um programa comum de PC. Isso permite que você teste a lógica do seu firmware — como protocolos de comunicação ou lógica de controle — em questão de segundos, sem precisar gravar o código no chip físico a cada pequena alteração. Essa agilidade é o grande trunfo do teste unitário: separar o que é "lógica de software" do que é "problema de hardware".

Simulando o Hardware: O Conceito de Mocks Simplificado

Talvez a maior dúvida de quem começa em embarcados seja: "Como vou testar meu código se ele precisa ler um sensor ou acionar um motor, e eu estou rodando o teste apenas no meu computador?" É aqui que entra a principal diferença em relação aos testes de software web. Em um sistema web, muitas vezes simulamos um banco de dados. No embarcado, simulamos o hardware. Utilizamos uma ferramenta chamada CMock para criar "dublês" das peças físicas. Imagine que seu código tem uma função chamada `LigarMotor()`. Se você rodar isso no seu computador, daria erro, pois seu PC não tem um motor conectado. O CMock cria automaticamente uma versão falsa dessa função `LigarMotor()`. Quando seu teste roda, em vez de tentar enviar energia elétrica para um motor real, o código chama essa função falsa que apenas registra: "Opa, fui chamada uma vez". Assim, você consegue verificar se o seu software tentou ligar o motor na hora certa, validando a lógica do seu código sem precisar do hardware físico na mesa. Essa técnica é essencial e muito mais frequente em embarcados do que em outras áreas, pois nos liberta da dependência da eletrônica durante a fase de escrita do código.

Os Desafios Impostos pela Eletrônica e pelo Mundo Físico

A Fronteira entre o Código e a Realidade Física

A transição dos testes puramente lógicos para a validação do sistema completo nos confronta com a maior barreira do desenvolvimento embarcado: a dependência inescapável da eletrônica. Diferente de um software de contabilidade, onde os dados de entrada são números digitados por um humano ou lidos de um arquivo, o firmware "sente" o mundo através de tensões elétricas, correntes e pulsos magnéticos. A primeira grande dificuldade enfrentada é a natureza imprevisível desses sinais. Em um ambiente de teste ideal, esperamos que uma entrada seja binária — ligado ou desligado, zero ou um. No entanto, na placa de circuito impresso real, um botão pressionado não gera um sinal limpo instantaneamente; ele gera um fenômeno chamado "bouncing", onde o contato elétrico trepida e envia dezenas de sinais falsos em milissegundos. Testar se o software lida corretamente com esse ruído físico é extremamente difícil de fazer apenas com simulações no computador. Além disso, sensores analógicos — como os de temperatura ou pressão — nunca entregam valores perfeitos devido ao ruído eletromagnético do ambiente. O desenvolvedor se vê obrigado a criar testes que não apenas verifiquem se a lógica funciona com números exatos, mas se o sistema é robusto o suficiente para não falhar quando a eletrônica entrega dados "sujos" ou oscilantes, uma complexidade que raramente existe no mundo do software puramente digital.

O Fator Tempo: Latência e Interrupções

Além da variabilidade dos sinais, o engenheiro de testes embarcados luta contra o relógio, mas não o relógio de parede e sim o tempo do processador. A eletrônica impõe restrições de tempo real que são difíceis de replicar em um ambiente controlado. Em muitos sistemas, o software não decide quando vai trabalhar; é o hardware que "interrompe" o processador para avisar que algo aconteceu, como a chegada de um dado pela porta serial ou o estouro de um temporizador. Essas interrupções ocorrem de forma assíncrona, ou seja, podem parar a execução do código principal a qualquer momento. Criar um teste automatizado que garanta que o sistema não vai travar se uma interrupção ocorrer exatamente no meio de um cálculo crítico é um dos maiores desafios da área. No desenvolvimento web, um atraso de 200 milissegundos no carregamento de uma página é apenas um inconveniente; em um controle de motor ou freio ABS, esse mesmo atraso devido a um teste mal planejado ou uma falha de sincronismo eletrônico pode significar a queima de um componente ou uma falha catastrófica. Portanto, a dificuldade não está apenas em saber se o código produz a resposta certa, mas se ele produz a resposta certa no tempo exato exigido pelo hardware.

A Intrusividade e a Limitação de Recursos

Por fim, enfrentamos o problema físico da "intrusividade" dos testes. Em computadores modernos, adicionar bibliotecas de teste ao código final tem impacto quase nulo, dada a abundância de memória. No entanto, microcontroladores possuem recursos finitos de memória Flash (para guardar o código) e RAM (para rodar as variáveis). Uma das dificuldades mais frustrantes no ambiente embarcado é que o simples ato de adicionar código de teste ou ferramentas de monitoramento pode alterar o comportamento do sistema ou, pior, fazer com que o programa não caiba mais na memória do chip. Isso cria uma situação paradoxal conhecida como "Heisenbug" (uma analogia ao princípio da incerteza), onde o erro desaparece ou muda de comportamento quando tentamos observá-lo. Se o engenheiro tentar enviar logs de teste via porta serial para o computador, isso consome tempo de processamento; se esse tempo for excessivo, o sistema pode perder o sincronismo com um sensor externo, causando uma falha que só existe porque estamos testando. Assim, o desafio final é conseguir observar e validar o funcionamento da eletrônica e do software sem que a própria ferramenta de teste interfira na operação delicada do hardware, exigindo um equilíbrio técnico muito apurado.

Estratégias e Métodos de Execução dos Testes

A Pirâmide de Testes: Do Computador à Bancada

Diante das dificuldades impostas pelo hardware, a engenharia de sistemas embarcados desenvolveu uma abordagem em camadas para mitigar riscos, onde cada etapa oferece um nível diferente de fidelidade e velocidade. A forma mais imediata e econômica de realizar testes é a execução no ambiente Host, ou seja, no próprio computador do desenvolvedor. Nesta etapa, utilizamos a técnica de dual-targeting mencionada anteriormente para compilar as regras de negócio do firmware (cálculos, máquinas de estado, processamento de protocolos) e executá-las localmente. Embora essa abordagem não consiga validar temporizações elétricas reais, ela é extremamente rápida, permitindo rodar milhares de cenários lógicos em segundos. É a primeira linha de defesa: se a lógica matemática falhar aqui, certamente falhará no microcontrolador. No entanto, para avançar na fidelidade sem ainda depender do hardware físico, recorre-se frequentemente à emulação de conjunto de instruções, utilizando ferramentas como o QEMU ou simuladores proprietários fornecidos pelos fabricantes de chips. Diferente do teste no Host, a emulação tenta replicar o comportamento do processador alvo virtualmente, permitindo testar instruções de assembly específicas e o layout de memória do dispositivo, sendo uma excelente opção para integração contínua em servidores na nuvem onde não há placas conectadas.

O Padrão Ouro: Hardware-in-the-Loop (HIL)

Quando a validação lógica e a emulação já não são suficientes para garantir a segurança do sistema, entra em cena a metodologia de testes mais robusta e característica deste setor: o Hardware-in-the-Loop (HIL). Nesta configuração, o teste deixa de ser apenas software e torna-se um sistema físico. O microcontrolador com o firmware final é colocado em uma bancada e conectado a outros instrumentos ou placas de controle que simulam o mundo real. Em vez de um ser humano apertar botões ou aquecer um sensor, um hardware de teste automatizado injeta sinais elétricos nas entradas do dispositivo e monitora as saídas com osciloscópios ou analisadores lógicos digitais. Por exemplo, para testar um controlador de ar-condicionado, o sistema HIL pode gerar uma tensão que simula um aumento de temperatura e verificar se, em resposta, o microcontrolador acionou o relé do compressor no tempo correto. Essa abordagem é a única capaz de validar a integração completa entre software e eletrônica, capturando falhas sutis de comunicação, problemas de ruído e erros que só ocorrem quando o sistema está operando em tempo real. Embora seja mais custoso e complexo de montar, o HIL oferece a garantia final de que o produto funcionará conforme o esperado no campo.

Conclusão: A Importância da Qualidade no Firmware

O Custo da Falha e a Natureza Imutável do Firmware

Ao encerrarmos esta introdução aos testes automatizados para sistemas embarcados, é imperativo refletir sobre o "porquê" de todo esse esforço. A justificativa primordial para a adoção rigorosa de testes reside na própria natureza do firmware: ele é um software "firme", difícil de alterar depois de embarcado. Diferente de uma aplicação web ou de um aplicativo de celular, onde um erro (bug) pode ser corrigido com uma atualização no servidor ou um patch enviado silenciosamente em questão de horas, uma falha em um sistema embarcado pode ter consequências logísticas e financeiras devastadoras. Imagine um erro no código de freio de um automóvel ou no termostato de uma incubadora neonatal; a falha aqui não resulta apenas em uma tela de erro inconveniente, mas pode causar danos físicos, perda de equipamentos caros ou risco à vida humana. Mesmo em produtos de consumo, como cafeteiras ou roteadores, um erro grave descoberto após a fabricação pode exigir um recall físico dos produtos, custando milhões à empresa e destruindo sua reputação no mercado.

Testes como Garantia de Evolução Sustentável

Além da segurança e da economia, os testes automatizados são a chave para a sustentabilidade do desenvolvimento a longo prazo. Projetos embarcados tendem a ter ciclos de vida longos, durando anos ou décadas. Sem uma suíte de testes que garanta que o sistema funciona como deveria, os desenvolvedores desenvolvem um medo paralisante de alterar códigos antigos ("código legado"), receando que uma melhoria em uma função quebre algo em outra parte do sistema que ninguém mais lembra como funciona. Os testes automatizados funcionam, portanto, como uma documentação viva e executável do projeto. Eles permitem que a equipe realize melhorias, refatorações e adicione novas funcionalidades com a confiança de que, se algo sair do esperado, o sistema de testes avisará imediatamente. Utilizar testes em sistemas embarcados não é, portanto, um luxo ou um excesso de zelo acadêmico, mas sim uma ferramenta indispensável de engenharia que transforma a "arte" de programar chips em um processo industrial previsível, seguro e profissional. É o investimento que garante que a tecnologia invisível que controla o nosso mundo continue operando de forma confiável e segura.

Bibliografia

FATEC ARARAS. Faculdade de Tecnologia de Araras. Disponível em: <https://fatecararas.cps.sp.gov.br/>. Acesso em: 11 nov. 2024.

PYTEST. Documentation. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 11 nov. 2024.

SARAIVA JUNIOR, Orlando. ebook_qualidade_teste_software. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software. Acesso em: 11 nov.

Grenning, James. *Test Driven Development for Embedded C*. The Pragmatic Programmers, 2011. (Livro fundamental sobre TDD e a técnica de *Dual-Targeting*).

Throw The Switch. Documentação do Ceedling, Unity & CMock. Disponível em: <http://www.throwtheswitch.org/>. Acesso em: 2024.

CppUTest. Framework de testes unitários e *mocking* para C/C++. Disponível em: <https://cpputest.github.io/>. Acesso em: 2024.

Google. Guia do Usuário do GoogleTest (GTest). Disponível em: <https://google.github.io/googletest/>. Acesso em: 2024.

QEMU. Emulador e Virtualizador de Processadores de Código Aberto. Disponível em: <https://www.qemu.org/>. Acesso em: 2024.

Fowler, Martin. *Mocks Aren't Stubs* (Mocks não são Stubs). (Artigo fundamental sobre a teoria de Mocks). Disponível em: <https://martinfowler.com/articles/mocksArentStubs.html>. Acesso em: 2024.

Embedded.com. Estratégias de teste de software para sistemas embarcados. Disponível em: <https://www.embedded.com/>. Acesso em: 2024.

Renode. Framework de Desenvolvimento Virtual da Antmicro para Sistemas Embarcados Complexos (Alternativa moderna ao QEMU). Disponível em: <https://renode.io/>. Acesso em: 2024.

Ferramentas de Teste para Projetos em Dart e Flutter

Maria Vitoria Suzarth
maria.suzarth@hotmail.com
maria.suzarth@fatec.sp.gov.br

Resumo

Este eBook apresenta uma abordagem objetiva e prática sobre as principais ferramentas de teste utilizadas no desenvolvimento de aplicações com Dart e Flutter, destacando especialmente os testes unitários e os testes de widget, recursos nativos fundamentais para garantir a qualidade do software. O material explora a importância dos testes dentro do ciclo de desenvolvimento, demonstrando como essas ferramentas contribuem para a detecção precoce de falhas, melhoria na manutenção do código e aumento da confiabilidade do produto final. São apresentados exemplos simples e aplicáveis, permitindo ao leitor reproduzir os testes sugeridos e compreender sua utilidade no contexto real de desenvolvimento mobile. Além disso, o eBook evidencia a relevância da automação e da análise contínua, ressaltando como práticas estruturadas de teste fortalecem a estabilidade do sistema e auxiliam na entrega de aplicações mais robustas. Assim, o conteúdo busca orientar estudantes e profissionais a adotarem estratégias de teste como parte essencial do processo de desenvolvimento em Flutter, promovendo maior eficiência, segurança e qualidade ao software produzido.

Palavras-chaves: Flutter, Dart, Testes unitários. Teste de Widget, Qualidade de Software, Automação.

Introdução

O desenvolvimento de software moderno exige cada vez mais práticas que assegurem a confiabilidade, estabilidade e qualidade das aplicações. No contexto do Flutter, framework que ganhou destaque pela sua eficiência na criação de interfaces multiplataforma, o uso de testes estruturados tornou-se parte essencial do ciclo de desenvolvimento. As ferramentas nativas disponibilizadas pelo ecossistema Flutter como os testes unitários e os testes de widget oferecem aos desenvolvedores os

recursos necessários para validar comportamentos, lógica interna e componentes visuais de forma rápida e precisa.

Além de proporcionarem maior segurança na evolução do código, essas práticas contribuem para a prevenção de falhas, facilitam a manutenção contínua e promovem maior confiança na entrega de novas funcionalidades. A possibilidade de testar tanto regras de negócio quanto elementos da interface sem a necessidade de dispositivos físicos torna o processo mais ágil e eficiente, especialmente em equipes que trabalham com integração contínua.

Este eBook tem como objetivo apresentar essas ferramentas, demonstrando sua importância e aplicação prática, oferecendo exemplos claros para auxiliar o leitor a compreender e implementar testes em seus próprios projetos Flutter. O intuito é reforçar que a adoção de testes não apenas melhora a qualidade técnica, mas também impulsiona a maturidade e a profissionalização do desenvolvimento.

Objetivos

Este eBook tem como objetivo apresentar e analisar as principais ferramentas nativas de teste disponíveis no ecossistema Flutter, destacando sua relevância na garantia da qualidade de software. Busca-se demonstrar como os testes unitários e os testes de widget contribuem para a validação da lógica interna, da interface e do comportamento da aplicação, tornando o desenvolvimento mais seguro, eficiente e estruturado. Além disso, pretende-se orientar o leitor na execução prática desses testes, fornecendo exemplos claros que podem ser reproduzidos em projetos reais. O material visa também reforçar a importância da adoção de práticas de teste no cotidiano do desenvolvimento mobile, incentivando estudantes e profissionais a incorporarem essas técnicas como parte essencial do processo de construção e manutenção de aplicações Flutter.

Desenvolvimento

Ferramentas principais de Teste no Flutter

Nesta seção serão apresentadas duas ferramentas fundamentais do ecossistema Flutter para garantir a qualidade de aplicações: Testes Unitários e Testes de Widget.

Testes Unitários (package:test)

Os testes unitários representam a base da pirâmide de testes em Flutter. Utilizando o pacote oficial test, eles permitem validar funções, classes e comportamentos isolados da lógica da aplicação, assegurando que o código funciona como o esperado.

Aplicações

A Testagem Unitária tem como objetivo central isolar comportamento sem dependências externas para focar na qualidade da menor parte testável do código. Isso permite validar funções puras e testar classes de lógica e regras de negócio, garantindo de forma eficiente que métodos individuais retornem valores esperados sob condições controladas.

Vantagens

O desenvolvimento de testes unitários resulta em um processo leve e rápido, pois eles são projetados para rodar em milissegundos, permitindo feedback imediato sobre a saúde do código. Essa característica de velocidade e leveza faz com que a integração seja facilitada, promovendo a integração fácil com pipelines de CI/CD (Integração Contínua/Entrega Contínua), onde os testes são executados automaticamente a cada mudança de código para garantir que nenhuma regressão seja introduzida. Por fim, a maioria das ferramentas e frameworks de testes unitários desfrutam de suporte oficial e documentação ampla, o que facilita a adoção, a resolução de problemas e o aprendizado por parte das equipes de desenvolvimento.

calculadora.dart

Figura 1 - calculadora.dart

```
// lib/utils/calculadora.dart

/// Função que recebe dois inteiros e retorna a soma deles.
int somar(int a, int b) {
    return a + b;
}

/// Exemplo adicional:
/// Função que recebe dois doubles e retorna a diferença entre eles.
double subtrair(double a, double b) {
    return a - b;
}
```

Fonte: próprio autor

Figura 2 - main.dart

```
// main.dart

import 'package:/lib/utils/calculadora.dart';

void main() {
    // Usando a função somar
    int resultadoSoma = somar(15, 7);
    print('O resultado da soma é: $resultadoSoma'); // Saída: 22

    // Usando a função subtrair
    double resultadoSubtracao = subtrair(25.5, 10.2);
    print('O resultado da subtração é: $resultadoSubtracao'); // Saída: 15.3
}
```

Fonte: próprio autor

O arquivo calculadora.dart foi criado com o objetivo de demonstrar o uso de funções simples com tipagem forte e documentação na linguagem Dart, permitindo compreender melhor como a linguagem trata dados, organiza responsabilidades e incentiva boas práticas de desenvolvimento.

Esse módulo apresenta duas funções básicas que são somar e subtrair cada uma utilizando tipos diferentes para reforçar o conceito de tipagem estática. A função somar recebeu dois valores inteiros e retornou um resultado também inteiro, mostrando uma aplicação direta e objetiva. Já a função subtrair utilizou valores de ponto flutuante, retornando um resultado do tipo double, o que ajudou a entender o tratamento de números não inteiros no Dart. Durante o desenvolvimento desse módulo, pude consolidar conhecimentos essenciais sobre os fundamentos da linguagem. Compreendi melhor a sintaxe das funções, a obrigatoriedade de declarar o tipo dos parâmetros e dos retornos, e a estrutura de um arquivo de biblioteca dentro da

pasta utils. O processo de criação desse módulo contribuiu significativamente para o entendimento de boas práticas, como clareza na nomenclatura, responsabilidade única e preocupação com a manutenibilidade. Mesmo sendo simples, o `calculadora.dart` foi uma peça crucial para aprimorar meus conhecimentos em Dart e fortalecer minha base para projetos maiores e mais complexos.

Aplicação

Os Testes de Widget no ambiente Flutter/Dart, apoiados pela biblioteca `flutter_test`, possuem três principais Aplicações. Eles são projetados para validar a estrutura visual da tela, assegurando que o layout do aplicativo esteja montado corretamente. Complementarmente, a testagem de widgets serve para garantir que textos, botões e elementos renderizam como esperado, confirmando a presença e a correta exibição de todos os componentes de interface definidos. Por fim, uma aplicação essencial é testar mudanças de estado e interações, verificando como a interface do usuário reage e se atualiza após simulações de eventos como cliques ou digitações.

Benefícios

Essas aplicações trazem diversos Benefícios importantes para o processo de desenvolvimento. O primeiro é que a execução dos testes não requer dispositivo físico, permitindo que sejam realizados em um ambiente simulado em qualquer máquina de desenvolvimento. Além disso, a sua natureza leve e focada em componentes individuais garante uma execução rápida, o que fornece feedback imediato aos desenvolvedores sobre qualquer falha introduzida no código. Por último, o framework de teste permite simular toques, digitação e navegação em um widget, oferecendo um alto grau de fidelidade à experiência real do usuário sem a necessidade de um dispositivo real, tornando o processo de debugging e validação mais eficiente.

Figura 3 - `main.dart`

```

import 'package:flutter/material.dart';
import 'contador.dart';

void main() {
  runApp(const MeuAppContador());
}

class MeuAppContador extends StatelessWidget {
  const MeuAppContador({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Aplicativo Contador',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      // Define a tela inicial
      home: const TelaContador(),
    );
  }
}

class TelaContador extends StatelessWidget {
  const TelaContador({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Meu Contador Flutter'),
        centerTitle: true,
      ),
      body: const Center(
        child: Contador(),
      ),
    );
  }
}

```

Fonte: próprio autor

Figura 4 - contador.dart

```

import 'package:flutter/material.dart';

class Contador extends StatefulWidget {
  const Contador({super.key});

  @override
  State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int _valor = 0;

  void _incrementarContador() {
    setState(() {
      _valor++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        const Text(
          'Você apertou o botão:',
          style: TextStyle(fontSize: 18),
        ),
        Text(
          '$_valor',
          style: Theme.of(context).textTheme.headlineLarge,
        ),
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: _incrementarContador,
          child: const Text('Incrementar'),
        ),
      ],
    );
  }
}

```

Fonte: próprio autor

Durante o processo, busquei compreender tanto as características da ferramenta Flutter quanto sua forma de estruturar interfaces e lidar com estado.

O primeiro passo foi analisar o arquivo main.dart, que representa o ponto de partida da aplicação. Ali, identifiquei que o método runApp() é responsável por iniciar toda a interface, e que no Flutter tudo é construído a partir de Widgets.

O projeto utiliza o MaterialApp, que fornece a base visual do Material Design e define elementos gerais, como tema e tela inicial. Em seguida, a tela principal faz uso do Scaffold, que organiza a estrutura básica da interface, contendo a barra superior e o corpo da página. Nessa etapa, compreendi como os Widgets são aninhados e como cada um contribui para a organização da UI.

A parte mais significativa do aprendizado ocorreu ao estudar o arquivo do contador em si. Nele, explorei como o Flutter trabalha com Widgets de estado: o `StatefulWidget` e a classe `State`.

Percebi que o `StatefulWidget` funciona como uma estrutura fixa, responsável apenas por criar a instância do estado. Já a classe `State` é onde ficam os dados que podem mudar no caso, o valor do contador.

O método que modifica esse valor utiliza `setState()`, que foi um conceito central na minha compreensão. Aprendi que, ao alterar o estado por meio desse método, o Flutter entende que precisa reconstruir apenas a parte da tela afetada. Assim, o método `build()` é chamado novamente para exibir o novo valor de forma reativa

Considerações Finais

A aplicação de testes em projetos desenvolvidos com Dart e Flutter é fundamental para assegurar que o software seja confiável, funcional e estável. As ferramentas exploradas neste eBook principalmente os testes unitários e testes de widget, nativos do ecossistema Flutter demonstram como é possível garantir qualidade de maneira simples, consistente e eficiente.

Ao utilizar os recursos fornecidos pelo próprio framework, desenvolvedores conseguem validar lógica, comportamento visual, interações e fluxos do usuário sem depender de ferramentas externas. Essa abordagem reduz custos, facilita manutenção e amplia a confiabilidade do código, especialmente em projetos que crescem com rapidez.

Os testes apresentados, aliados às boas práticas de organização, automação e análise estática, contribuem para a criação de aplicações mais robustas e preparadas para produção. Dessa forma, o uso sistemático de testes deve ser encarado como parte essencial do processo de desenvolvimento em Flutter, fortalecendo tanto a qualidade técnica quanto a experiência final do usuário.

Bibliografia

FLUTTER DOCS (org.). An introduction to unit testing. 1. [S. l.], 2025. Disponível em: <https://docs.flutter.dev/cookbook/testing/unit/introduction>. Acesso em: 25 nov. 2025.

FLUTTER DOCS (org.). Flutter_test library. 1. [S. l.], 2025. Disponível em: https://api.flutter.dev/flutter/flutter_test/. Acesso em: 25 nov. 2025.

FLUTTER DOCS (org.). Check app functionality with an integration test. 1. [S. l.], 2025. Disponível em: <https://docs.flutter.dev/testing/integration-tests>. Acesso em: 25 nov. 2025.

Testes de Carga com k6: Garantindo Performance em Sistemas de Alto Tráfego

Autor: Miran Romeiro

E-mail: miranromeiro@gmail.com

E-mail institucional: miran.silva@fatec.sp.gov.br

Resumo

Este trabalho apresenta a ferramenta k6 como solução para testes de carga em aplicações web modernas que enfrentam picos de demanda. O objetivo principal é demonstrar como o k6 pode ser utilizado para simular cenários realistas de alta concorrência, identificar gargalos de performance e garantir a estabilidade de sistemas críticos. A metodologia envolveu o desenvolvimento de uma API de checkout simulada utilizando FastAPI, containerização com Docker, e execução de testes progressivos que simulam desde situações normais até picos extremos de tráfego. Os resultados demonstraram que a ferramenta é eficaz na identificação de limites de capacidade, permitindo otimizações proativas antes da exposição em ambientes de produção. A abordagem apresentada pode ser aplicada em diversos contextos, incluindo e-commerce em eventos de Black Friday, sistemas de venda de ingressos, lançamentos especiais em plataformas de jogos e operações financeiras de alta frequência.

Palavras-chave: k6. Testes de Carga. Performance. FastAPI. Docker.

1. Introdução

A crescente dependência de sistemas digitais para operações críticas de negócio tornou a performance e disponibilidade requisitos não funcionais essenciais. Eventos como a Black Friday, lançamento de ingressos para shows populares, chegada de novas skins em jogos multiplayer e operações em mercados financeiros geram picos de acesso que podem comprometer completamente a experiência do usuário e resultar em perdas financeiras significativas.

O objetivo geral deste trabalho é apresentar o k6 como ferramenta de teste de carga, demonstrando sua aplicação prática na validação de sistemas que enfrentam alta concorrência. Como objetivos específicos, busca-se: (1) compreender os fundamentos dos testes de carga e sua importância; (2) explorar as funcionalidades principais do k6; (3) desenvolver um ambiente de testes realista utilizando Docker; (4) executar cenários progressivos de carga; e (5) analisar métricas de performance.

A escolha do k6 se justifica por sua arquitetura moderna baseada em JavaScript, facilidade de integração em pipelines CI/CD, capacidade de executar milhares de usuários virtuais com baixo consumo de recursos, e geração de métricas detalhadas em formato compatível com ferramentas de observabilidade modernas como Grafana e Prometheus.

A metodologia adotada incluiu revisão bibliográfica sobre testes de performance, estudo da documentação oficial do k6, desenvolvimento de uma API REST simulando um sistema de checkout, implementação de scripts de teste progressivos e análise dos resultados obtidos. Todo o desenvolvimento foi realizado nas dependências da FATEC utilizando tecnologias open source.

Este trabalho está organizado em quatro seções principais: esta introdução, o desenvolvimento que apresenta os conceitos fundamentais e a jornada de implementação, as considerações finais com conclusões e propostas futuras, e as referências bibliográficas.

2. Desenvolvimento

2.1. Contexto e Importância dos Testes de Carga

Sistemas modernos enfrentam desafios únicos de escalabilidade. Durante eventos promocionais, não é incomum que plataformas de e-commerce experimentem aumentos de 10x a 100x no tráfego normal. A Ticketmaster, por exemplo, precisa lidar com milhões de usuários simultâneos tentando comprar ingressos nos primeiros minutos após um lançamento. Jogos populares como League of Legends ou CS:GO frequentemente lançam skins limitadas que geram corridas massivas de jogadores. No mercado financeiro, sistemas de trading precisam processar milhares de transações por segundo com latência mínima.

Sem testes adequados, esses sistemas podem sofrer:

- **Degradação de performance:** tempos de resposta que vão de milissegundos para dezenas de segundos
- **Timeouts e falhas em cascata:** quando um componente falha, afeta todo o ecossistema
- **Perda de receita:** usuários frustrados abandonam a compra
- **Danos à reputação:** falhas em momentos críticos geram cobertura negativa na mídia

2.2. O que é k6?

O k6 é uma ferramenta de teste de carga moderna e open source desenvolvida pela Grafana Labs. Diferentemente de ferramentas legadas como JMeter, o k6 foi projetado desde o início para integração em ambientes DevOps e cloud native.

Principais características:

- **Scripting em JavaScript:** facilita a adoção por desenvolvedores front-end e full-stack
- **Arquitetura leve:** capaz de gerar alta carga com consumo mínimo de recursos

- **Execução CLI:** ideal para automação e CI/CD
- **Métricas ricas:** coleta automática de latência, throughput, taxa de erro e métricas customizadas
- **Thresholds:** permite definir critérios de sucesso/falha automaticamente
- **Checks:** validações inline durante a execução do teste
- **Stages:** simulação de cenários realistas com ramp-up e ramp-down

2.3. Instalação e Configuração do Ambiente

Todo o projeto foi estruturado utilizando containers Docker para garantir reprodutibilidade e isolamento. A arquitetura consiste em dois serviços principais:

1. **API Service:** aplicação FastAPI simulando um sistema de checkout
2. **k6 Service:** container com a ferramenta k6 e scripts de teste

Figura 1 - Arquitetura do projeto com Docker

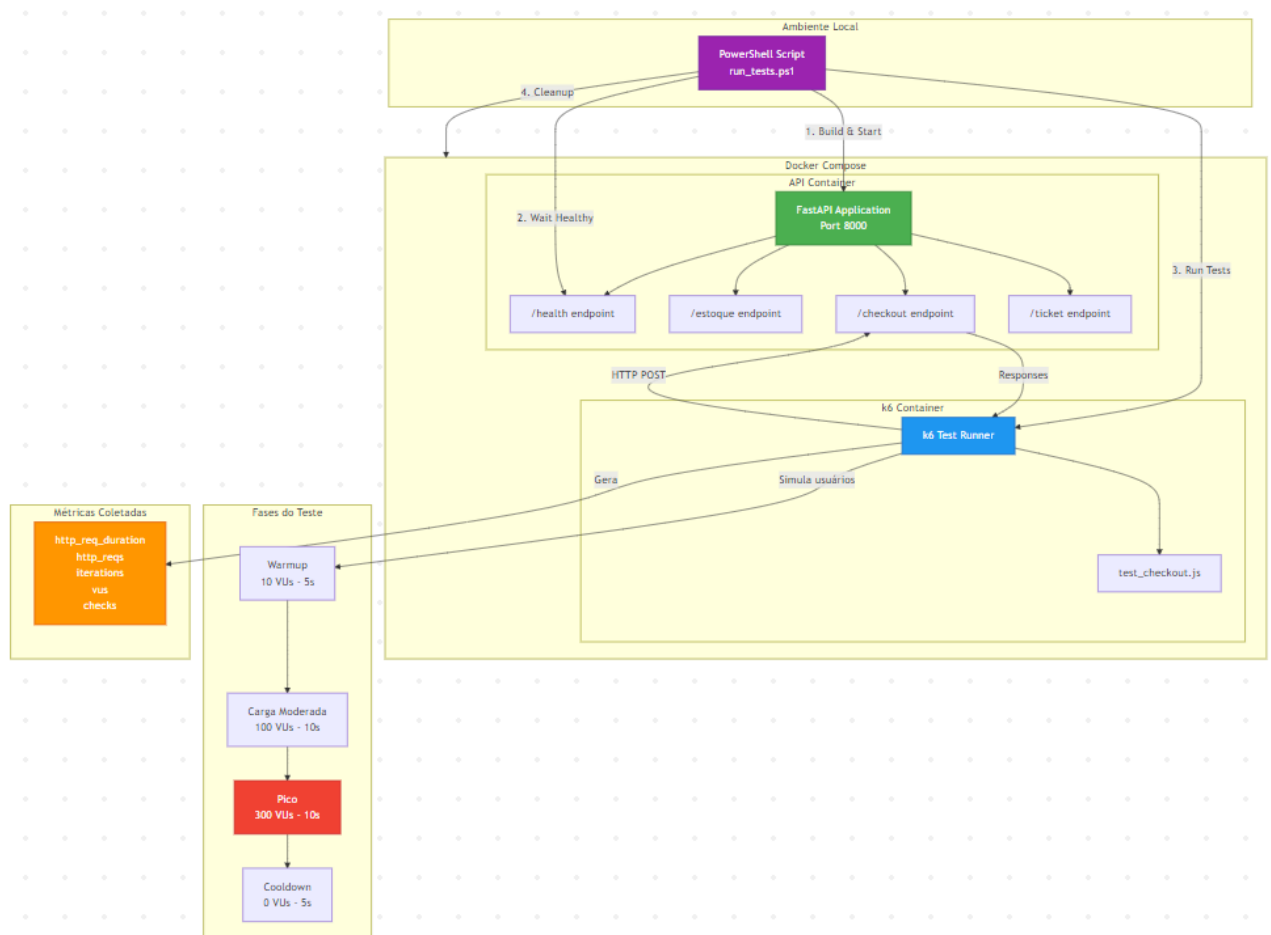


Figura 1 – Arquitetura do ambiente de testes

Fonte: próprio autor

Conforme observado na Figura 2, a separação em containers permite executar os testes de forma consistente em qualquer ambiente, seja local ou em pipelines de CI/CD.

2.3.1. Estrutura do Projeto

O projeto foi organizado seguindo boas práticas de estruturação:

```
|— src/  
|   |— main.py      # Código da API
```

```
src > main.py > ...  
12  
13 @app.get("/health")  
14 def health():  
15     return {"status": "ok"}  
16  
17 @app.get("/events")  
18 def list_events():  
19     """Retorna uma lista simulada de eventos"""  
20     events = [  
21         {"id": 1, "name": "Rock Festival", "price": 250},  
22         {"id": 2, "name": "Jazz Night", "price": 180},  
23         {"id": 3, "name": "Tech Conference", "price": 400},  
24         {"id": 4, "name": "Motor Show", "price": 320},  
25         {"id": 5, "name": "Comic Expo", "price": 200},  
26     ]  
27     return events  
28  
29 @app.post("/checkout")  
30 def checkout(req: CheckoutRequest):  
31     """Simula o processo de checkout"""  
32     # chance de erro de pagamento  
33     if random.random() < 0.1:  
34         raise HTTPException(status_code=400, detail="Payment failed")  
35  
36     # simula tempo de processamento  
37     processing_time = round(random.uniform(0.2, 1.2), 2)  
38     time.sleep(processing_time)  
39     return {  
40         "message": f"Purchase successful for event {req.event_id}",  
41         "delay": processing_time,  
42         "user": req.user  
43     }  
44  
45 @app.get("/tickets/{ticket_id}")  
46 def get_ticket(ticket_id: int):  
47     """Consulta um ticket aleatório"""  
48     if ticket_id % 10 == 0:  
49         raise HTTPException(status_code=404, detail="Ticket not found")  
50     return {"ticket_id": ticket_id, "status": "valid"}  
51
```

└─ tests/
└─ test_checkout.js # Script de teste k6

```
tests > JS test_checkoutjs > ...  
  
1  import http from 'k6/http';  
2  import { sleep, check } from 'k6';  
3  
4  export const options = {  
5    stages: [  
6      { duration: '10s', target: 100 },  
7      { duration: '20s', target: 200 },  
8      { duration: '10s', target: 0 },  
9    ],  
10   thresholds: {  
11     http_req_duration: ['p(95)<800'],  
12     http_req_failed: ['rate<0.05'],  
13   },  
14 };  
15  
16 const BASE_URL = 'http://api:8000';  
17  
18 export default function () {  
19   let events = http.get(`${BASE_URL}/events`);  
20   check(events, { 'Listagem OK': (r) => r.status === 200 });  
21  
22   sleep(0.5);  
23  
24   const eventId = Math.floor(Math.random() * 5) + 1;  
25   const checkout = http.post(  
26     `${BASE_URL}/checkout`,  
27     JSON.stringify({  
28       event_id: eventId,  
29       quantity: Math.floor(Math.random() * 3) + 1,  
30       user: `user${Math.floor(Math.random() * 1000)}@mail.com`,  
31     }),  
32     { headers: { 'Content-Type': 'application/json' } }  
33   );  
34  
35   check(checkout, {  
36     'Checkout OK': (r) => r.status === 200 || r.status === 400,  
37   });  
38  
39   sleep(1);  
40 }  
41
```

2.3.2. Implementação da API de Teste

A API foi desenvolvida em FastAPI para simular um cenário realista de checkout. O sistema gerencia um inventário de produtos (A, B, C) e processa pedidos com as seguintes características:

- **Endpoint /health:** verificação de disponibilidade
- **Endpoint /estoque/{item_id}:** consulta de disponibilidade (latência simulada: 30ms)
- **Endpoint /checkout:** processamento de pedidos (latência simulada: 80ms)
- **Endpoint /ticket:** informações de ticket

A simulação de latência é crucial para tornar o teste mais realista, já que aplicações reais sempre têm delays de processamento, acesso a banco de dados e chamadas externas.

2.4. Desenvolvendo Scripts de Teste com k6

O script de teste foi estruturado para simular um cenário progressivo de carga, representando o comportamento real de usuários acessando o sistema.

2.4.1. Configuração de Stages

Os stages definem como a carga aumenta e diminui ao longo do tempo:

```
stages: [  
  { duration: '5s', target: 10 }, // warmup: 10 usuários  
  { duration: '10s', target: 100 }, // carga moderada: 100 usuários  
  { duration: '10s', target: 300 }, // pico: 300 usuários  
  { duration: '5s', target: 0 }, // cooldown: 0 usuários  
]
```

Esta progressão simula:

1. **Warmup (5s)**: período inicial onde o sistema "aquece" com carga baixa
2. **Carga moderada (10s)**: representa uso normal em horário de pico
3. **Pico extremo (10s)**: simula eventos como lançamento de produto ou promoção relâmpago
4. **Cooldown (5s)**: retorno gradual ao estado normal

2.4.2. Thresholds e Critérios de Sucesso

Thresholds permitem definir objetivos mensuráveis de performance:

```
thresholds: {  
  http_req_duration: ['p(95)<1000'], // 95% das requisições < 1000ms  
}
```

Este threshold estabelece que 95% das requisições devem completar em menos de 1 segundo, um padrão razoável para sistemas de checkout.

2.4.3. Função Principal do Teste

A função executada por cada usuário virtual simula o comportamento real de um cliente:

1. Seleciona aleatoriamente um produto (A, B ou C)
2. Prepara payload JSON com item e quantidade
3. Envia requisição POST para /checkout
4. Valida resposta (sucesso 200 ou esgotado 400)
5. Aguarda 100ms antes da próxima iteração

A aleatoriedade na seleção de produtos distribui a carga entre diferentes recursos, simulando cenários mais realistas.

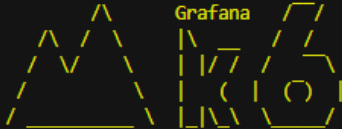
2.5. Execução dos Testes

A execução foi automatizada através do docker, que orquestra todo o processo:

1. **Build e inicialização:** constrói as imagens Docker e sobe o serviço da API
2. **Health check:** aguarda até que a API esteja saudável e pronta
3. **Execução do teste:** roda o container k6 com o script de teste
4. **Limpeza:** remove todos os containers após conclusão

Figura 2 - Terminal executando os testes

```
(.venv) PS C:\Users\miran\OneDrive\Desktop\repos\testes-na-pratica-k6> docker compose run --rm k6
✓ Container testes-na-pratica-api Running
✓ Container influxdb Running



  execution: local
    script: /test_checkout.js
    output: InfluxDBv1 (http://influxdb:8086)

  scenarios: (100.00%) 1 scenario, 200 max VUs, 1m10s max duration (incl. graceful stop):
    * default: Up to 200 looping VUs for 40s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

  running (0m17.1s), 135/200 VUs, 554 complete and 0 interrupted iterations
  default [=====>-----] 135/200 VUs 17.1s/40.0s
```

Figura 2 – Execução automatizada dos testes

Fonte: próprio autor

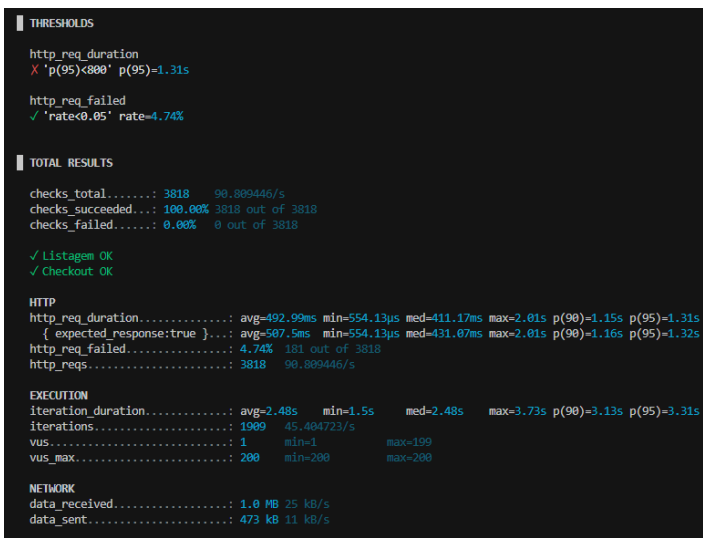
2.6. Análise de Resultados

Durante a execução, o k6 gera métricas em tempo real que permitem avaliar a performance do sistema:

2.6.1. Métricas Principais

- **http_req_duration:** tempo total da requisição (média, mediana, p95, p99)
- **http_req_waiting:** tempo aguardando resposta do servidor
- **http_reqs:** taxa de requisições por segundo
- **iterations:** número total de iterações completadas
- **vus (Virtual Users):** usuários virtuais ativos em cada momento
- **checks:** porcentagem de validações que passaram

Figuras 3 e 4 - Outputs do k6 com métricas



Figuras 3 e 4 – Métricas de performance geradas pelo k6 no console e no painel do Grafana
Fonte: próprio autor

2.6.2. Identificação de Gargalos

A análise das métricas revelou:

1. **Latência crescente:** o percentil 95 ultrapassou o threshold de 1.31s durante o pico
2. **Esgotamento de estoque:** produtos com menor inventário (item C com 20 unidades) esgotaram rapidamente

3. **Capacidade máxima:** a API conseguiu sustentar ~100 usuários simultâneos confortavelmente, mas degradou com 300

Estes insights são fundamentais para decisões de arquitetura, como implementação de cache, otimização de queries ou escalabilidade horizontal.

2.7. Aplicações em Cenários Reais

2.7.1. E-commerce e Black Friday

Durante a Black Friday, e-commerces podem experimentar aumentos de 50x no tráfego. Testes com k6 permitem:

- Validar se a infraestrutura aguenta o pico esperado
- Identificar o ponto de quebra do sistema
- Otimizar queries e cache antes do evento
- Definir limites de taxa (rate limiting) adequados

2.7.2. Venda de Ingressos (Ticketmaster)

Quando ingressos para shows populares são lançados, milhões de pessoas acessam simultaneamente. O k6 ajuda a:

- Simular filas virtuais
- Testar sistemas de CAPTCHA sob carga
- Validar mecanismos anti-bot
- Garantir justiça no acesso (primeiros que chegam são atendidos)

2.7.3. Lançamentos em Jogos

Jogos como Fortnite, League of Legends e CS:GO frequentemente lançam skins ou itens colecionáveis que movimentam o mercado. Testes ajudam a:

- Evitar crashes de servidor durante lançamentos
- Validar sistemas de microtransação
- Garantir experiência fluida mesmo com alta demanda
- Prevenir exploits de duplicação de itens

2.7.4. Mercado Financeiro

Sistemas de trading precisam de latência ultrabaixa (< 10ms) e alta disponibilidade. O k6 permite:

- Validar capacidade de processamento de ordens
- Testar cenários de alta volatilidade
- Garantir consistência em transações concorrentes
- Simular condições de mercado extremas

2.8. Integração em CI/CD

Um dos maiores benefícios do k6 é sua facilidade de integração em pipelines:

```
# Exemplo de pipeline GitLab CI
performance-test:
  stage: test
  script:
    - docker compose up -d api
    - docker compose run k6 run test_checkout.js
  only:
    - main
```

Isso permite executar testes de performance automaticamente a cada deploy, detectando regressões antes da produção.

2.9. Jornada de Aprendizado

O processo de aprendizado da ferramenta k6 envolveu várias etapas:

1. **Estudo da documentação oficial:** compreensão dos conceitos fundamentais
2. **Experimentos simples:** testes básicos com HTTP GET/POST
3. **Incremento de complexidade:** adição de stages, checks e thresholds
4. **Containerização:** isolamento do ambiente com Docker
5. **Automação:** criação de scripts para execução repetível
6. **Análise crítica:** interpretação de métricas e identificação de melhorias

A curva de aprendizado foi facilitada pela familiaridade com JavaScript e pelo excelente material disponível na documentação oficial do k6 e na comunidade.

3. Considerações Finais

Este trabalho demonstrou a aplicabilidade da ferramenta k6 para testes de carga em sistemas que enfrentam picos de demanda. A implementação de um ambiente completo com API simulada, containerização e automação permitiu experimentar cenários realistas e compreender os desafios de performance em aplicações modernas.

Os resultados obtidos evidenciaram que testes de carga não são apenas recomendados, mas essenciais para sistemas críticos. A capacidade de identificar gargalos antes da produção pode evitar prejuízos financeiros significativos e proteger a reputação da marca.

Como trabalhos futuros, sugere-se: (1) integração do k6 com pipeline; (2) implementação de testes distribuídos usando k6 Cloud para simular cargas ainda maiores; (3) desenvolvimento

de cenários mais complexos envolvendo múltiplos endpoints e fluxos de usuário; (4) análise comparativa com outras ferramentas como Locust e Gatling; e (5) estudo de técnicas de otimização de performance baseadas nos insights obtidos.

A experiência prática com k6 revelou uma ferramenta madura, bem documentada e adequada tanto para desenvolvedores quanto para equipes de QA, consolidando-se como opção preferencial para testes de performance em ambientes DevOps modernos.

Bibliografia

GRAFANA LABS. **k6 Documentation**. Disponível em: <https://k6.io/docs/>.

K6. **Testing guide**. Disponível em: <https://grafana.com/docs/k6/latest/testing-guides/>.

FASTAPI. **FastAPI Documentation**. Disponível em: <https://fastapi.tiangolo.com/>.

DOCKER. **Docker Documentation**. Disponível em: <https://docs.docker.com/>.

SARAIVA JUNIOR, Orlando. **ebook_qualidade_teste_software**. GitHub. Disponível em: https://github.com/orlandosaraivajr/ebook_qualidade_teste_software.

BEYER, Betsy et al. **Site Reliability Engineering: How Google Runs Production Systems**. O'Reilly Media, 2016.

dbt (Data Build Tool) aplicado à qualidade de dados em pipelines analíticos

Valdir Garcia

valdijc2002@gmail.com

valdir.garcia@fatec.sp.gov.br

Sumário

RESUMO..... 3

INTRODUÇÃO..... 4

DESENVOLVIMENTO.....5

CONSIDERAÇÕES FINAIS..... 6

REFERÊNCIAS..... 7

RESUMO

Este e-book apresenta uma análise introdutória e aplicada do dbt (data build tool) como ferramenta de apoio à qualidade de dados em pipelines analíticos. A partir de uma pesquisa bibliográfica sobre engenharia de dados, qualidade de dados e boas práticas de desenvolvimento de software, são descritos os principais recursos do dbt relacionados à modularização de modelos SQL, controle de versionamento, documentação automática e execução de testes automatizados. A metodologia adotada combina estudo de referências especializadas com experimentação da ferramenta em ambiente de laboratório, simulando cenários comuns em projetos de Business Intelligence. Como resultado, o trabalho sistematiza conceitos como camadas de modelagem (staging, intermediária e mart), tipos de testes de dados (singularidade, não nulidade, integridade referencial e valores aceitos) e o uso de macros e arquivos YAML para padronizar a implementação. O material busca oferecer ao leitor uma visão prática de como o dbt contribui para tornar pipelines analíticos mais confiáveis, auditáveis e fáceis de manter, aproximando a engenharia de dados de princípios consolidados de qualidade de software.

Palavras-chave: dbt. Engenharia de dados. Qualidade de dados. Testes automatizados. Pipelines analíticos.

INTRODUÇÃO

A crescente demanda por soluções orientadas a dados, impulsionada por organizações que buscam tomar decisões rápidas e embasadas, tornou a qualidade dos dados um elemento central na engenharia e na análise modernas (REIS; HOUSLEY, 2023). Nesse contexto, este trabalho aborda o uso do dbt (data build tool) como ferramenta destinada a estruturar, padronizar e testar pipelines analíticos, oferecendo uma abordagem sistemática para garantir a confiabilidade das transformações de dados e a rastreabilidade das fontes (MARQUES, 2024). O tema é delimitado ao estudo dos mecanismos de testes automatizados, versionamento, modelagem modular e documentação integrada disponíveis no dbt, aplicados principalmente a cenários de engenharia de dados voltados para Business Intelligence e analytics.

O objetivo geral deste e-book é apresentar uma compreensão sólida do funcionamento do dbt sob a perspectiva da qualidade de software e da qualidade de dados (MOSES; GAVISH; VORWERCK, 2024). Como objetivos específicos, propõe-se: descrever seus principais componentes; demonstrar, de forma conceitual, a aplicação de testes de dados; discutir boas práticas de modelagem e documentação; e evidenciar como o dbt se integra a um ciclo de desenvolvimento orientado a testes. A escolha do tema justifica-se pela expansão do uso de pipelines analíticos em ambientes de negócio e pela necessidade de profissionais capazes de aplicar técnicas de validação de dados de maneira estruturada e reproduzível. A metodologia adotada envolve pesquisa bibliográfica, análise documental da ferramenta e experimentação em ambiente de laboratório. Por fim, o texto organiza-se em quatro partes: introdução, desenvolvimento técnico, considerações finais e referências.

DESENVOLVIMENTO

O dbt destaca-se no ecossistema moderno de engenharia de dados por incorporar práticas clássicas da engenharia de software — como modularização, versionamento, revisão de código, testes automatizados e documentação — ao ciclo de transformação analítica de dados (REIS; HOUSLEY, 2023; THOUGHTWORKS, [s.d.]). Em sua essência, a ferramenta permite definir modelos SQL versionados em repositórios Git, organizados em camadas lógicas de *staging*, camada intermediária e camada de apresentação (*marts*), o que facilita a rastreabilidade entre tabelas e a compreensão da linhagem de dados (DATACAMP, 2024).

Um dos pilares do dbt é seu mecanismo de testes de dados, que disponibiliza testes genéricos — como *unique*, *not_null*, *relationships* e *accepted_values* — além da possibilidade de criar testes personalizados em SQL ou em macros. Esses testes podem ser executados a cada execução do pipeline, validando integridade referencial, consistência de regras de negócio e conformidade de domínios de valores, alinhando a ferramenta às recomendações de literatura recente sobre qualidade e confiabilidade de dados (MOSES; GAVISH; VORWERCK, 2024). A detecção precoce de quebras e anomalias reduz retrabalho e evita a propagação de dados incorretos para dashboards e relatórios estratégicos.

Outro recurso central é a documentação automática gerada a partir de arquivos YAML, nos quais são descritos modelos, colunas, testes aplicados e exemplos de uso. A partir desses metadados, o dbt constrói uma interface web navegável que registra dependências, descrições funcionais e histórico de execução, favorecendo a transparência entre times de engenharia de dados, analytics e negócio (SANTOS, 2022). Essa documentação, aliada ao uso disciplinado de revisão de código, facilita a entrada de novos membros na equipe e reduz a dependência de conhecimento tácito, aspecto frequentemente citado em relatos de adoção da ferramenta em empresas de tecnologia.

Do ponto de vista da jornada de aprendizado, o estudo da ferramenta iniciou-se com a instalação do dbt em ambiente controlado e a criação de um projeto simples, contendo poucos modelos e testes básicos. Em seguida, foram exploradas boas práticas de modelagem em camadas, parametrização via macros e organização de pastas, tomando como referência materiais introdutórios e estudos de caso disponíveis em português, incluindo experiências relatadas em contexto acadêmico (MARQUES, 2024) e corporativo (DATACAMP, 2024; SANTOS, 2022). Ao longo desse processo, observou-se que a curva de aprendizado da ferramenta é suavizada quando se parte de exemplos concretos e se adota uma disciplina de pequenos incrementos: cria-se um modelo, escreve-se um teste, executa-se o pipeline e analisa-se o resultado. Essa dinâmica reforça a cultura de qualidade de dados e aproxima a prática de engenharia de dados do que se espera de projetos de software profissionais.

CONSIDERAÇÕES FINAIS

As discussões desenvolvidas ao longo deste e-book evidenciam que o dbt se consolidou como uma peça central na engenharia de dados moderna quando o objetivo é elevar a qualidade de pipelines analíticos. A combinação de modelagem em camadas, controle de versionamento, documentação integrada e testes automatizados permite reduzir significativamente falhas em produção, aumentar a confiança das áreas de negócio e encurtar o ciclo entre desenvolvimento e correção de problemas. Observou-se que a adoção de boas práticas herdadas da engenharia de software — como revisão de código, uso de Git, padronização de nomenclaturas e desenvolvimento orientado a testes — torna o trabalho com dados mais previsível e colaborativo, aproximando a atuação de analistas, engenheiros e profissionais de BI.

Como perspectivas futuras, recomenda-se aprofundar o uso do dbt em conjunto com ferramentas de observabilidade e monitoramento de dados, de modo a identificar incidentes de qualidade de forma pró-ativa (MOSES; GAVISH; VORWERCK, 2024). Também se sugere explorar integrações com orquestradores e plataformas de nuvem, bem como estudar padrões de projeto específicos para *analytics engineering*, à luz de guias modernos de arquitetura de dados (REIS; HOUSLEY, 2023; THOUGHTWORKS, [s.d.]). Dessa forma, o dbt pode ser não apenas uma ferramenta de transformação, mas o núcleo de um ecossistema analítico mais robusto, governado e alinhado às necessidades estratégicas das organizações.

REFERÊNCIAS

DATA CAMP. **O que é dbt? Uma introdução prática para engenheiros de dados.** 2024. Disponível em: <<https://www.datacamp.com/pt/tutorial/what-is-dbt>>. Acesso em: 1 dez. 2025. [DataCamp](#)

MARQUES, Arthur Gabriel Camacho. **Engenharia de dados com DBT: entendendo conceito e recursos da ferramenta.** 2024. Trabalho de Conclusão de Curso (Curso Superior de Tecnologia em Informática para Negócios) – Faculdade de Tecnologia de São José do Rio Preto, São José do Rio Preto, 2024. ric.cps.sp.gov.br

MOSES, Barr; GAVISH, Lior; VORWERCK, Molly. **Fundamentos da qualidade de dados: guia prático para criar pipelines de dados confiáveis.** Rio de Janeiro: Alta Books, 2024. [Estante Virtual+2Google Play+2](#)

REIS, Joe; HOUSLEY, Matt. **Fundamentos de engenharia de dados: projete e construa sistemas de dados robustos.** São Paulo: Novatec, 2023. novatec.com.br+2Mercado Livre+2

SANTOS, Bianca. **dbt: uma ferramenta para ETL e ELT.** Florianópolis: Indicium, 2022. Disponível em: <<https://www.indicium.tech/pt-br/blog/dbt-uma-ferramenta-para-etl-e-elt>>. Acesso em: 1 dez. 2025. [Indicium](#)

THOUGHTWORKS. **Manual para uma engenharia de dados moderna.** [S.l.]: Thoughtworks, [s.d.]. E-book. Disponível em: <<https://www.thoughtworks.com/pt-br/insights/e-books/modern-data-engineering-playbook>>. Acesso em: 1 dez. 2025. [Thoughtworks](#)

A ferramenta PestPHP

Davi Bonelli Rodrigues

davi33013@gmail.com

davi.rodrigues7@fatec.sp.gov.br

Resumo

O PestPHP é um framework de testes moderno desenvolvido para tornar o processo de escrita e execução de testes em PHP mais simples, expressivo e produtivo. Este capítulo apresenta um estudo detalhado da ferramenta, analisando suas características, vantagens, limitações e aplicabilidade no contexto do desenvolvimento de software. A metodologia adotada envolveu pesquisa bibliográfica, consulta à documentação oficial, análise comparativa com o PHPUnit — ferramenta tradicional do ecossistema PHP — e a implementação de exemplos práticos para observação de sua usabilidade. Os resultados evidenciam que o PestPHP proporciona ganhos significativos em legibilidade, velocidade de escrita e experiência do desenvolvedor (DX), além de manter compatibilidade total com o PHPUnit. Conclui-se que, embora ainda seja uma ferramenta relativamente nova, o PestPHP representa uma alternativa robusta e moderna para projetos que valorizam qualidade, clareza e manutenção eficiente do código.

Palavras-chave: PestPHP. Testes. PHP. TDD.

Introdução

O presente capítulo tem como objetivo apresentar um estudo aprofundado sobre o PestPHP, um framework de testes para PHP criado em 2020 por Nuno Maduro. A ferramenta vem ganhando destaque no ecossistema PHP devido à sua abordagem minimalista, sintaxe elegante e foco em proporcionar uma experiência de desenvolvimento moderna.

O objetivo geral deste trabalho é compreender o funcionamento do PestPHP e seu papel no desenvolvimento de testes automatizados. Os objetivos específicos incluem:

- analisar sua arquitetura e sua relação com o PHPUnit;
- identificar seus pontos fortes e fracos;
- compreender sua instalação e uso na prática;
- comparar sua sintaxe com a de ferramentas tradicionais;
- refletir sobre sua aplicação em diferentes tipos de testes.

A escolha do tema se justifica pela crescente adoção do PestPHP em projetos contemporâneos e pela necessidade de ferramentas mais amigáveis e produtivas para o processo de testes. A metodologia aplicada envolveu estudo teórico, análise comparativa e a criação de exemplos práticos que permitiram observar o comportamento da ferramenta em situações reais.

Este capítulo está estruturado em seções que abordam a descrição da ferramenta, sua análise técnica, exemplos práticos e considerações finais sobre sua importância no desenvolvimento moderno em PHP.

Desenvolvimento



Figura 1-Logotipo oficial do PestPHP

O PestPHP é um framework inspirado em ferramentas modernas de teste, como Jest (JavaScript), e foi projetado para simplificar significativamente a escrita de testes em PHP. Construído sobre a base sólida do PHPUnit, o Pest não reinventa a roda: ele aproveita toda a estrutura do PHPUnit, mas oferece uma camada de sintaxe mais expressiva e minimalista.

Sua principal característica é o uso de *closures* (funções anônimas) em vez de classes, eliminando a necessidade de estruturas verbosas e repetitivas. Isso torna o código mais limpo, intuitivo e fácil de manter. Além disso, sua execução gera relatórios claros e coloridos no terminal, facilitando a identificação rápida de falhas nos testes.

```

FAIL Tests\Unit\CalculatorTest
✓ soma dois números positivos
✓ soma com número negativo
✓ subtração funciona corretamente
✓ multiplicação funciona corretamente
✓ divisão funciona corretamente
✓ divisão por zero lança exceção
✓ potência funciona corretamente
✓ verifica se número é par with (2, true)
✓ verifica se número é par with (4, true)
✓ verifica se número é par with (6, true)
✓ verifica se número é par with (1, false)
✓ verifica se número é par with (3, false)
✓ verifica se número é par with (5, false)
✓ verifica se número é par with (0, true)
✓ calcula média de números
✓ média com array vazio lança exceção
✓ operações matemáticas básicas

```

Figura 2-Saída de testes executados com PestPHP sucesso

```

FAIL Tests\Unit\ValidatorTest
✓ Validação de CPF → aceita CPF válido com pontuação 0.02s
✓ Validação de CPF → aceita CPF válido sem pontuação
✓ Validação de CPF → rejeita CPF com menos de 11 dígitos
✓ Validação de CPF → rejeita CPF com todos dígitos iguais
✓ Validação de Senha Forte → aceita senha forte válida
✓ Validação de Senha Forte → rejeita senha muito curta
✓ Validação de Senha Forte → rejeita senha sem maiúscula
✓ Validação de Senha Forte → rejeita senha sem minúscula
✓ Validação de Senha Forte → rejeita senha sem números
✓ Validação de URL → valida URLs válidas with ('https://www.google.com')
✓ Validação de URL → valida URLs válidas with ('http://example.com')
✓ Validação de URL → valida URLs válidas with ('https://site.com.br/pagina')
✓ Validação de URL → valida URLs válidas with ('ftp://files.example.com')
✓ Validação de URL → rejeita URLs inválidas with ('not-a-url')
✓ Validação de URL → rejeita URLs inválidas with ('google.com')
✗ Validação de URL → rejeita URLs inválidas with ('http://wrong.com') 0.01s
✓ Validação de URL → rejeita URLs inválidas with ('')
✓ valida idade dentro do range
✓ rejeita idade fora do range
✓ valida idade com range customizado

```

Figura 3-Saída de testes executados com PestPHP falha

Instalação e Configuração do PestPHP

Para iniciar o uso do PestPHP, é necessário possuir o **PHP 8.1 ou superior** e o **Composer** instalados. O processo de instalação é simples e baseado em três etapas principais.

Realiza a instalação:

```
COMPOSER REQUIRE PESTPHP/PEST --DEV
```

Inicia o Pest:

```
./VENDOR/BIN/PEST --INIT
```

Executa os testes:

```
./VENDOR/BIN/PEST
```

Figura 4-Etapas da Instalação do PestPHP

Análise Técnica da Ferramenta

Durante o estudo, observou-se que os principais pontos fortes do Pest são:

- **Sintaxe simples e elegante:** reduz linhas de código e melhora a legibilidade;
- **Compatibilidade total com PHPUnit:** permite adoção gradual em sistemas já existentes;
- **Fácil configuração:** poucos comandos são necessários para iniciar;
- **Relatórios legíveis:** saída colorida no terminal facilita a leitura;
- **Adequado para times modernos:** integra-se facilmente com frameworks como Laravel e Symfony.

No entanto, também foram identificadas limitações:

- ferramenta relativamente nova, ainda pouco utilizada em ambientes conservadores;
- paradigma funcional pode causar estranhamento inicial;
- dependência direta do PHPUnit, que impacta seu funcionamento;

Exemplo Prático Comparativo

A diferença de verbosidade entre PHPUnit e PestPHP evidencia o principal benefício da ferramenta:

```
use PHPUnit\Framework\TestCase;

class CalculadoraTest extends TestCase {
    public function testSoma() {
        $calc = new Calculadora();
        $this->assertEquals(4, $calc->soma(2, 2));
    }
}
```

Figura 5-Sintaxe PHPUnit

```
test('soma dois números', function () {
    $calc = new Calculadora();
    expect($calc->soma(2, 2))->toBe(4);
});
```

Figura 6-Sintaxe PestPHP

A experiência prática demonstrou que, além de testes unitários, o Pest é igualmente eficiente na realização de **testes de integração** e **testes funcionais**, reforçando sua versatilidade.

Considerações Finais

Com base no estudo realizado, conclui-se que o PestPHP representa uma evolução significativa nas ferramentas de testes para PHP. Sua proposta minimalista e elegante torna a escrita de testes mais ágil, produtiva e acessível. A

compatibilidade total com o PHPUnit garante uma transição suave e segura para equipes que já utilizam a ferramenta tradicional.

Embora ainda seja uma solução recente e com menor volume de documentação, sua comunidade cresce rapidamente, impulsionada pela adoção em projetos modernos e pela busca por ferramentas que priorizam a experiência do desenvolvedor.

Para trabalhos futuros, recomenda-se aprofundar a aplicação do PestPHP em ambientes corporativos, explorar o desenvolvimento de *plugins* e investigar seu uso em arquiteturas complexas. Assim, o PestPHP consolida-se como uma alternativa sólida e promissora para projetos que buscam clareza, simplicidade e alta qualidade nos testes automatizados.

Bibliografia

PHPUNIT. Documentation. Disponível em: <https://phpunit.de>.

PESTPHP. Documentation. Disponível em: <https://pestphp.com>.

Figura 1 Disponível em: <https://laravel-news.com/pestphp-released-as-open-source>.

Figuras 2-6: Próprio autor.